APACHE SPARK

Logo of Spark?

1. Star component
2. Lightning Speed
3. Five faces of star –
   5 components & 5 prog lang
4. Map Reduce -> Spark



All about Apache Spark

Our Learning Roadmap Journey

Stage1:

a. DWH(Legacy)
b. SQL (upto speed)
c. Python (Core prog) + AI

Stage2 (DE/DBDE/AIE/AIDE):

a. PySpark (SQL + Py + DWH-M)
b. Databricks (PySpark+)
c. AI ()

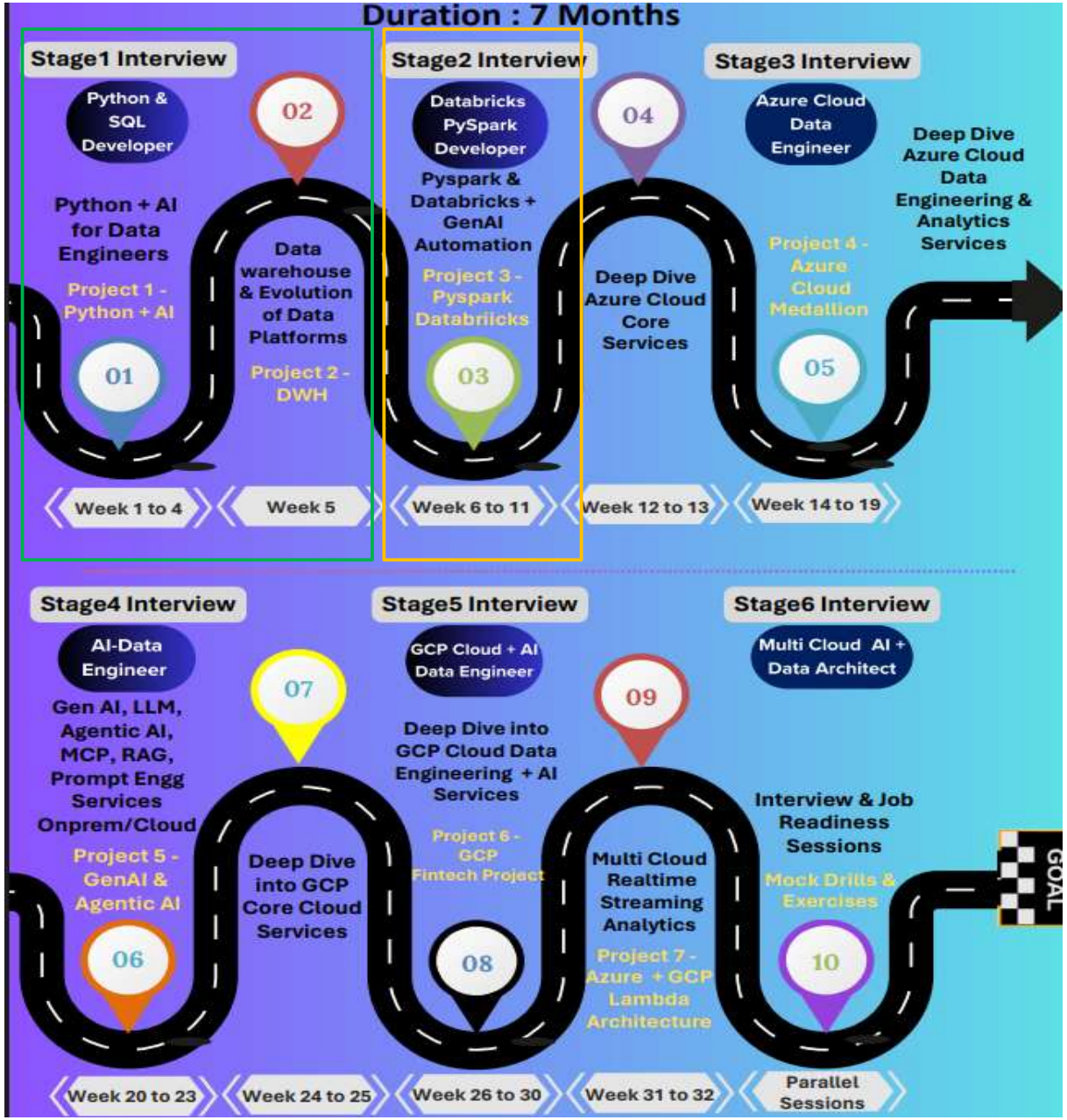
Stage3+4*(AzCDE/AzCAIDE):

a. Azure Cloud (DBC)

Stage5(GCPDE/GCPAIDE):

a. GCP Cloud (BQ)

FDE/

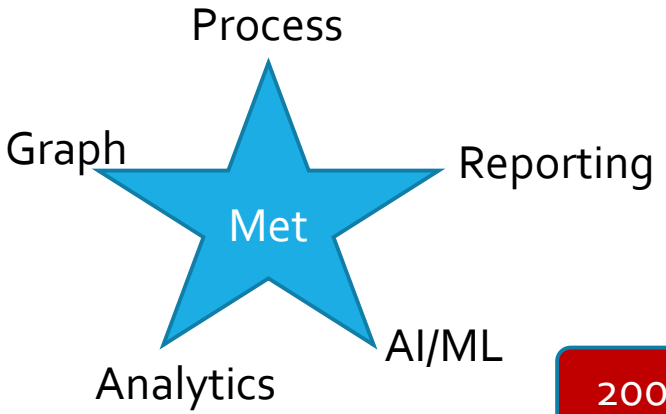


1. All About Spark

1. Spark Introduction (definition) – Distributed + In-Memory + Framework
2. Prerequisites (Platform/Terminologies, Interface, PL, FW)
3. History
4. Facts and Fantasies
5. Components & APIs(Functions)
6. Why Spark

Introduction to Apache Spark (Distributed Inmemory Data Processing Open Source FW)

Spark is a **Lightning fast, Unified** & **Distributed** (**Inmemory**) computation **Apache** Open Source **framework**, that performs large scale Data **processing** & **analytics** (leveraging the power of RAM) running on the **Cluster** of Computers.

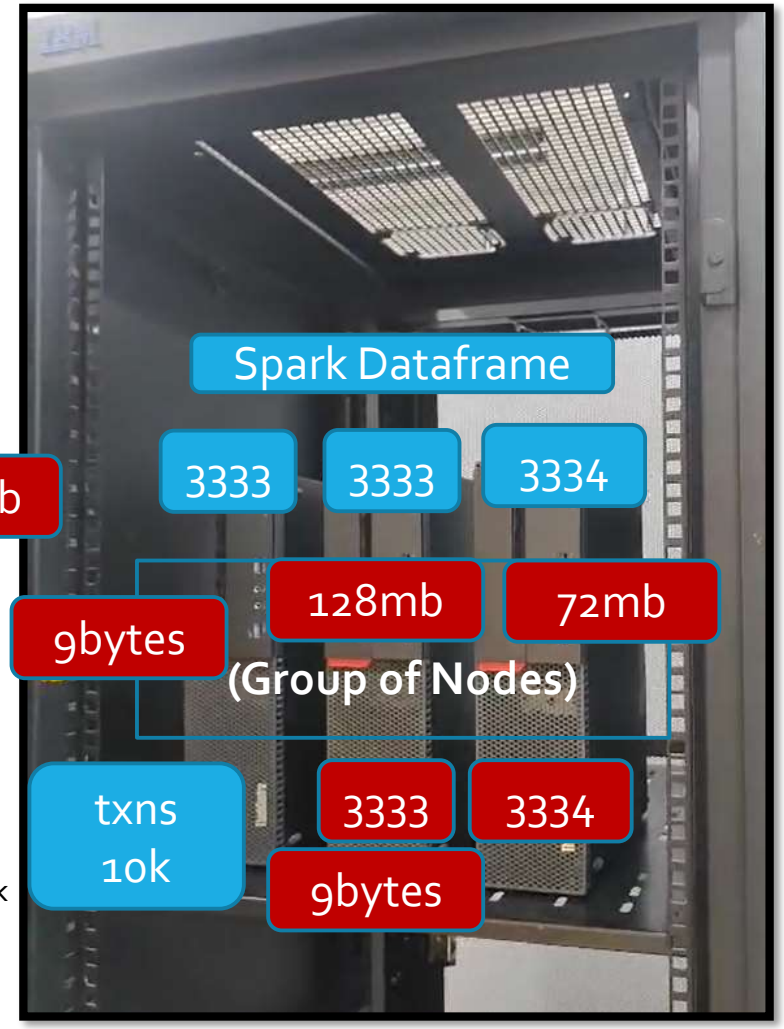


Distributed = Delegation/Split/Partition

In-Memory = Use more of RAM for processing, rather than disk

Framework = Library + Protocol, if we leverage by following the rules, we can get solution for our problem

Problem: Managing 3vs (Bigdata)
Solution : leveraging & following the rule of Hadoop & Spark

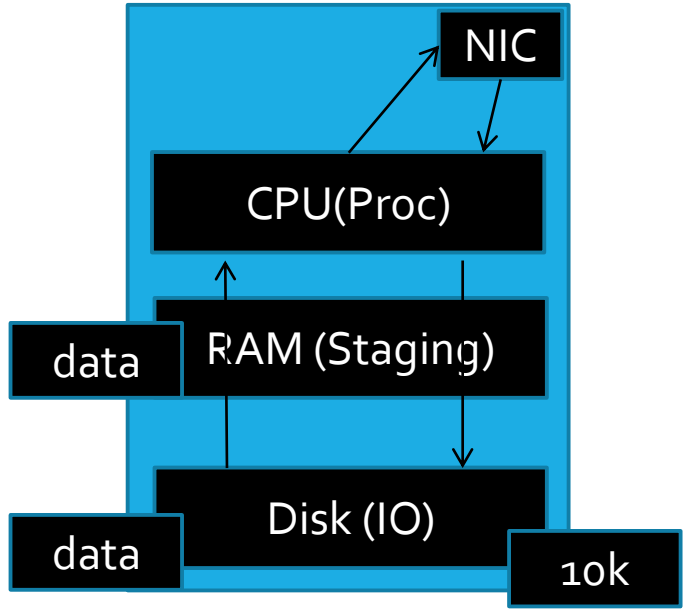



```
Single_Df=pd.read_csv("path")

from pyspark.sql.session import SparkSession
spark=SparkSession()
Dist_Df=spark.read.csv("path")
Dist_Df.count()
```

Id			
1			
2			
3			

data



Prerequisites before we enter into Spark (Sparkers)

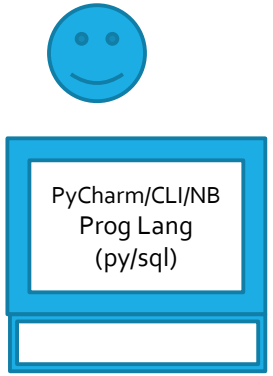
Infrastructure ? OnPrem/Cloud

Platform? OS (Win/Linux) & Supporting Tools (DBC/Dataproc/EMR/DF/HDI)

Software/Framework? Distributed Inmemory Proc FW (Spark) + Distributed Storage FW (Hadoop) -
Rules defined in a form of OOPS+functions, We can get a solution for a problem (3vs data store/proc), If we leverage a framework.

Interfaces? CLI(REPL), WebUI, Notebook, IDE

Programming language? Python + SQL





What we have to do to become a Sparker?

1. Pillars of Spark
2. Components of Spark
3. Architecture
4. Performance Optimization
5. End to End Spark Application (Data pipeline/Analytical/AI)

Cluster

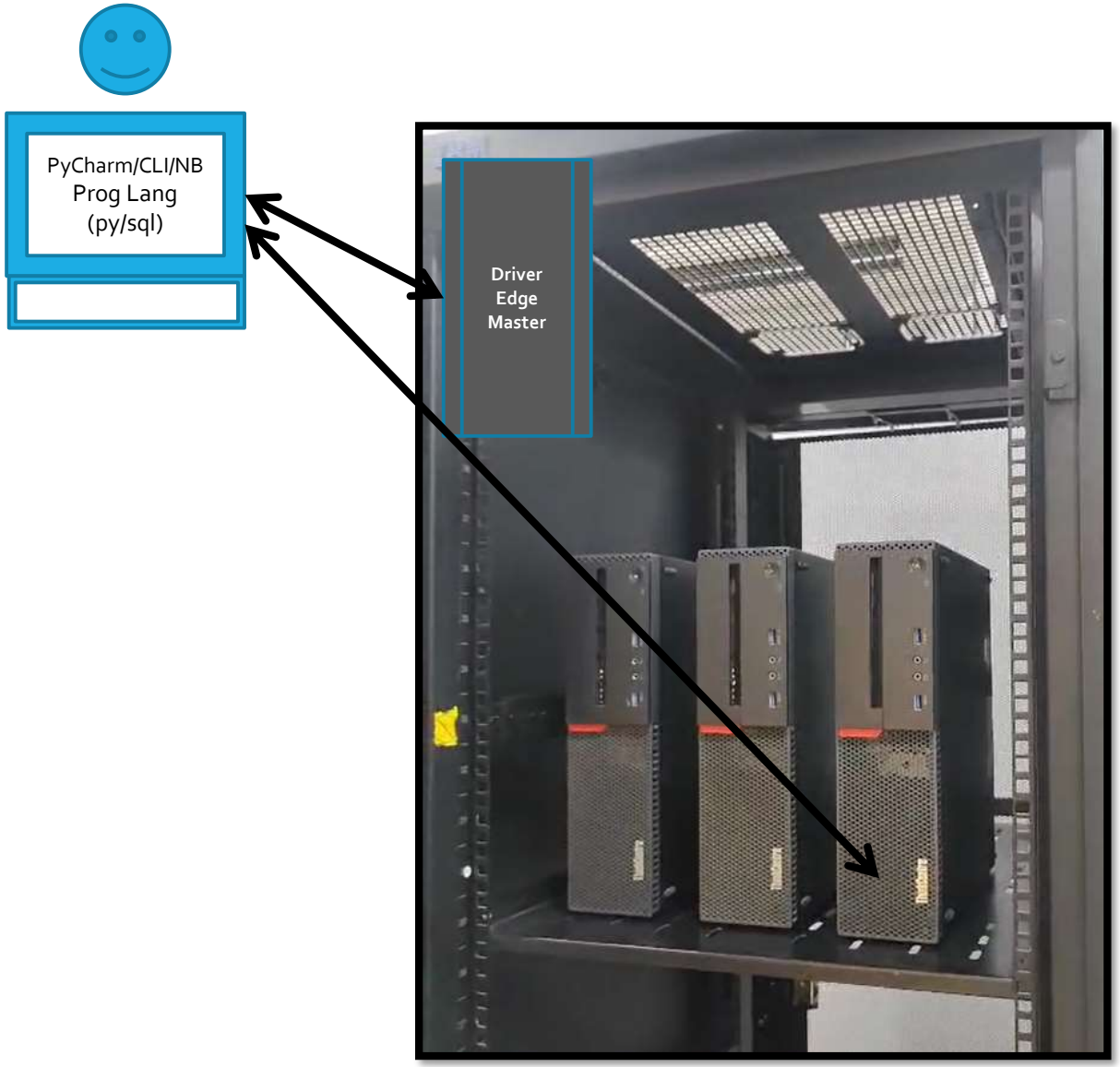
Infrastructure ? OnPrem/Cloud

Platform? OS (Win/Linux) & Supporting Tools (DBC/Dataproc/EMR/DF/HDI)

Software/Framework? Distributed Inmemory Proc FW (Spark) + Distributed Storage FW (Hadoop) -
Rules defined in a form of OOPS+functions, We can get a solution for a problem (3vs data store/proc), If we leverage a framework.

Interfaces? CLI(REPL), WebUI, Notebook, IDE

Programming language? Python + SQL

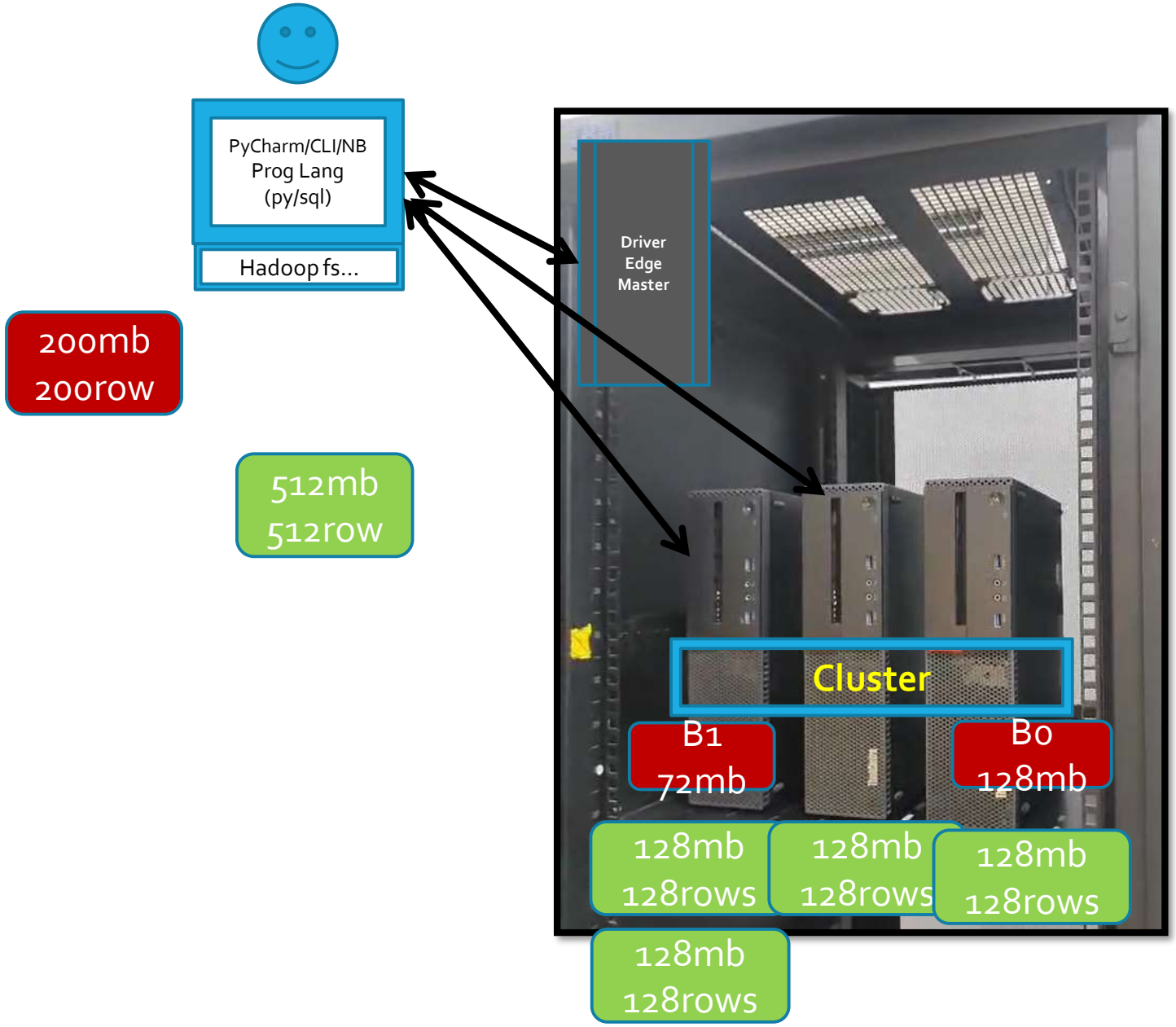


Distributed

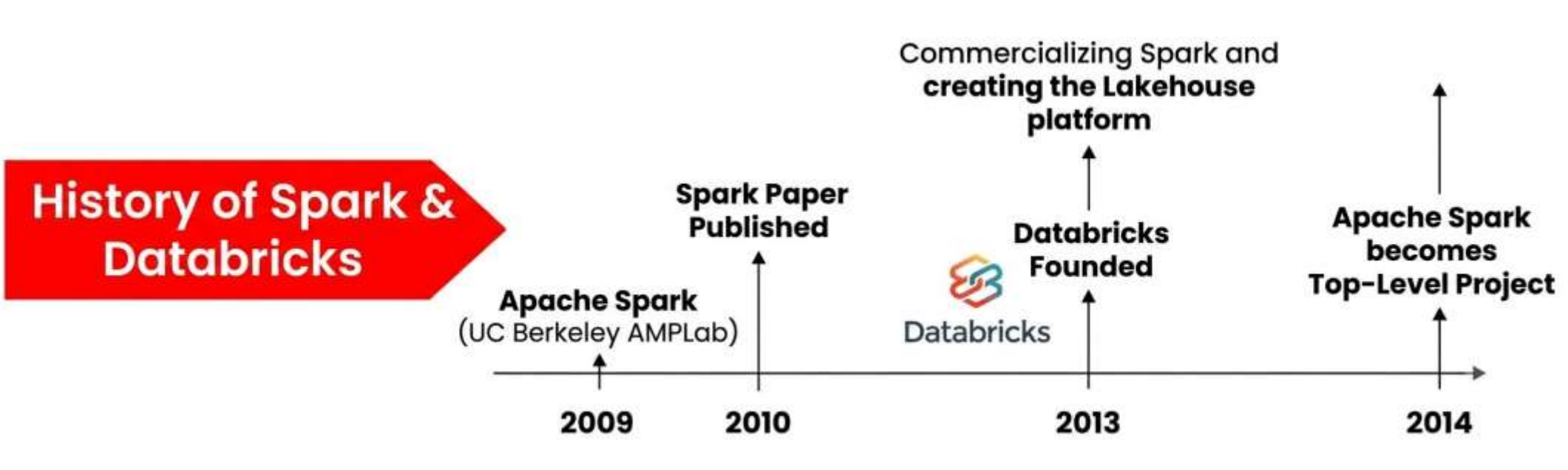
Hadoop Distributed Storage:

Splitting data into multiple blocks and storing across multiple nodes

CBP (Command Based Programming)
hadoop fs –cp gs://… hdfs://



Brief History



Introduction to Apache Spark – Quick Facts

https://www.openhub.net/p/apache-spark

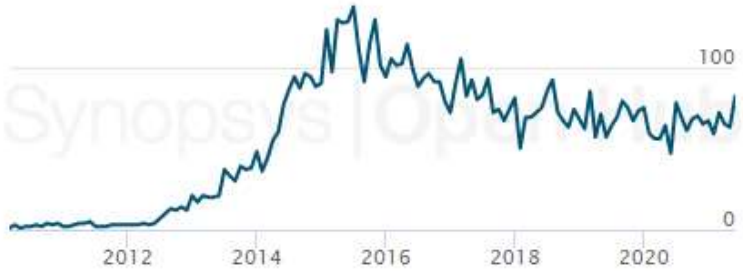
Lines of Code



Commits per Month



Contributors per Month



Languages



Scala	70%	Python	14%
Java	9%	11 Other	7%

30 Day Summary

Jun 9 2021 – Jul 9 2021

386 Commits

79 Contributors

including 20 new contributors

12 Month Summary

Jul 9 2020 – Jul 9 2021

3122 Commits

Up + 205 (7%) from previous 12 months

352 Contributors

Up + 26 (7%) from previous 12 months

Most Recent Contributors

- Takuya UESHIN
- Hyukjin Kwon
- Yikun Jiang
- Xinrong Meng
- ulysses-you
- Angerszhuuuu

Ratings

8 users rate this project: 5.0/5.0

Click to add your rating

Review this Project!

INCEPTEZ TECHNOLOGIES

Introduction to Apache Spark – LOC & Contributors



Apache Spark

Settings | Report Duplicate



Very High Activity

56

I Use This!

Analyzed 7 days ago. based on code collected 7 days ago.

Languages

Total Lines :	1,589,877	Code Lines :	1,024,217	Percent Code Lines :	64.4%
Number of Languages :	14	Total Comment Lines :	365,333	Percent Comment Lines :	23.0%
		Total Blank Lines :	200,327	Percent Blank Lines :	12.6%

Analyzed about 11 hours ago. based on code collected about 12 hours ago.

Commits

	All Time	12 Month	30 Day
Commits:	30670	3122	386
Contributors:	2354	352	79
Files Modified:	29961	5596	1514
Lines Added:	8235769	431256	36887
Lines Removed	6618192	334845	16461

INCEPTEZ TECHNOLOGIES

Introduction to Apache Spark – Languages Used %



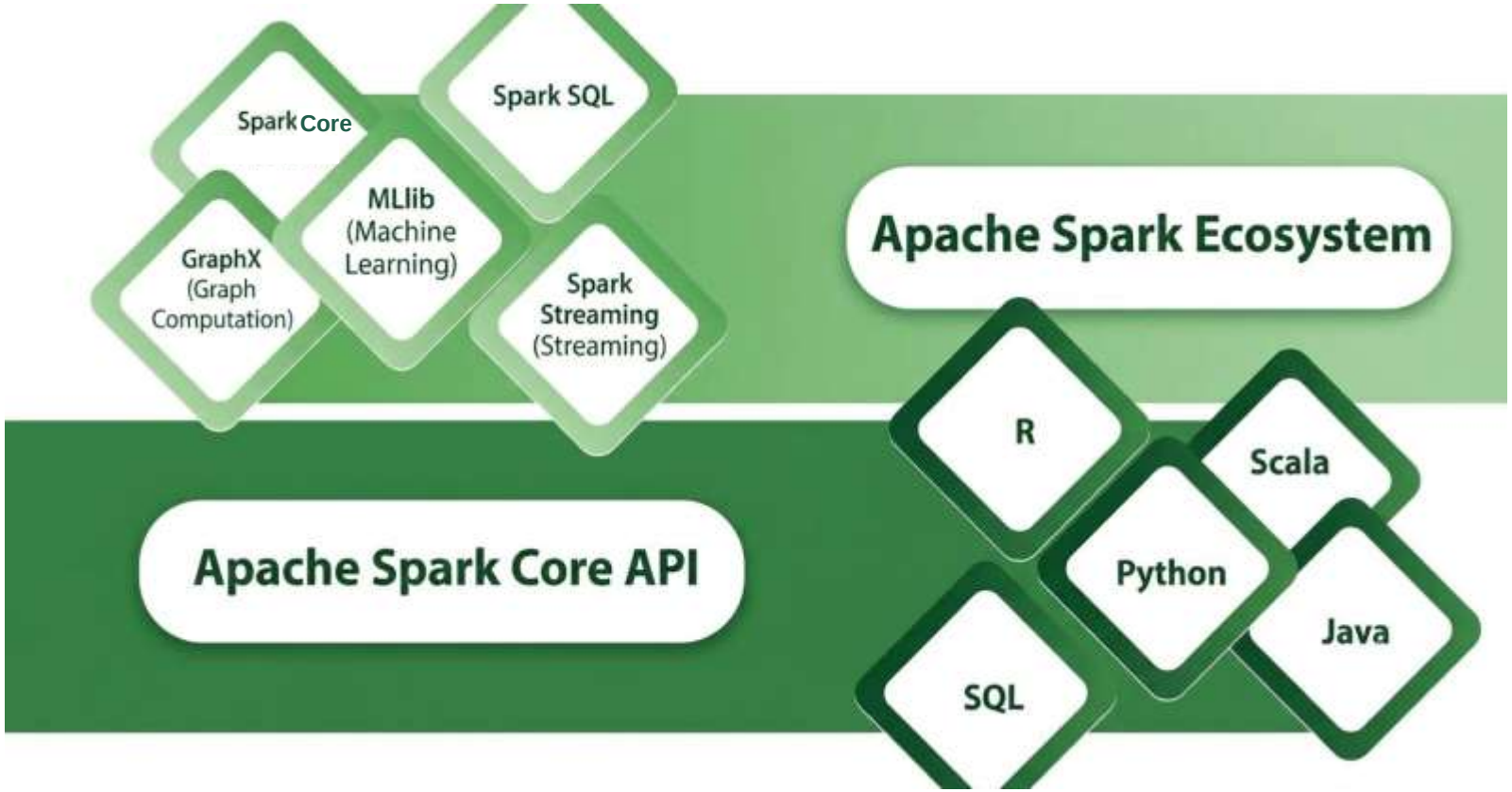
Language Breakdown

Language	Code Lines	Comment Lines	Comment Ratio	Blank Lines	Total Lines	Total Percentage
Scala	757,354	220,277	22.5%	135,497	1,113,128	70.0%
Python	117,007	74,613	38.9%	35,301	226,921	14.3%
Java	85,779	37,405	30.4%	17,755	140,939	8.9%
SQL	26,769	12,718	32.2%	5,478	44,965	2.8%
R	17,097	14,935	46.6%	3,746	35,778	2.3%
XML	9,242	1,499	14.0%	493	11,234	0.7%
JavaScript	4,778	1,138	19.2%	605	6,521	0.4%
shell script	3,710	2,264	37.9%	1,020	6,994	0.4%
CSS	1,304	241	15.6%	289	1,834	0.1%
HTML	898	53	5.6%	35	986	0.1%
Ruby	207	77	27.1%	69	353	0.0%
DOS batch script	36	58	61.7%	19	113	0.0%
C	19	20	51.3%	10	49	0.0%
Make	17	35	67.3%	10	62	0.0%
Totals	1,024,217	365,333		200,327	1,589,877	

https://www.openhub.net/p/apache-spark/analyses/latest/languages_summary

INCEPTEZ TECHNOLOGIES

Components & Programming APIs



Component	Importance	Order of Learn	Complexity
Spark Core			
Spark SQL			
Streaming			

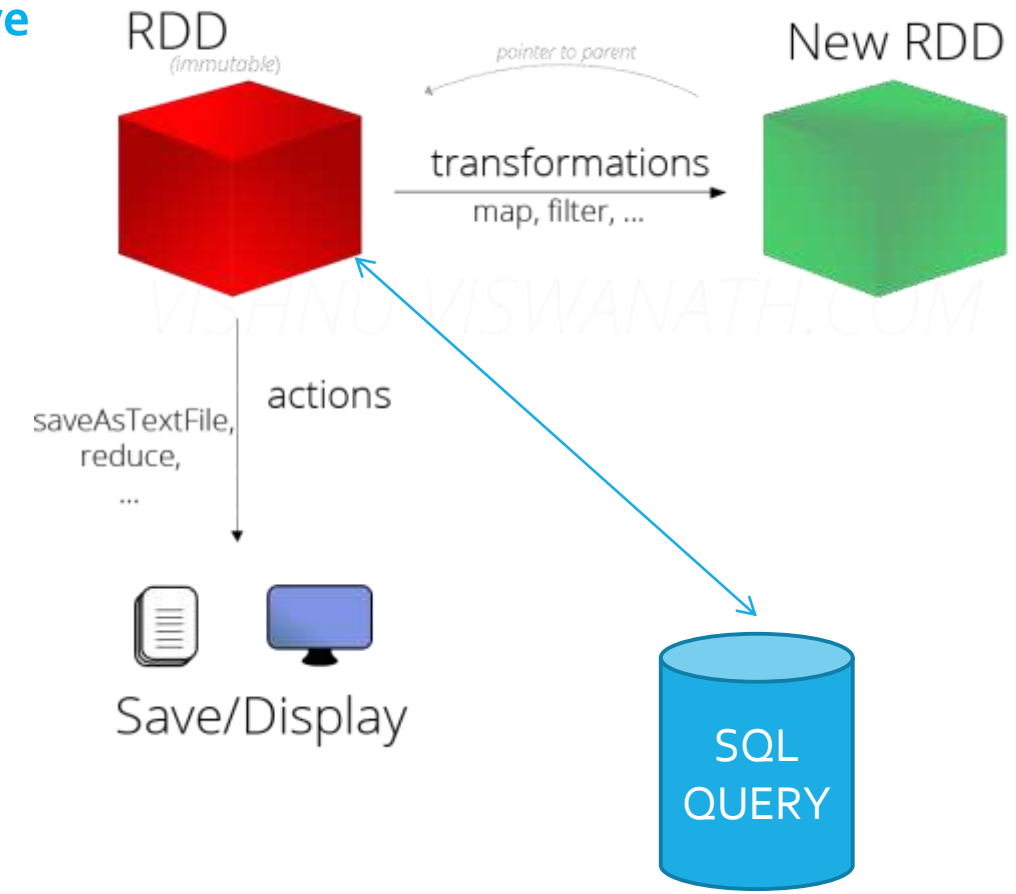
LOGIES

3. Spark Core (Fundamentals)

Pillars of Spark

All about RDDs

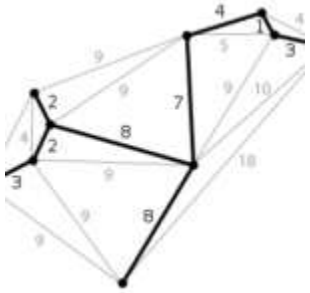
- RDDs are Core Spark **abstraction (what ever spark prog we write will be abstracted (backgrounded/landed) by RDDs)**
- **Immutable** (We can't change the given RDD, but it can be transformed to another RDD)
- RDDs are **Lazily evaluated & executed** (If we transform an RDD, it will not be transformed immediately, rather it will create a plan).
- **RDDs are Partitioned (RDDs are divided into partitions based on the size/configuration/resource availability to achieve Distributed)**
- **Fault tolerant** (If the data in the partition is lost, it can be rebuilt using Resiliency feature via (DAG – Lineage (relation)))



INCEPTEZ TECHNOLOGIES

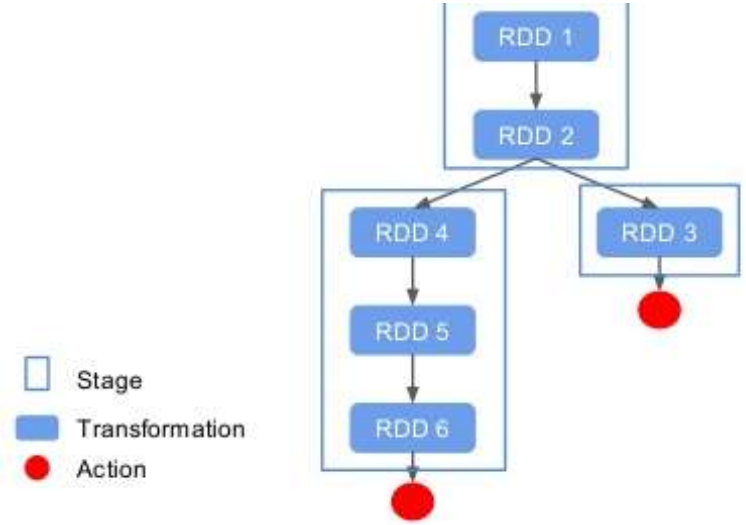
What is **DAG**

An **acyclic graph** is a graph without cycles (a cycle is a complete circuit). When following the graph from node to node**, you will never visit the same node twice (Acyclic)**.

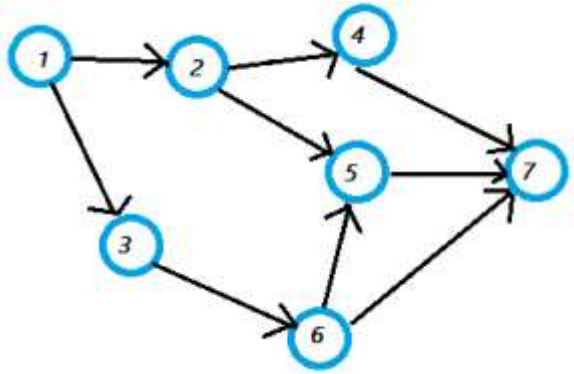


This graph (the thick black line) is acyclic, as it has no cycles (complete circuits).

A **directed acyclic graph** is an acyclic graph that has a direction as well as a lack of cycles



Acyclic – If the same node is not visited again

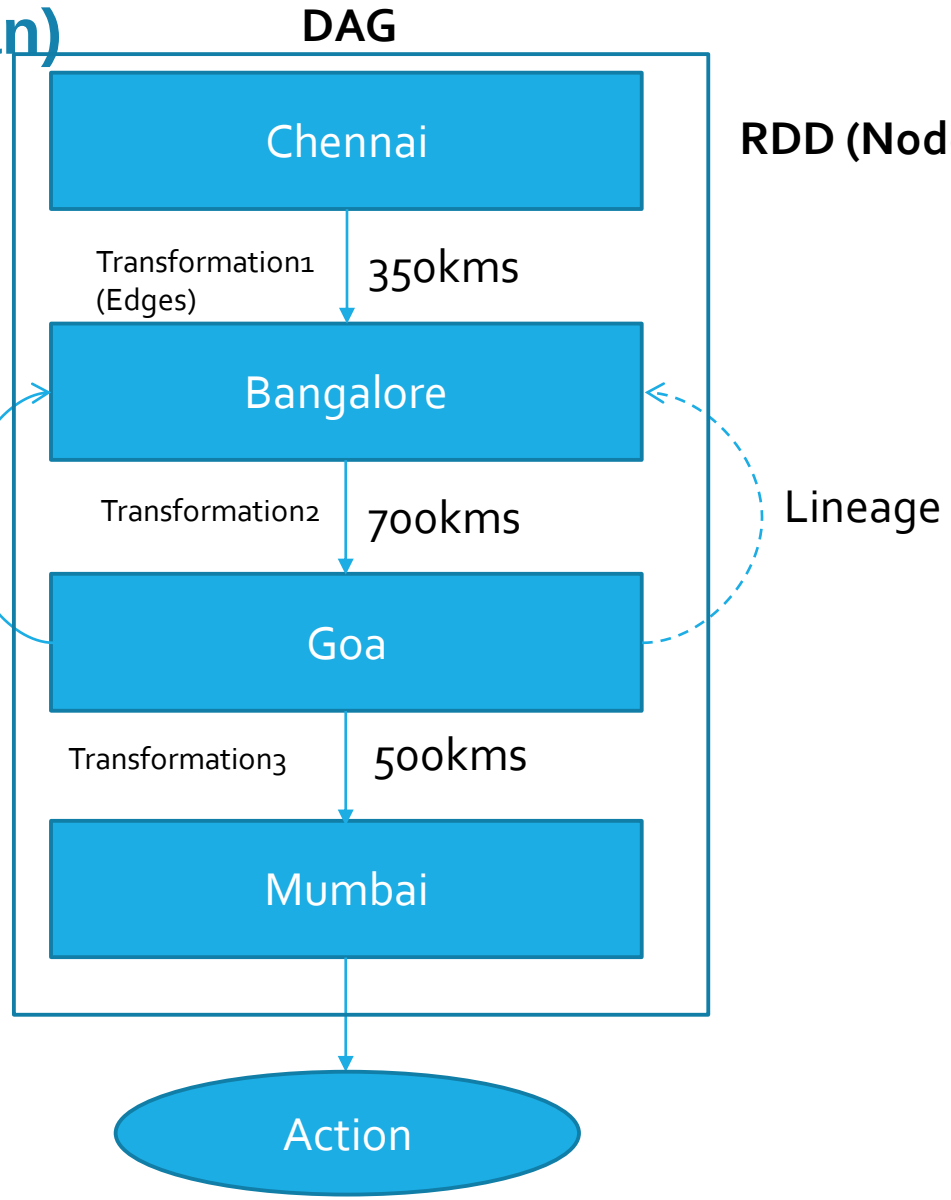


What is **Graph (tour plan)**

Directed – starting, progressing & ending

A-Cyclic – Don't visit same city again

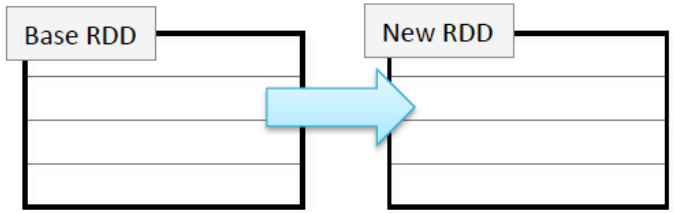
Graph - Plan



Cyclic – Is costly (700 kms 2 times)

RDD Operations: Transformations

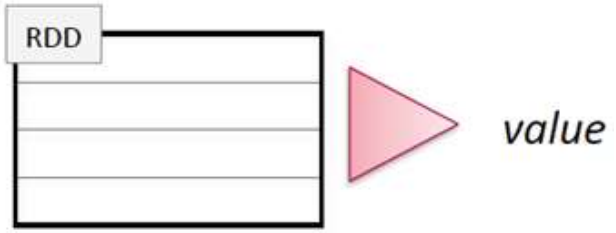
- **Transformations create a new RDD from an existing one**



- **RDDs are immutable**
  - Data in an RDD is never changed
  - Transform in sequence to modify the data as needed
- **Some common transformations**
  - **`map(`***`function`***`)`** – creates a new RDD by performing a function on each record in the base RDD
  - **`filter(`***`function`***`)`** – creates a new RDD by including or excluding each record in the base RDD according to a boolean function

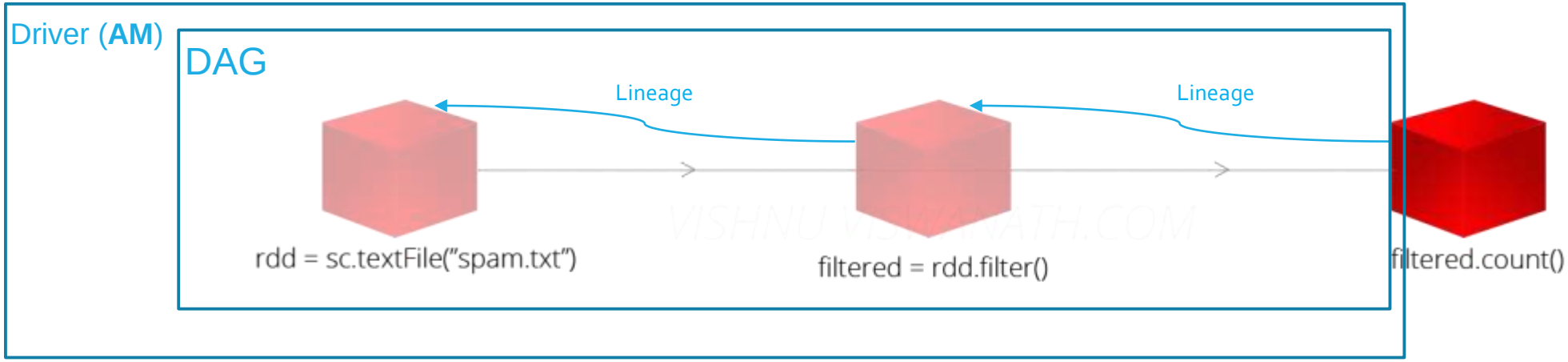
RDD Operations: Actions

- **Some common actions**
  - `count()` – return the number of elements
  - `take(n)` – return an array of the first *n* elements
  - `collect()` – return an array of all elements
  - `saveAsTextFile(filename)` – save to text file(s)



RDD Lineage

- When you call a transformation, Spark does not execute it immediately, instead it creates a **lineage**.
- Lineages in the form of DAG stored in Driver, keeps track of what all transformations has to be applied on that RDD.



VISHNU VISWANATH.COM

INCEPTEZ TECHNOLOGIES

4. Spark Core – Handson (Obsolute)
Terminologies
Architecture
Performance Optimization

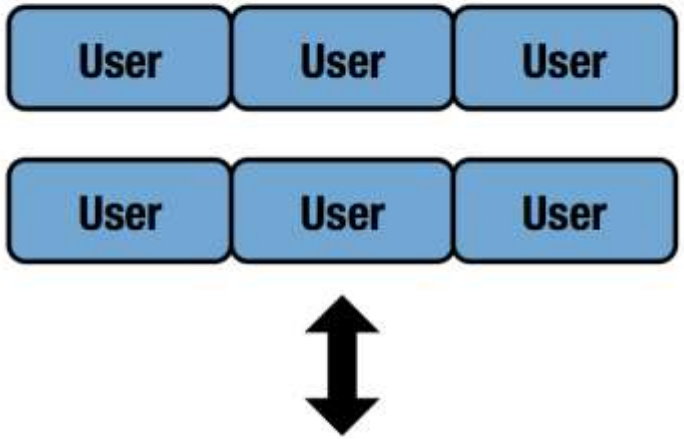
5. Spark SQL - Theory

gcloud dataproc clusters create cluster-2e47 --enable-component-gateway --region us-east4 --subnet default --no-address --master-machine-type e2-standard-2 --master-boot-disk-type pd-standard --master-boot-disk-size 100 --num-workers 2 --worker-machine-type e2-standard-2 --worker-boot-disk-type pd-standard --worker-boot-disk-size 500 --secondary-worker-boot-disk-type pd-standard --secondary-worker-boot-disk-size 100 --image-version 2.3-debian12 --properties spark:spark.dataproc.engine=lightningEngine,spark:spark.dataproc.lightningEngine.runtime=default,dataproc:dataproc.logging.stackdriver.job.driver.enable=true --optional-components ICEBERG,DELTA,JUPYTER --scopes 'https://www.googleapis.com/auth/cloud-platform' --secondary-worker-type non-preemptible --project project-b809fdce-d1d0-4f79-b86

Spark SQL

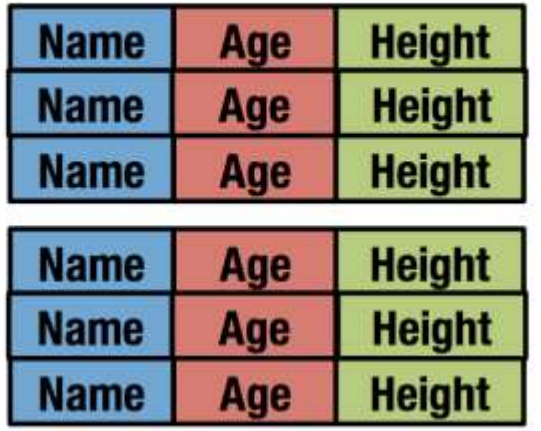
Spark + RDDs

Functional transformations on partitioned collections of **opaque** objects.

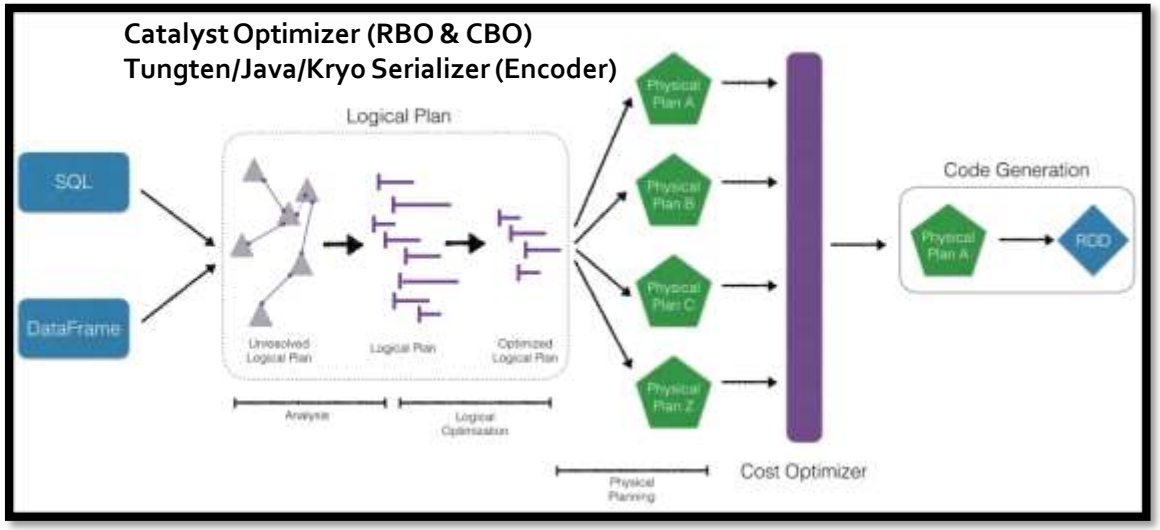


SQL + SchemaRDDs

Declarative transformations on partitioned collections of **tuples.**



Spark SQL Architecture





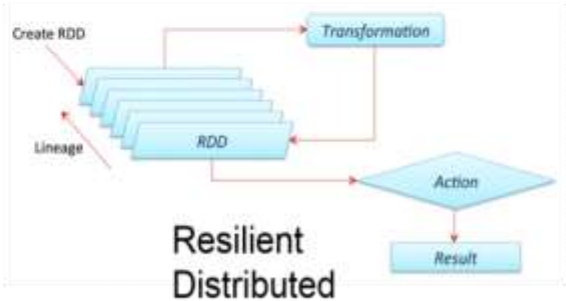
Data Source API

DataFrame API

Named Columns



Interpreter & Optimizer



Resilient Distributed Dataset

Spark SQL Service

Spark SQL RESULTS

Why Spark SQL

- **Coding Complexity is less** (no need to work on collection types (foggy objects)
- **Let optimizer do the hard work(Catalyst/Tungten)**
- **Integrated (Unified/Federated)**
  - Seamlessly mix SQL queries with Spark programs (Unified)
  - Connect to any data source/targets the same way (Federated)
- **Hive Compatibility**
  - Run unmodified Hive queries on existing data.

Functional Transformation

RDD

```
auctionRDD = spark.sparkContext.textFile("file:///home/hduser/sparkdata/auctiondata")
ebay = auctionRDD.map(lambda x:x.split("~")).map(lambda p: (p[0], f loat(p[1]), f loat(p[2]),p[3])
```

Declarative Transformation

DF

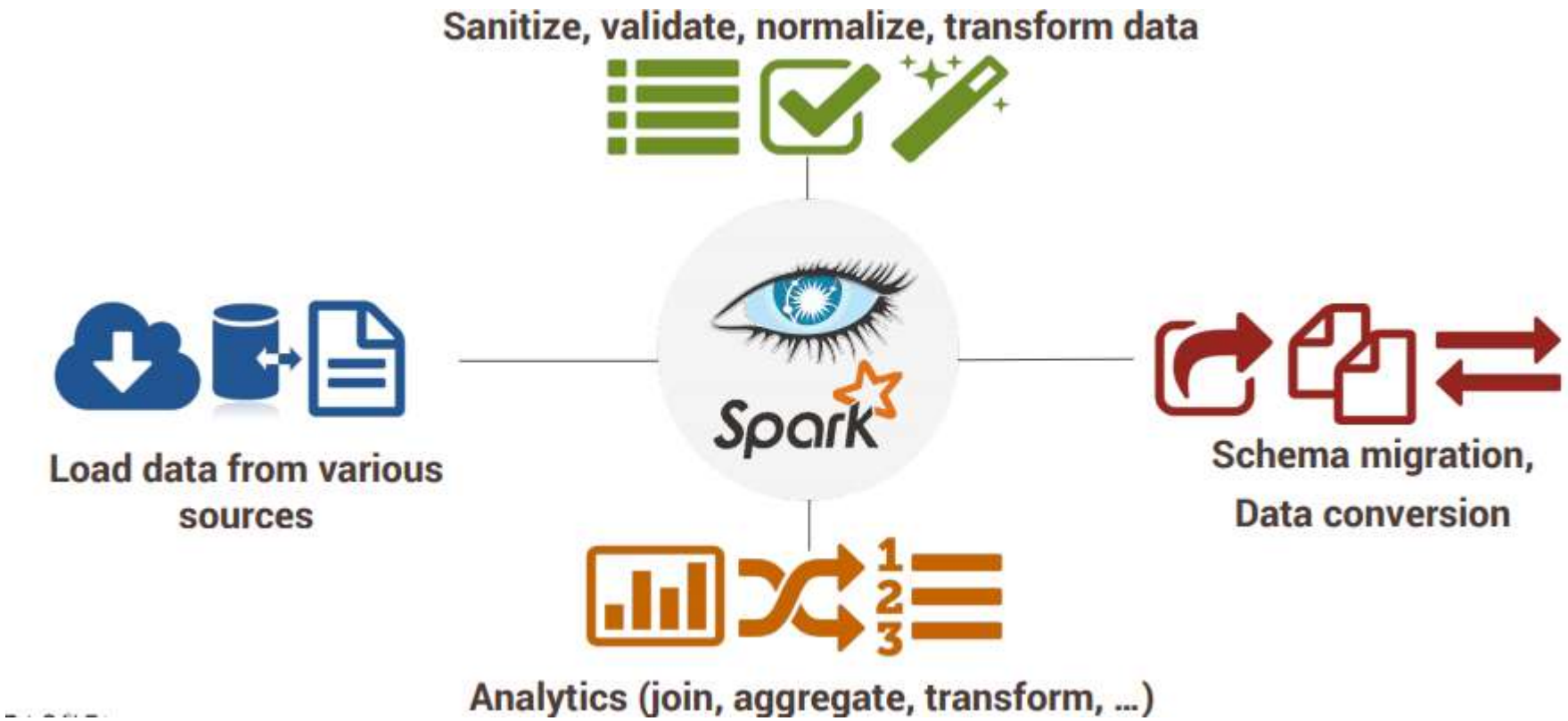
```
auction = spark.createDataFrame(ebay)
grpbid =
auction.groupBy("item").agg(max("bid").alias("max_bid"),sum("price").alias("sp")).filter($"max_bid">10
0).sort($"sp".desc)
```

SQL

```
auction.createOrReplaceTempView("view1")

sql("""select item,max(bid) as max_bid,sum(price) as sp from view1
group by item
having max(bid)>100
order by sp desc""").show(10,false)
```

Applications of Spark SQL – (Try to include some of these terminologies in your CV)



INCEPTEZ TECHNOLOGIES

6. Spark SQL - Handson

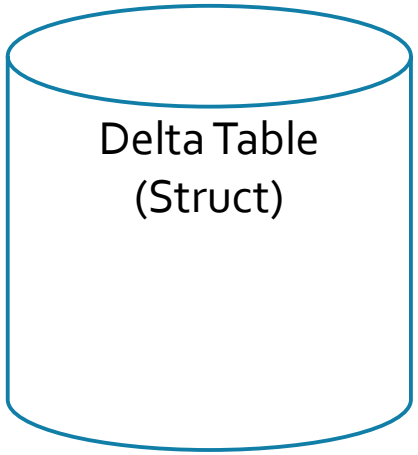


7. Databricks ~~Distribution~~

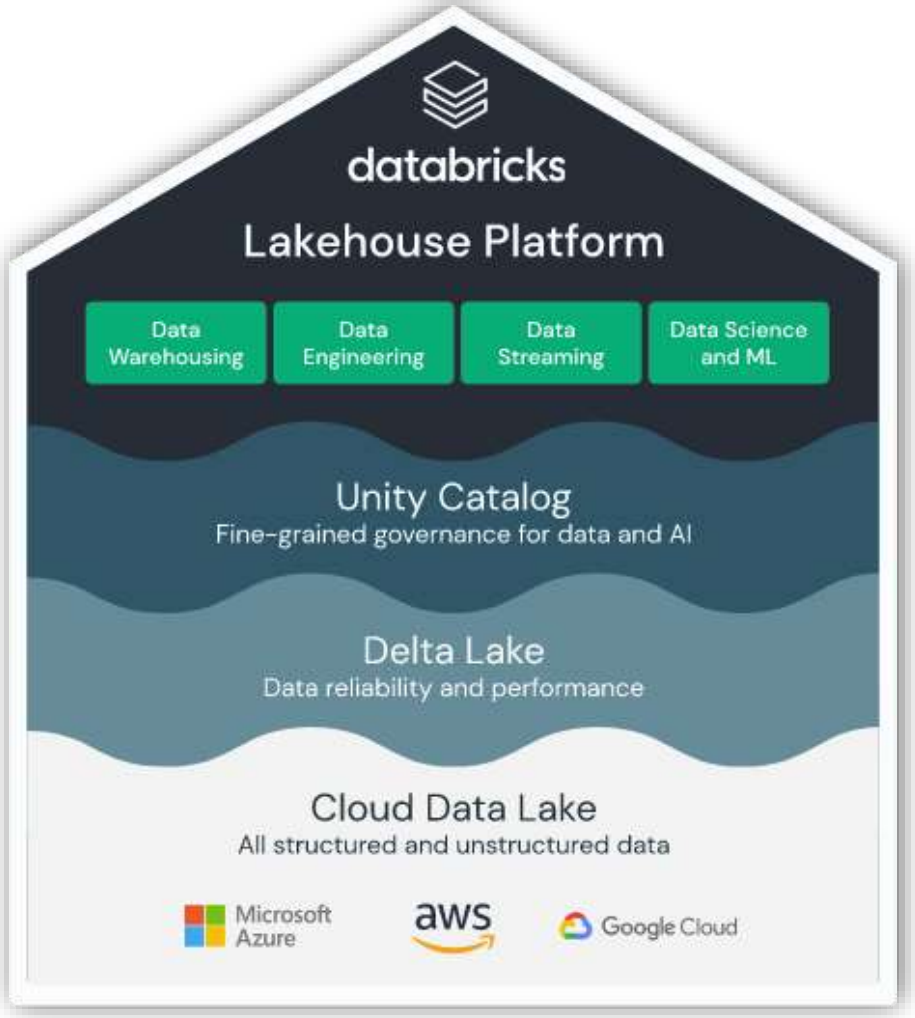
Managed Cloud Platform powered by Apache Spark

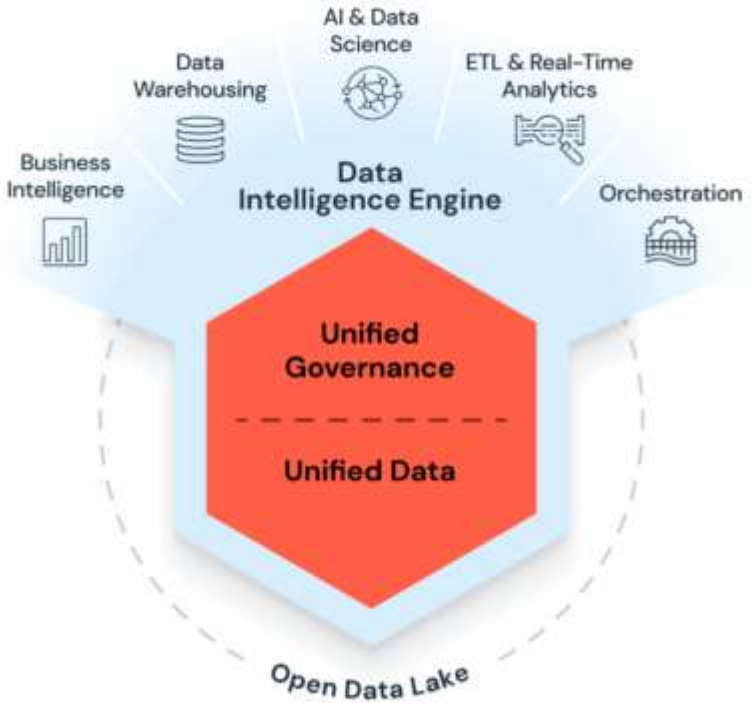
Databricks (Managed Cloud Platform) Overview

- Databricks = **Managed Apache Spark Platform** + **Cloud** (Data Lake(house))
- **Databricks** is designed to manage large datasets across clusters of computers, providing a comprehensive, enterprise-ready cloud platform for ***data engineering, data warehousing, machine learning, AI, and analytics with security+catalog+advanced libraries powered by Databricks on top of Apache Spark***.

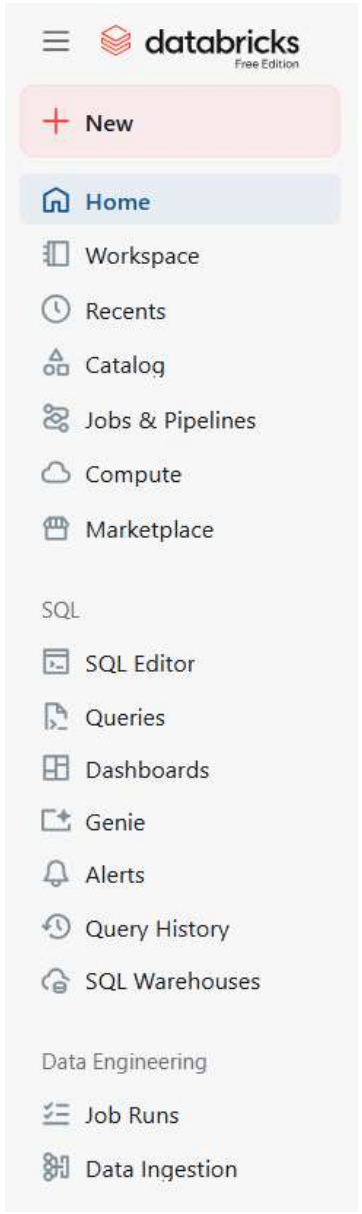


Dbfs
Delta(parquet)





Databricks Key features and components of Databricks distribution



Main Menu

Home – Landing page to access quick actions, recent items, and shortcuts.
Workspace – Stores notebooks, folders, files, Query, dashboards, Genie and project assets.
Recents – Displays recently opened or edited notebooks, queries, and files.
Unity **Catalog** – Centralized governance layer for managing data, schemas, tables, and permissions via Unity Catalog.
Jobs & Pipelines – Create, schedule, and orchestrate workflows, jobs, and Delta Live Tables pipelines.
Compute – Manage compute clusters, SQL warehouses, runtimes, and **serverless** resources.
Marketplace – Discover and install partner datasets, AI models, connectors, and integrations.

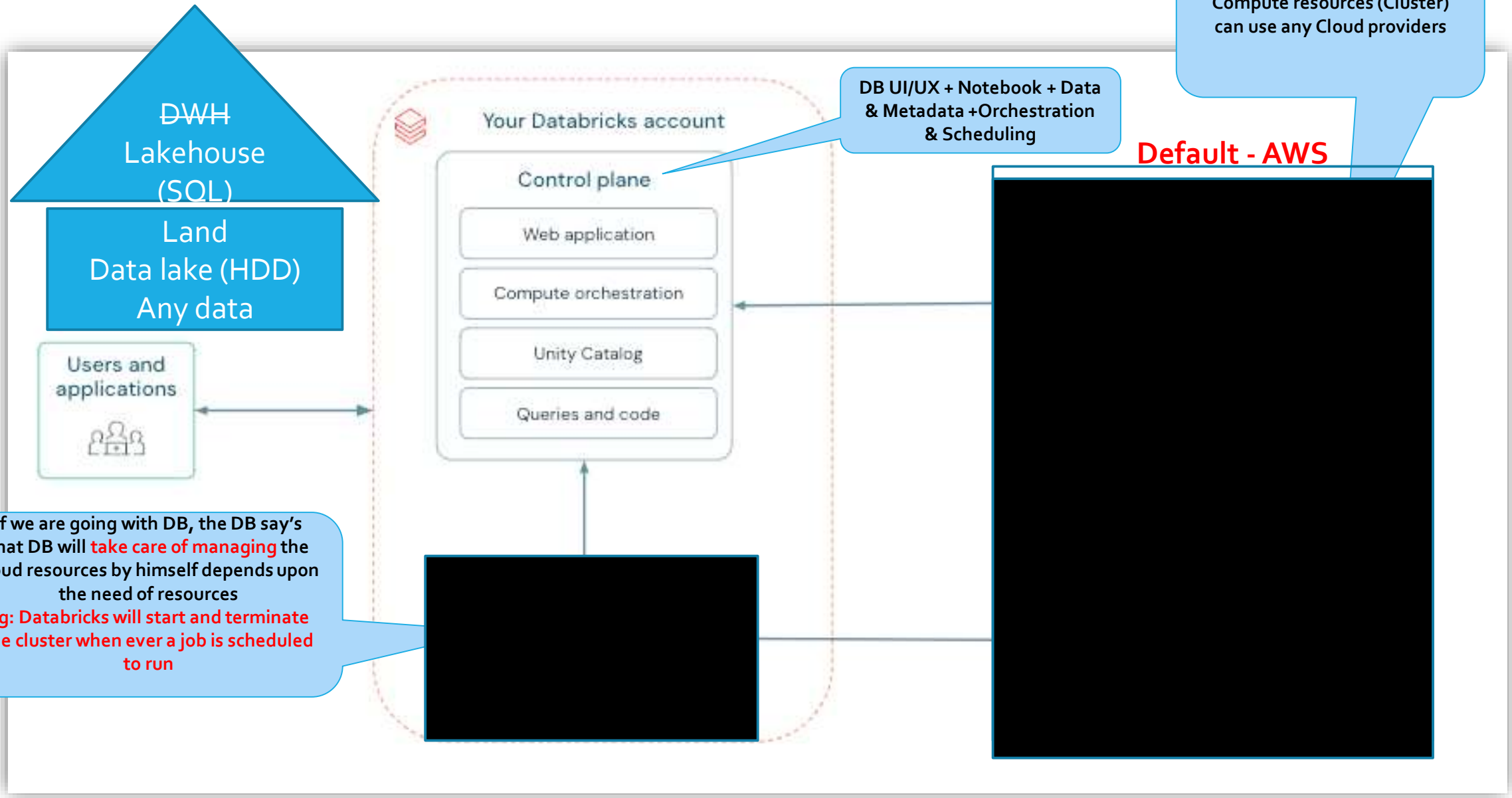
SQL Section

SQL Editor – Interactive editor to write, run, and visualize SQL queries.
Queries – Save, organize, and manage SQL queries.
Dashboards – Build and share interactive dashboards from query results.
Genie – AI assistant for conversational SQL and automated insights.
Alerts – Create alerts based on query results and threshold conditions.
Query History – Review historical SQL queries executed in the workspace.
SQL Warehouses – Manage SQL compute engines optimized for BI and analytics workloads.

Data Engineering Section

Job Runs – View logs, history, and status of executed jobs.
Data Ingestion – Tools to ingest structured and unstructured data into Delta Lake.

Databricks Architecture



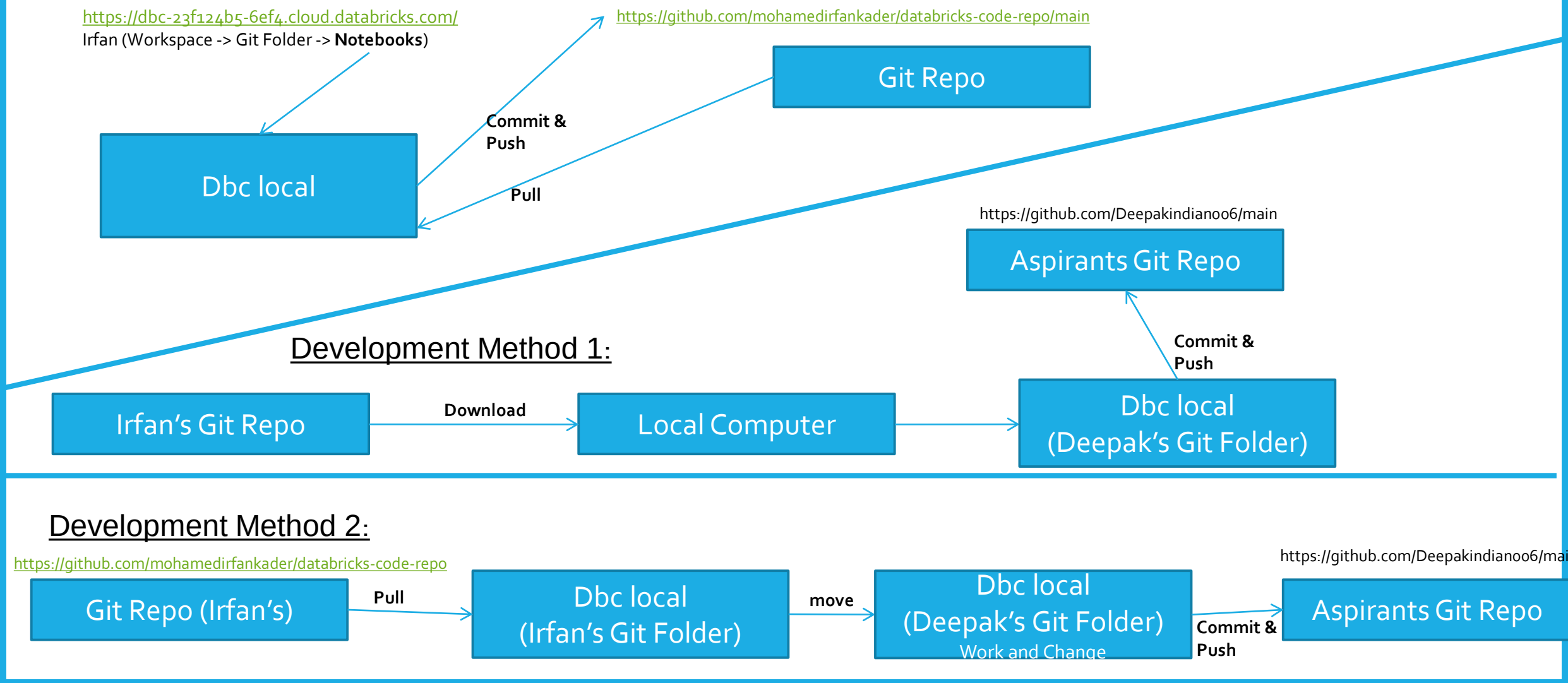
Databricks Notebooks Components

https://github.com/mohamedirfankader/databricks-code-repo

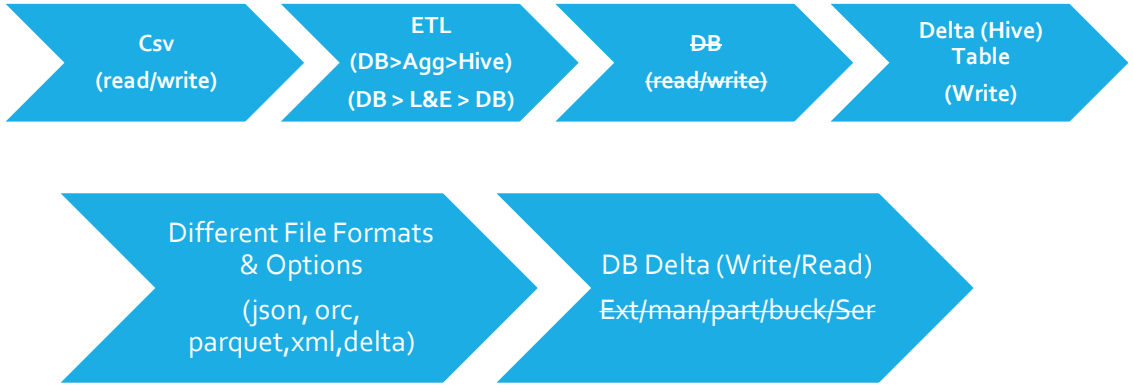
Notebooks Basics

- ✓ **All About Notebooks operations (Import/Create/Export/push/pull)**
- ✓ **Magic Commands (%run/sh/fs/sql/python/md)**
- ✓ **Markdown Commands**
- ✓ **DB Utils – WidgetsUtils, Notebook run Utils, fsUtils**
- ✓ **Widgets – WidgetsUtils**

How we are using Databricks & Git (Between Irfan and Aspirants)



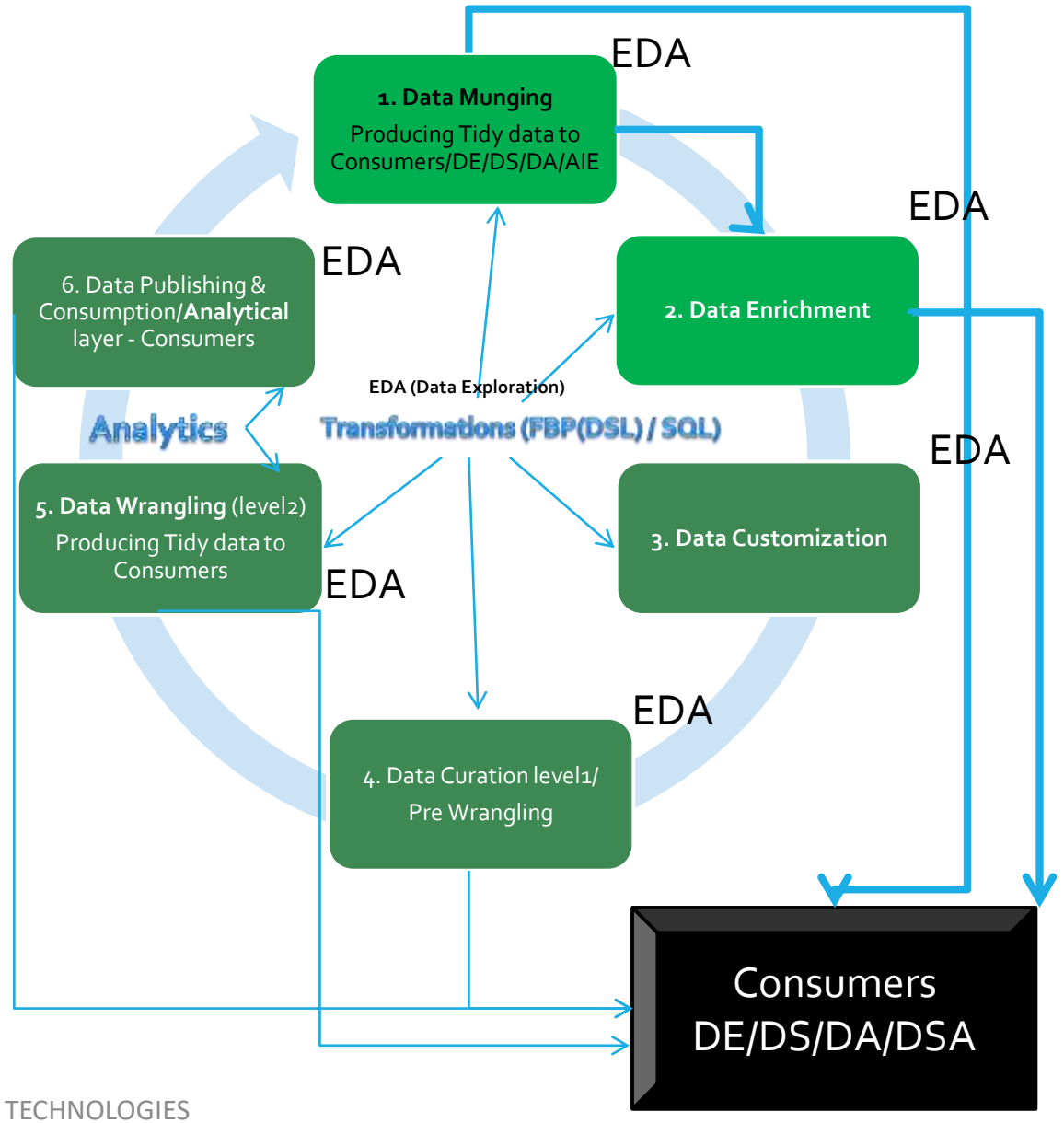
EL BB1 (Built in functions & Options) Pipeline



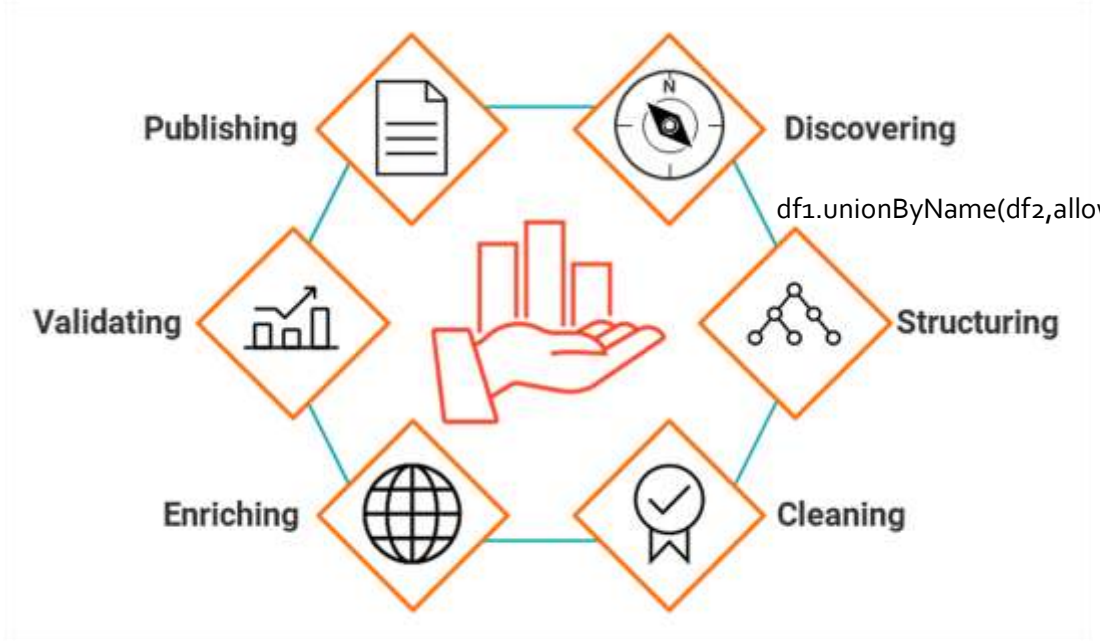
BB1 Program – Extract & Load

BB2 Program – Transformation & Analytics + AI

ETAL Pipeline (BB2) using Pyspark

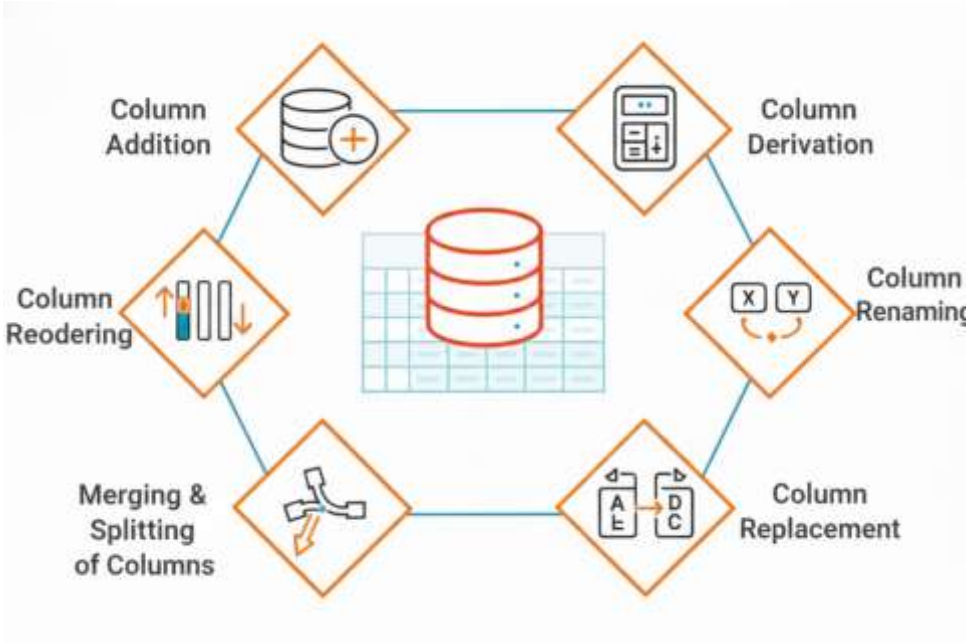


Data Munging Steps



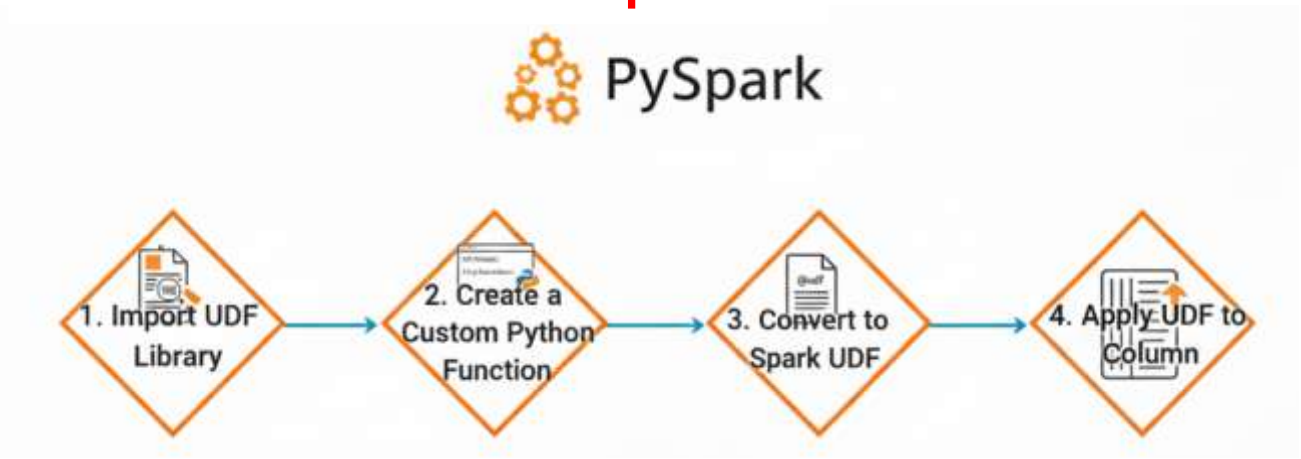
df1.unionByName(df2,allowmissingcols)

Data Enrichment Steps



DF function - withColumn, withColumnRenamed, select, selectExpr, drop
Column level functions – col, lit etc.,

Data Customization Steps



When do we go for Schema Evolution?
Over the time, if no. of col are keep added by source
Serialization while writing+ mergeSchema while reading
When do we go for Schema Merging?
In a given day, If we get multiple files of related (not same) structure
After reading in dataframe format -> unionByName + allowMissingColumns

Data Curation Steps



Data Wrangling Steps



Joins

DATASET1 (left)

TXNNO	AMT
1	100000
2	200000
3	150000
4	250000
5	100000
6	200000
7	150000
8	250000
16	100000
17	200000

DATASET2 (right)

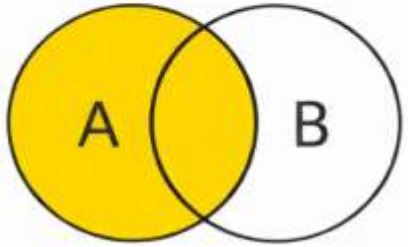
TXNNO	AMT
1	100000
2	200000
3	150000
4	250000
5	100000
6	200000
7	150000
8	250000
9	100000
10	200000
11	200000
12	150000
13	250000
14	100000
15	200000

TYPES OF JOINS	REGULARY USED	GOOD TO USE (PERFORMANCE)	NO. OF ROWS AS RESULT	SHORT DESC	DESCRIPTION
join/inner/equi/natural	Yes	Good to Use	8	Matching data both sides	Data from dataset1 joined with dataset2 using some join/filter conditions that returns matching values between 2 datasets
outer join - left	Yes	Good to Use	10	Matching data + excess data from left sides	Data from dataset1 joined with dataset2 using some join/filter conditions that returns ALL values from dataset1 (left) and nulls for unmatched value of dataset2
outer join - right	No	not required to be used - subjective	15	Matching data + excess data right side	Data from dataset1 joined with dataset2 using some join/filter conditions that returns ALL values from dataset2 (right) and nulls for unmatched value of dataset1
outer join - full	No	not good to use but subjective	17 (8+2+7)	Matching data + excess data from both sides	Data from dataset1 joined with dataset2 using some join/filter conditions that returns ALL values from dataset1 (left) & dataset1 (right) with nulls for unmatched value of both datasets
left semi	No	good to use, subjective	8	Matching data from left dataset alone	Data from dataset1 joined with dataset2 using some join/filter conditions that returns MATCHING values from dataset1 (left) alone (Works like a suquery with **in** condition) Otherwise: Left semi will do inner join and returns only the left data
left anti	No	good to use, subjective	2	Unmatched data from left dataset alone	Data from dataset1 joined with dataset2 using some join/filter conditions that returns UN-MATCHING values from dataset1 (left) alone (Works like a suquery with **not in** condition) Otherwise: Left anti will do left join and returns only the left data which is not matching with right data
self join (dataset1)	No	good to use, subjective	10	Matching data both sides	Data from dataset1 joined with dataset1 using some join/filter conditions that returns matching values between 2 datasets
cross join or cartesian product or product join	No	Not Good to Use	150	All Data will be returned with n*n iterations output	Data from dataset1 joined with dataset2 WITHOUT using any join conditions that returns values between 2 datasets in an iteration of (10*15=150 rows will be returned)

DATA WRANGLING - Joins

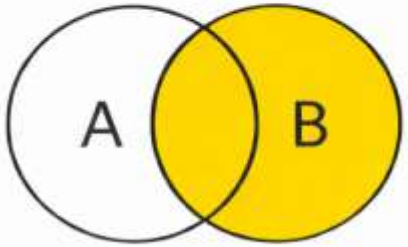
JOIN Types

Display all the rows of A & matching rows of B **with values** and unmatched rows of B with nulls



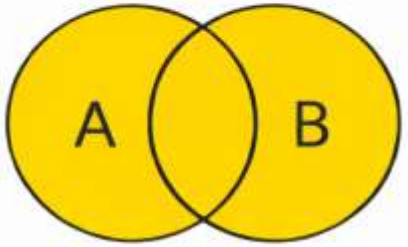
Left Outer

Display all the rows of B & matching rows of A **with values** and unmatched rows of A with nulls

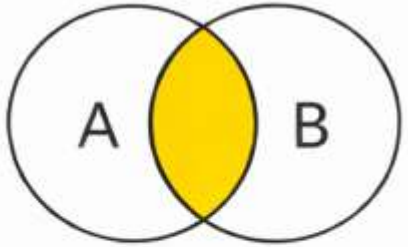


Right Outer

Display all the rows of A& B, matching rows of A **with values , matching rows of B with values** and unmatched rows of A with nulls , unmatched rows of B with nulls (apply inner,left,right)

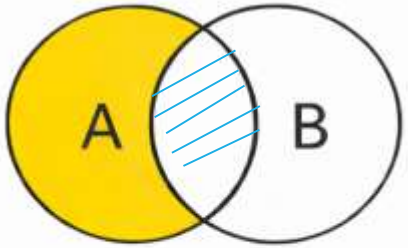


Full Outer



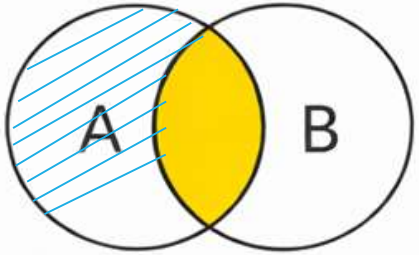
Inner

Display of matching records of A & B



Left Anti

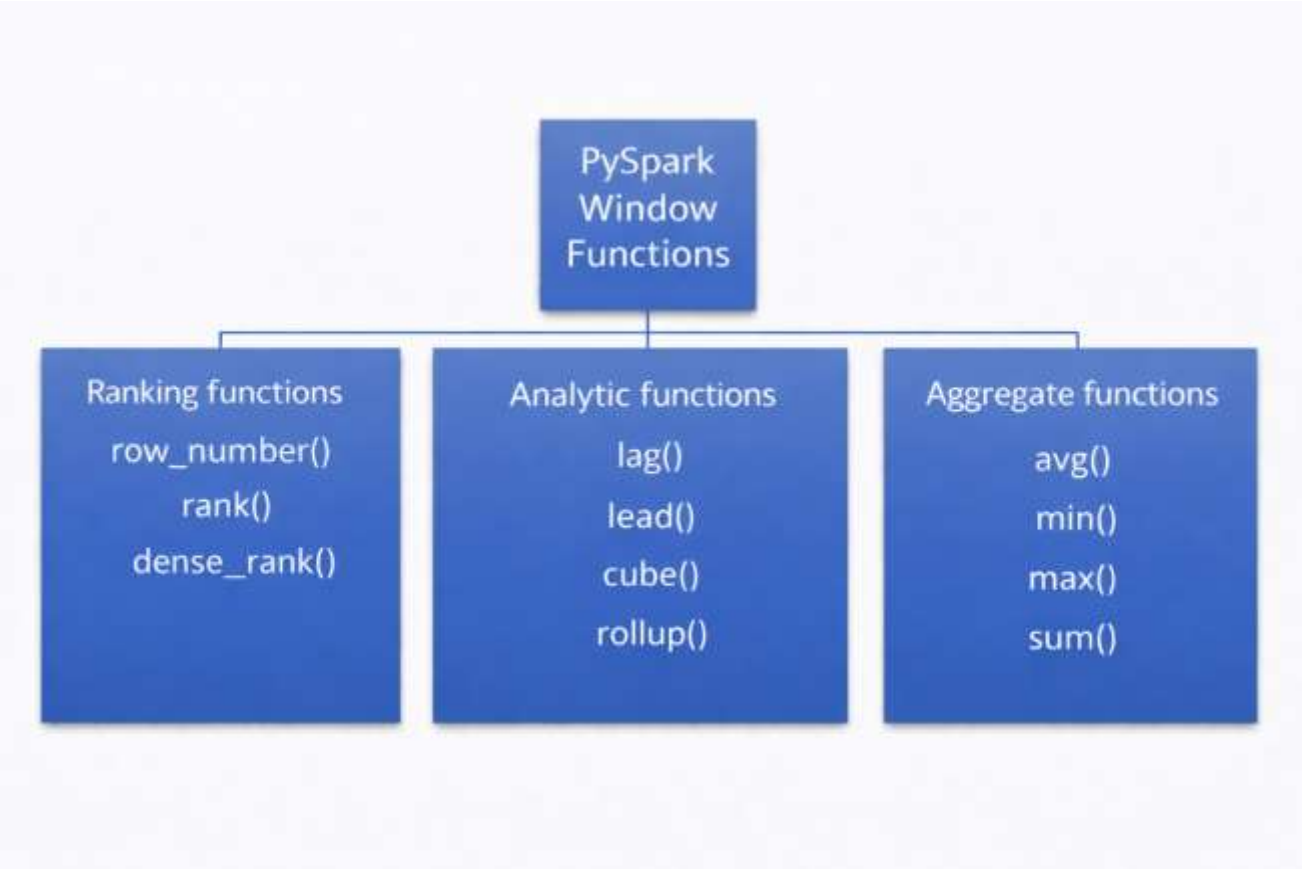
Display (only) A records of Non matching records of A & B



Semi Join

Display (only) A records of **matching** records of A & B

DATA WRANGLING – Windowing & Analytical Functions



Rownumber, Rank, DenseRank Functions

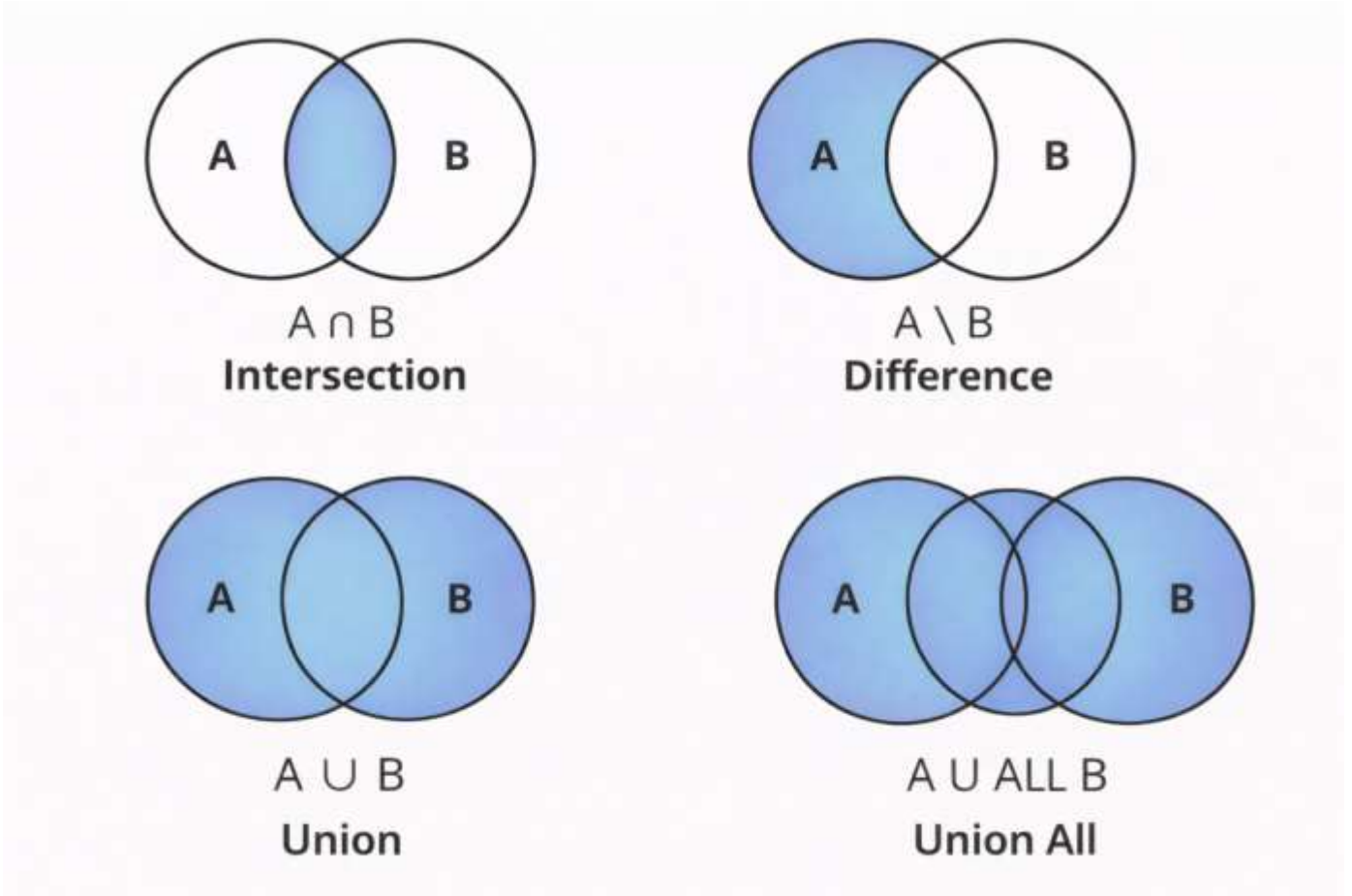
EMPID	Name	Salary	_Rank	DenseRank	RowNumber
EMP102	Sam	20000.00	1	1	1
EMP108	Rahul	20000.00	1	1	2
EMP111	akshay	20000.00	1	1	3
EMP106	Mahesh	18000.00	4	2	4
EMP101	Vishal	15000.00	5	3	5
EMP109	menaka	15000.00	5	3	6
EMP105	Ravi	10000.00	7	4	7

Lead, Lag Analytical Functions

	Year	Quarter	Sales	NextQuarterSales
1	2017	1	55000.00	0.00
2	2017	2	78000.00	55000.00
3	2017	3	49000.00	78000.00
4	2017	4	32000.00	49000.00
5	2018	1	41000.00	32000.00
6	2018	2	8965.00	41000.00
7	2018	3	69874.00	8965.00
8	2018	4	32562.00	69874.00
9	2019	1	87456.00	32562.00
10	2019	2	75000.00	87456.00
11	2019	3	96500.00	75000.00
12	2019	4	85236.00	96500.00

Default value

Set Operations



ETAL Pipeline (BB2) using Pyspark – **DE/DCD/DA**

Data Munging: Process of transforming and mapping data from Raw form into Tidy format

- Passive - Data Discovery (EDA) - Exploratory Data Analysis - attributes(columns/datatype) and patterns (format/sequence/alpha/numeric).
- ✓ Understand the data in its own format
- ✓ Understand the data by applying inferschema
- ✓ Apply Custom structure using the inferred schema
- ✓ Identify Nulls, De-Duplication, Prioritization, Datatypes, formats etc.,
- ✓ Apply reject strategy
- ✓ Raw data - Statistical Summarize the overall pattern of data
- Data Structurizing - Combining Data + Schema Evolution/Merging/Melting
- Cleansing (drop), Scrubbing (replace/fill)
- Standardization - Column name/type/order/number of columns changes/Replacement & Deletion of columns

Data Enrichment: Detailing & Richness of Data

- Add, Remove, Rename, Modify/replace, Repositioning
- Split, merge/Concat
- Type Casting, Format model & Schema Migration
- ***Format modeling (changing of format), Derivation (deriving additional columns/data from a given column)***

Data Customization (Processing) (Business logics): Applying custom logic on the data and do data enrichment

- How to create Python Functions & convert/Register it as a UDF to call these functions in the DSL/SQL Queries respectively
- Go for UDFs if it is in evitable – Because the usage of UDFs will degrade the performance of your pipeline
- Creation of the Reusable Frameworks with Functions (Later - Modernization/Industrialization)

Data Curation (Core/Fundamental Processing) (Pre Wrangling): Data Transformation or Curation Techniques

- Filter, Grouping, Aggregation, sorting, (select/SelectExpr) Wrapper Queries (inline view queries)…
- Transformations (built-in function coalesce/case, coalesce, trim, isNull, lit…)
- Curated Analytical (**Pivot/Group/Agg**)/Statistical (built-in functionalities – corr, sample, cov, freqItem, Summary)
- ***Format modeling (changing of format), Derivation (deriving additional columns/data from a given column) – cast, substr, to_date, concat***

**** Data Wrangling (Complete /Curation Processing/Complete Analytical) (Wrangling):** Data Transformation or Curation

- Different types of joins
- Interview related scenarios
- Joins, Lookup, Lookup & Enrichment, Denormalization (Schema modeling Star Schema -> Denormalized/Fat/Wide dataset), **Windowing, Analytical (Pivot/Group/Aggregation/hierarchical comparison/complete aggregation (cube/rollup)), Set Operation**…

Outbounding/Egress/Enabling or Consumption (Load part)

Rollup & Cube

Rollup

	Department	salary
1	IT	16000.00
2	Sales	13000.00
3	NULL	29000.00

	Department	Category	salary
1	IT	A	8000.00
2	Sales	A	7000.00
3	NULL	A	15000.00
4	IT	B	5000.00
5	Sales	B	4000.00
6	NULL	B	9000.00
7	IT	C	3000.00
8	Sales	C	2000.00
9	NULL	C	5000.00
10	NULL	NULL	29000.00

Cube

	Department	Salary
1	IT	16000.00
2	Sales	13000.00
3	NULL	29000.00

	Department	Category	Salary
1	IT	A	8000.00
2	Sales	A	7000.00
3	NULL	A	15000.00
4	IT	B	5000.00
5	Sales	B	4000.00
6	NULL	B	9000.00
7	IT	C	3000.00
8	Sales	C	2000.00
9	NULL	C	5000.00
10	NULL	NULL	29000.00
11	IT	NULL	16000.00
12	Sales	NULL	13000.00

What We are going to learn in Databricks



1. E2E Data Engineering Activities
--DE & DA operations you perform Dev/Testing/Learning/Apply
Extraction(Ingress) (BB1) -> T&A (BB2) Munging-Enrichment-Customization-Curation/Transformation-Wrangling
-> Load/Persisting(Egress)

2. Framework PySpark - SQL/Python , Platform - Databricks(Delta) (Independent/Non Agnostic fashion)
DF based DSL or SQL (Declarative)

3. Generic Framework (UDF, Common, Business Specific) + Config Driven Approach

4. Medallion Architecture (Some architecture)

5. Deltalake (Datalake (Delta File) + Lakehouse (Delta Table))

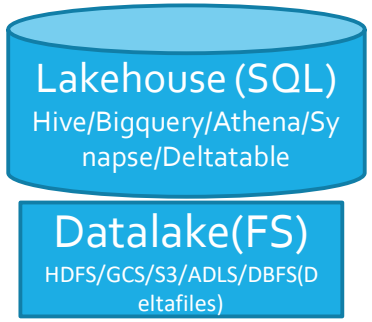
6. Databricks (Agnostic/Fully dependent)



Spark DSL(DF) to SQL(TV) Comparison

Who wins for what operations

Category	Use Case / Function	DataFrame DSL (PySpark)	Spark SQL	Better one	Description
Basic Operations	Select columns	df.select("col1","col2")	SELECT col1,col2 FROM temp_view	DSL	IDE-friendly, safer
Basic Operations	Filter rows	df.filter(col("age")>30)	WHERE age>30	DSL	Chainable
Basic Operations	Distinct	df.select("c").distinct()	SELECT DISTINCT c	SQL	Cleaner
Basic Operations	Limit	df.limit(10)	LIMIT 10	Equal	Same
Column Handling	Alias column	col("a").alias("a1")	a AS a1	SQL	Simple
Column Handling	Derived column	withColumn("t",col("q")*col("p"))	q*p AS t	DSL	Reusable
Column Handling	Drop column	df.drop("c")	SELECT * EXCEPT(c)	DSL	Explicit
Aggregations	Group by	groupBy("city").count()	GROUP BY city	SQL	Readable
Aggregations	Multiple aggs	agg(sum("a"),avg("a"))	SUM(a),AVG(a)	SQL	Analytics
Aggregations	Approx count	approx_count_distinct()	APPROX_COUNT_DISTINCT()	Equal	Same
Joins	Inner join	df1.join(df2,"id")	JOIN ON	DSL	Safer
Joins	Anti join	left_anti	LEFT ANTI JOIN	SQL	Clear intent
Joins	Broadcast join	broadcast(df)	/*+ BROADCAST */	DSL	Explicit
Window Functions	Row number	row_number().over(w)	ROW_NUMBER() OVER()	SQL	Cleaner
Window Functions	Lag / Lead	lag("a",1)	LAG(a,1)	SQL	Readability
Null Handling & Conditional Logic	Null check	isNull()	IS NULL	SQL	Simpler
Null Handling & Conditional Logic	Case when	when().otherwise()	CASE WHEN	SQL	Declarative
Null Handling & Conditional Logic	Fill null	fillna(0)	COALESCE()	DSL	Bulk
String Functions	Upper / Lower	upper(col("c"))	UPPER(c)	Equal	Same
String Functions	Substring	substring(c,1,3)	SUBSTRING(c,1,3)	SQL	Cleaner
String Functions	Regex replace	regexp_replace()	REGEXP_REPLACE()	Equal	Same
Date & Time	Current date	current_date()	CURRENT_DATE()	SQL	Simple
Date & Time	Date diff	datediff(d1,d2)	DATEDIFF()	Equal	Same
Date & Time	Add months	add_months()	ADD_MONTHS()	Equal	Same
Math Functions	Round	round(col("a"),2)	ROUND(a,2)	Equal	Same
Math Functions	Abs	abs(col("a"))	ABS(a)	Equal	Same
Collections	Explode	explode(col("arr"))	EXPLODE(arr)	DSL	Chainable
Collections	Array size	size(arr)	SIZE(arr)	Equal	Same
JSON & Struct	From JSON	from_json()	FROM_JSON()	DSL	Schema-driven
JSON & Struct	Get field	col("s.a")	s.a	SQL	Cleaner
Delta Lake	Merge (SCD)	DeltaTable.merge()	MERGE INTO	SQL	Declarative
Delta Lake	Time travel	option("versionAsOf")	VERSION AS OF	SQL	Readable
Delta Lake	Optimize	OPTIMIZE	OPTIMIZE	Equal	Same
Streaming	Read stream	readStream	CREATE STREAMING TABLE	DSL	Flexible
Streaming	Watermark	withWatermark()	WATERMARK	DSL	Programmatic
Performance	Cache	df.cache()	CACHE TABLE	DSL	Pipelines
Performance	Explain plan	df.explain()	EXPLAIN	Equal	Same
Metadata	Temp view	createOrReplaceTempView()	FROM view	DSL	Required
Metadata	Schema	df.printSchema()	DESCRIBE TABLE	DSL	Dev-friendly
Error Handling	Try cast	N/A	TRY_CAST()	SQL	Safer
Reusability	Functions	Python functions	SQL UDF	DSL	Maintainable
Dynamic Logic	Loops / Params	Supported	Limited	DSL	Programmatic
Best Practice	ETL pipelines	Preferred	Supported	DSL	Production
Best Practice	Analytics	Supported	Preferred	SQL	BI friendly



In this Fleet/Logistics Project we are going to amalgamate…

1. **BB1 & BB2 & Usecase (Logistics) we developed,** upgrading to the below model
2. **Generic Framework (Spark)**
3. **Medallion** Architecture (Bronze+Silver+Gold) (Any Platform)
4. Medallion + Lakehouse (**Deltalake**) Architecture (Any Platform)
5. **Databricks** 3 core Dataengg components (**Lakeflow Jobs** (Orchestration + Scheduling))



Building Generic (FBP) Framework / Plug N Play components

- Centralized and controllable
- Production Ready
- Reusability
- Seperation of Concerns
- Modularized
- Simple/Reasonable
- Optimization

Team who Develops these frameworks

- Framework Dev Team
- Modernization Team
- Engineering Excellence Team (FW+AI)

2_Generic Functions (Framework Team)

Generic Functions

Def
Read_file(path,sep=',',infsch=True, header=True.....):
Df=Spark.read.csv(path,sep,...)
Return Df

Def
write_file(path,sep=',',infsch=True, header=True.....):
Df=Spark.read.csv(path,sep,...)
Return Df

Def
write_table(path,sep=',',infsch=True,header=True.....):
Df=Spark.read.csv(path,sep,...)
Return Df

Specific/Inline Functions

Standardize_shipment()

Standardize_staff()

Udf_functions()

%run

Dbutils.nb.run(path,secs,parameters)

1_Configuration (Framework/DE Team)

All Catalog/Schema/Volume/Table (parameters & variables)
SOURCE LOCATION
BRONZE LOCATION
SILVER LOCATION/TABLE
GOLD LOCATION/TABLE
Returning using dbutils.notebook.**exit**(all the local variables to parent notebook)

3_Bronze Main program

%run Function library nb

dbutils.notebook.run(Configuration nb,120,params)

Df1=**Read_file**(Driver_sourcefile1,sep='~',header=False)
Df2=Read_file(Driver_sourcefile2,sep=',',header=False)
Df3=Read_file(Driver_sourcefile1,sep='~',header=False)Geolocation3
Shipment.json

Write the final curated data into Datalake

4_Silver Main program

%run Function library nb
Write the function

%run Configuration nb

Writing ETL Programs
Munging
Enrichment
Customization
Curation

Write the final curated data into Datalake/Lakehouse(preferably)

5_Gold Main program

%run Function library nb

%run Configuration nb

Writing ETL+Analytical Programs
Curation
Wrangling

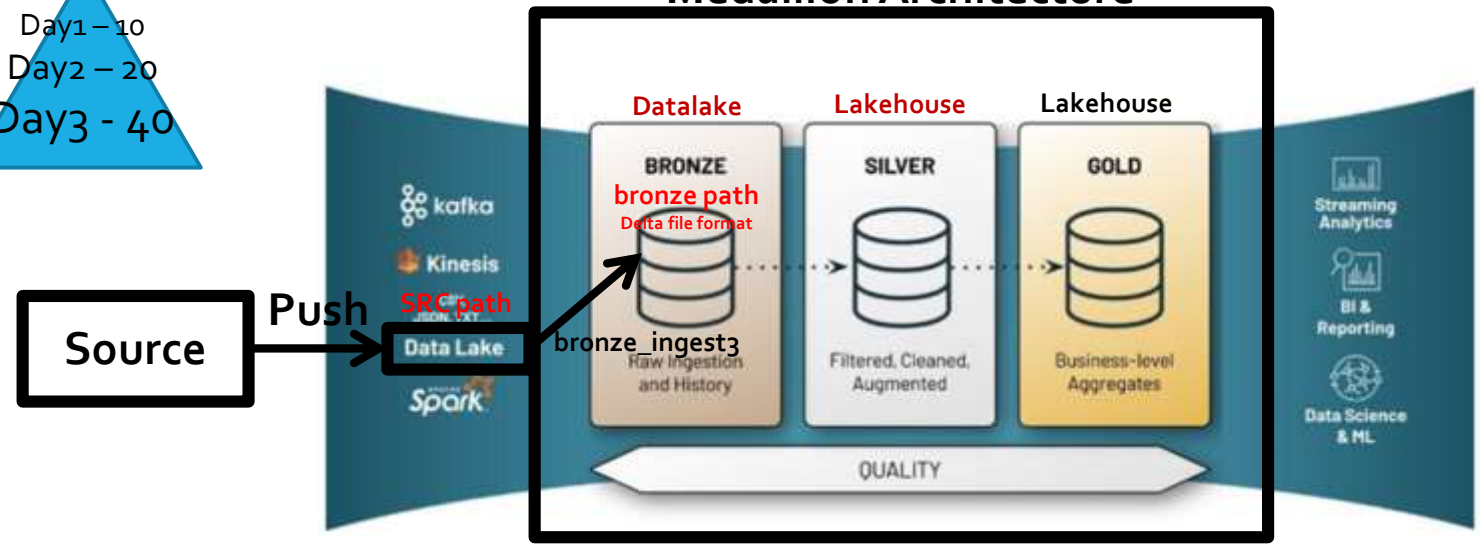
Generic Functions

Specific/Inline Functions

Main program

Day1 – 10
Day2 – 20
Day3 - 40

Medallion Architecture





Transformation from Datawarehouse to Medallion Architecture

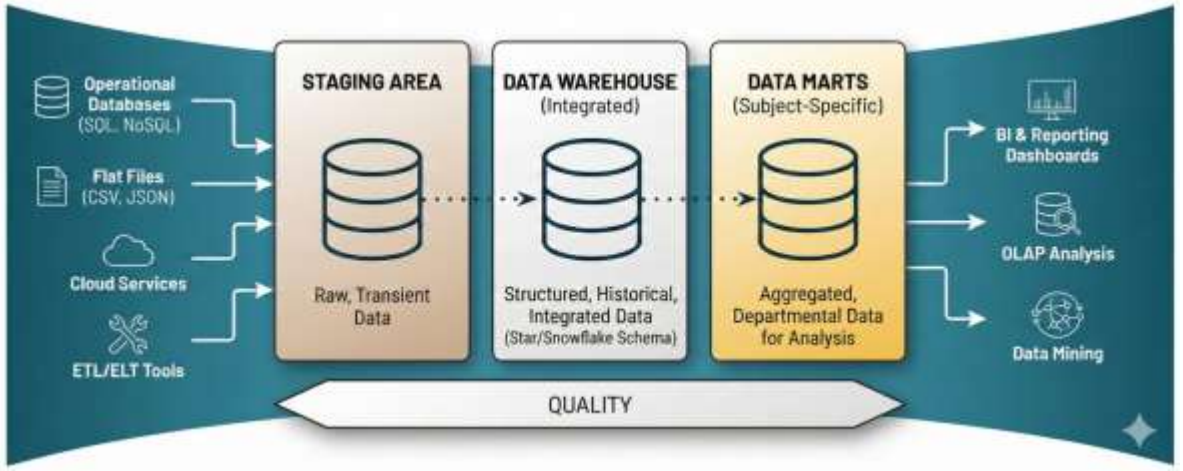
Medallion Architecture

The **Medallion Architecture** is a **layered data design pattern** used in the **Databricks(MS)** and modern data **lakehouses**. It organizes data as it moves from **raw** → **cleansed(curated)** → **business-ready(Wrangled)**, improving **data quality, governance, and performance+features**.

Key Benefits

- **Improved data quality** by progressively cleaning and validating data across layers
- **Clear separation of concerns** between raw ingestion, transformation, and analytics
- **Better performance** by serving pre-aggregated, optimized Gold tables
- **Scalability** to handle large batch and streaming workloads independently per layer
- **Full data lineage and auditability** from business metrics back to raw source data
- **Simpler debugging and reprocessing** by replaying data from Bronze or Silver layers
- **Schema evolution support** without breaking downstream consumers
- **Reusability of datasets** across multiple analytics, BI, and ML use cases
- **Cost optimization** by isolating heavy transformations and reducing repeated compute
- **Stronger governance and security** through controlled access at each layer

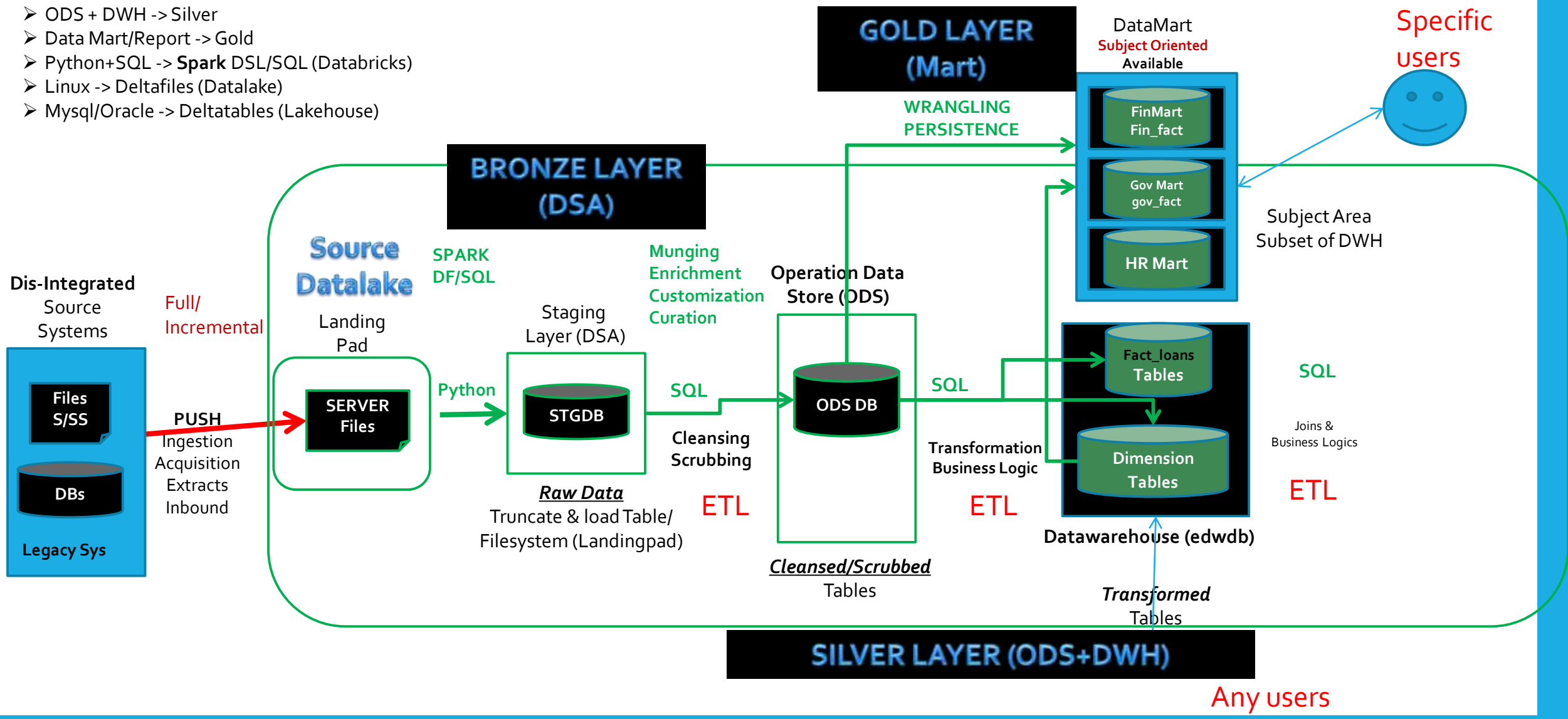




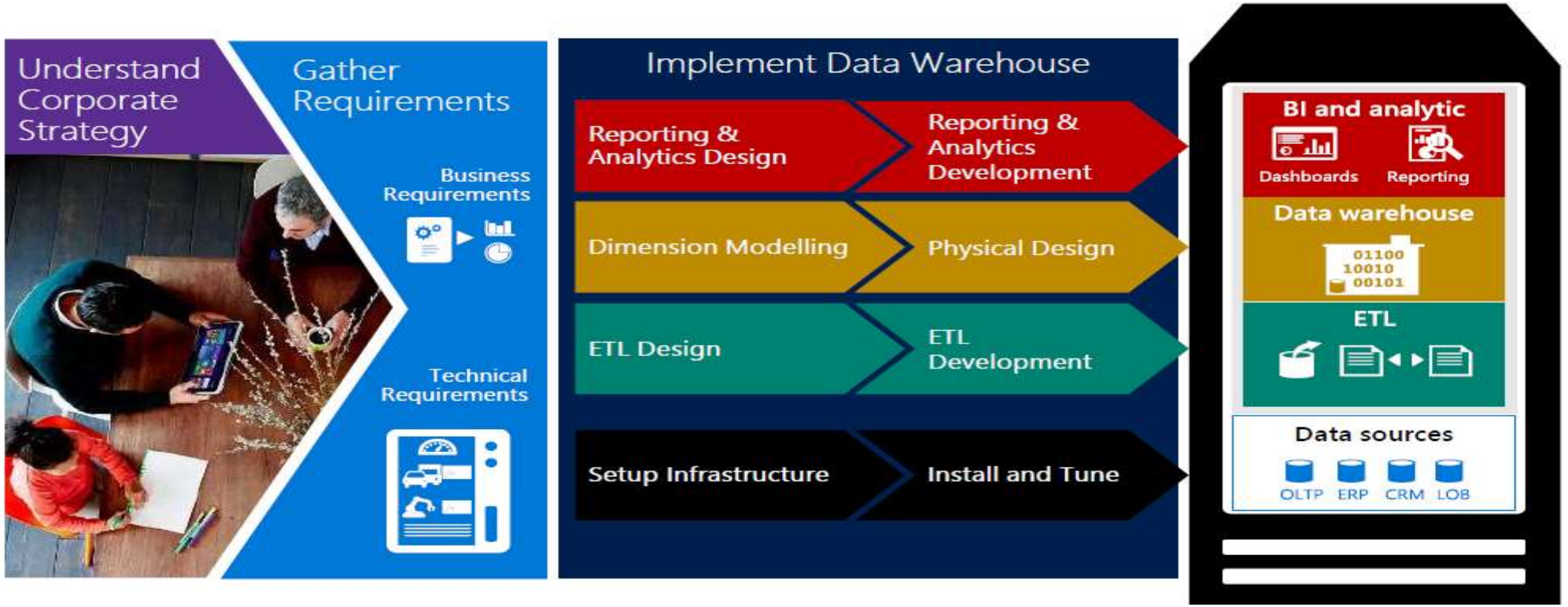
Modernization

- DWH -> Medallion Architecture
- ETL -> Curation (Dataengineering)
- DSA -> Bronze
- ODS + DWH -> Silver
- Data Mart/Report -> Gold
- Python+SQL -> **Spark** DSL/SQL (Databricks)
- Linux -> Deltafiles (Datalake)
- Mysql/Oracle -> Deltatables (Lakehouse)

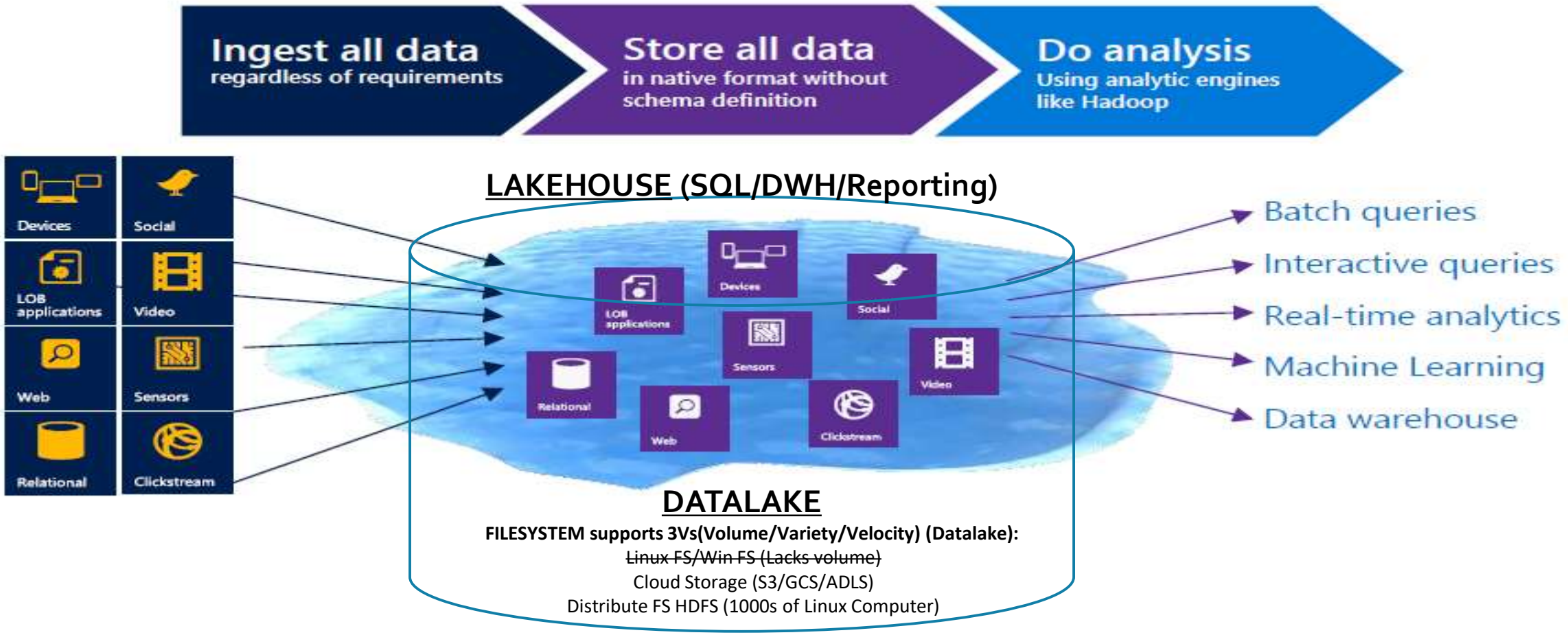
Datawarehouse ---> Medallion



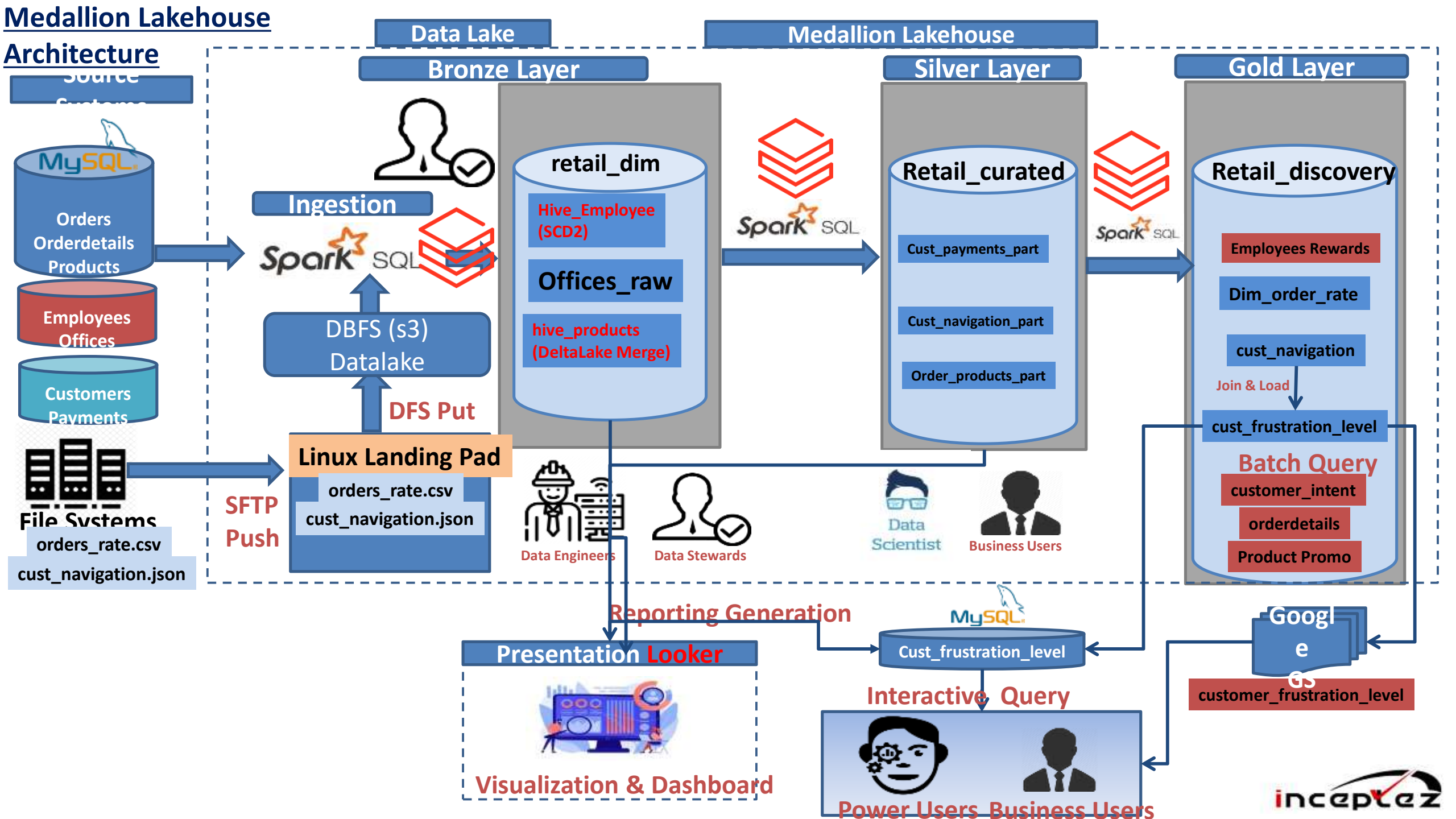
DWH (ETL)


Understand Corporate Strategy
Gather Requirements
Business Requirements
Technical Requirements
Implement Data Warehouse
Reporting & Analytics Design
Reporting & Analytics Development
Dimension Modelling
Physical Design
ETL Design
ETL Development
Setup Infrastructure
Install and Tune
BI and analytic
Dashboards
Reporting
Data warehouse
01100
10010
00101
ETL
Data sources
OLTP
ERP
CRM
LOB

DATA LAKE/Lakehouse (EL - T)



Medallion Lakehouse Architecture



Traditional DWH vs Medallion

Feature	Traditional Data Warehouse	Medallion (Lakehouse)
Approach	**ETL** (Transform before loading)	**ELT** (Load raw, transform after)
Data Types	Primarily structured (SQL tables)	Structured, semi-structured, & unstructured
Flexibility	**Rigid**: Schema must be defined upfront	**Flexible**: Schema can evolve over time
Storage Cost	High (expensive proprietary storage)	Low (cheap cloud object storage/S3/ADLS)
Primary Goal	Business Intelligence & Reporting	BI, Data Science, and Machine Learning
Use cases	• Pure BI & reporting • Stable schemas • Strong SQL-only workloads • Regulated legacy environments	• Real-time or near-real-time data • Streaming + batch together • ML / AI use cases • Cloud-first modernization • Cost optimization is important

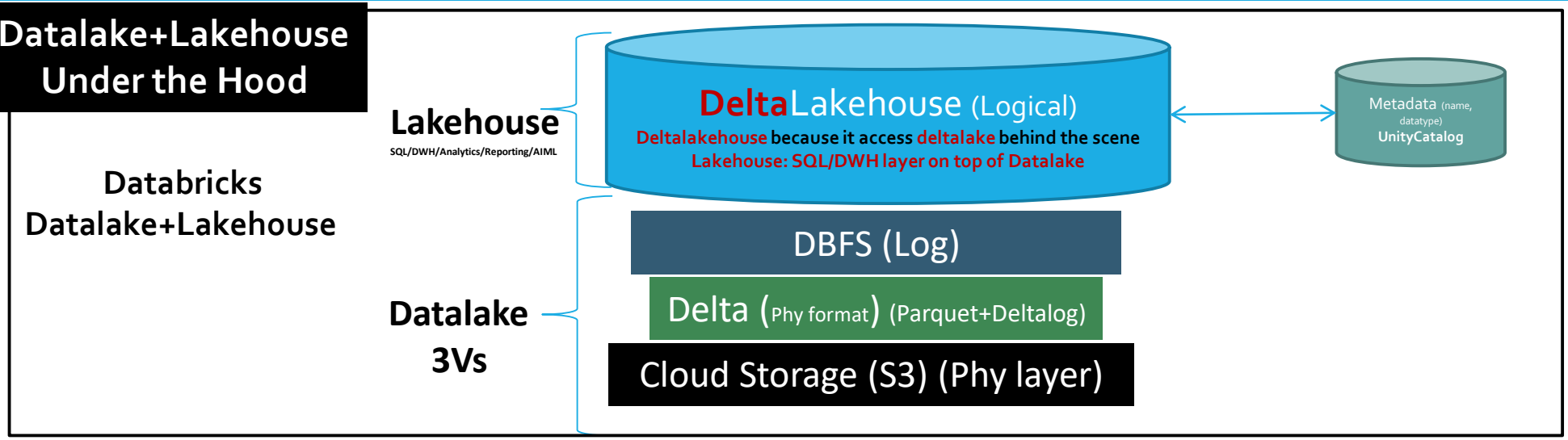
Medallion Architecture – 3 Layers

Name	Purpose	Data Characteristics	Typical Operations
Bronze (Raw)	Capture raw data exactly as received	• Append-only • Immutable • Schema-on-read • High volume	• Ingestion only • **CDC Handling** • Minimal parsing • No deduplication
Silver (Cleaned)	Clean, validate, and enrich data	• Deduplicated • Typed schema • Conformed data	• Data cleansing • Deduplication • Joins & lookups • **SCD handling**
Gold (Curated)	Business-ready analytics & KPIs	• Aggregated • Optimized • Stable schema	• Aggregations • KPI calculations • Dimensional modeling / Joins

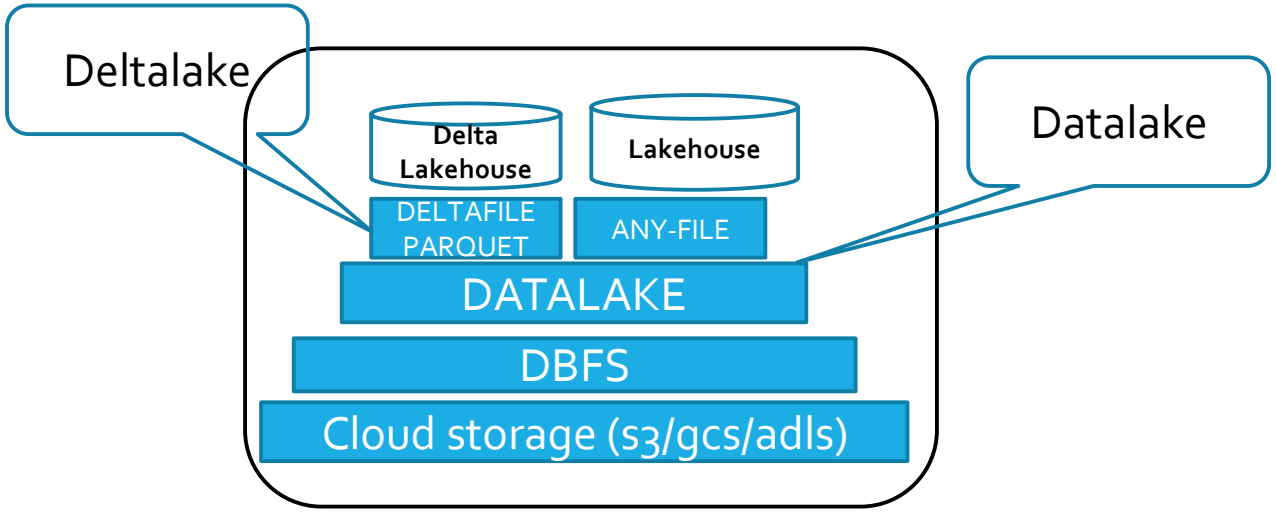
Datawarehouse Vs Medallion Layers (detailed view)

DW Layer	Medallion Equivalent	Primary Purpose	Data Characteristics	Typical Operations	Users / Consumers
Landing / Raw / Staging	**Bronze**	Capture source data exactly as received	• Raw, un validated • Schema-on-read • Duplicate records possible • Full history preserved	• Ingestion (batch/stream) • Minimal parsing • Metadata tagging • CDC capture	Data engineers
DSA (Data Staging Area)	**Bronze → early Silver**	Prepare raw data for standardization	• Lightly structured • Basic type casting • Still close to source • No business rules	• De-duplication • Basic cleansing • Standard timestamps • Partitioning	Data engineers
ODS (Operational Data Store)	**Silver**	Provide integrated, clean, near-current data	• Cleaned & conformed • Integrated across sources • Low latency • Current-state focused	• Business key resolution • Reference data joins • Soft deletes • SCD handling	Ops reporting, analysts
Enterprise Data Warehouse (EDW)	**Silver → Gold**	Store historical, enterprise-wide facts & dimensions	• Fully conformed • Historical (time-variant) • Subject-oriented • Trusted “single source of truth”	• Dimensional modeling • Fact table population • SCD Type 2 • Aggregations	BI developers, analysts
Data Marts	**Gold**	Optimize data for specific domains or teams	• Denormalized • Business-friendly • High query performance • Domain-specific	• Star / snowflake schemas • Business logic • KPI calculations	Business users, BI tools
Semantic / Cube Layer	**Gold (Consumption layer)**	Enable fast analytics & governed metrics	• Aggregated • Metric-driven • Cached / precomputed • Tool-optimized	• OLAP cube build • Metric definitions • Pre-aggregations • Security rules	Executives, dashboards

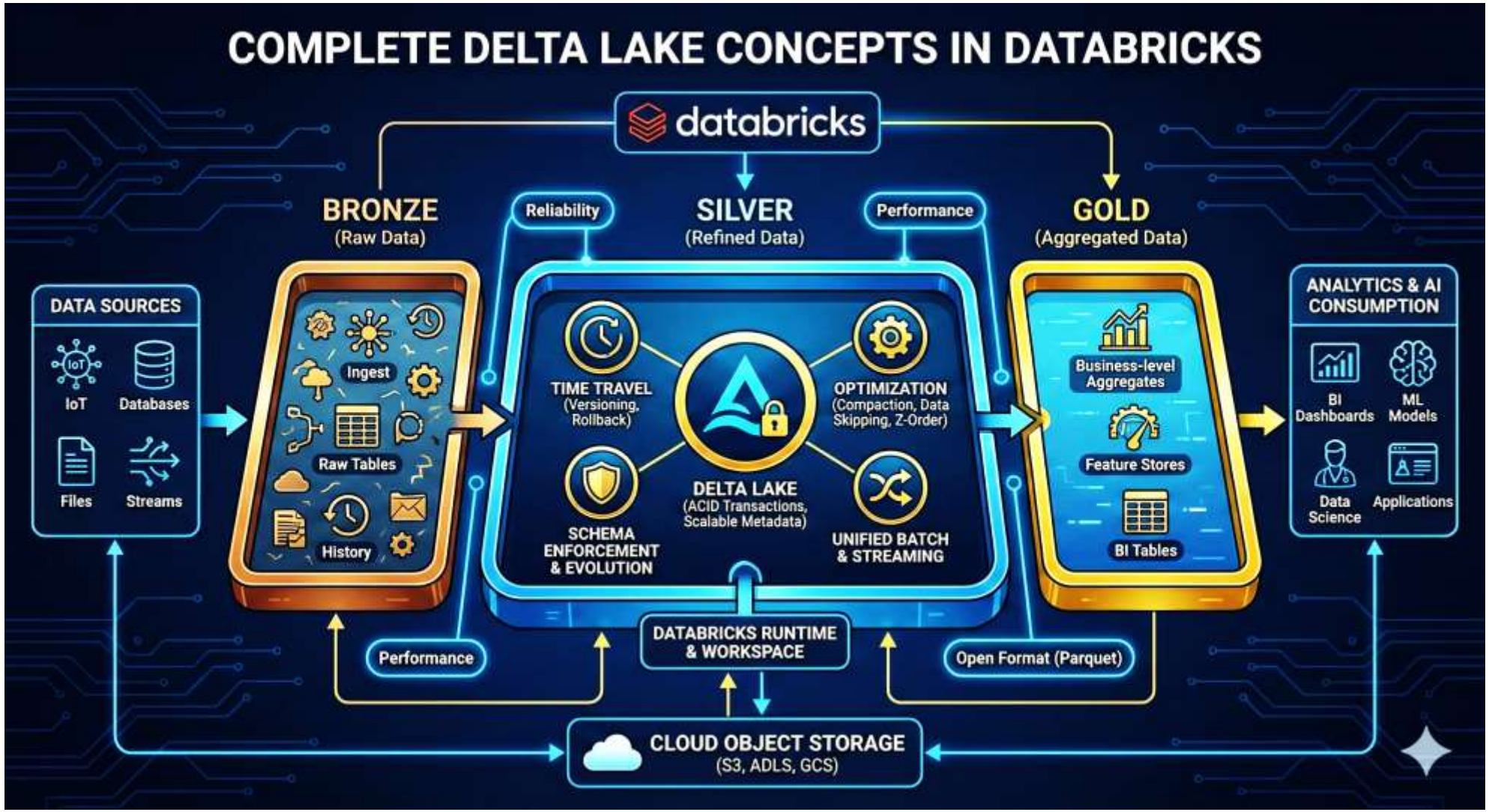
Datalake+Lakehouse Under the Hood



Datalake & Lakehouse Architecture (using Delta (Databricks))



Implementing Lakehouse + Medallion Architecture using Deltalake



Databricks – Delta Lake

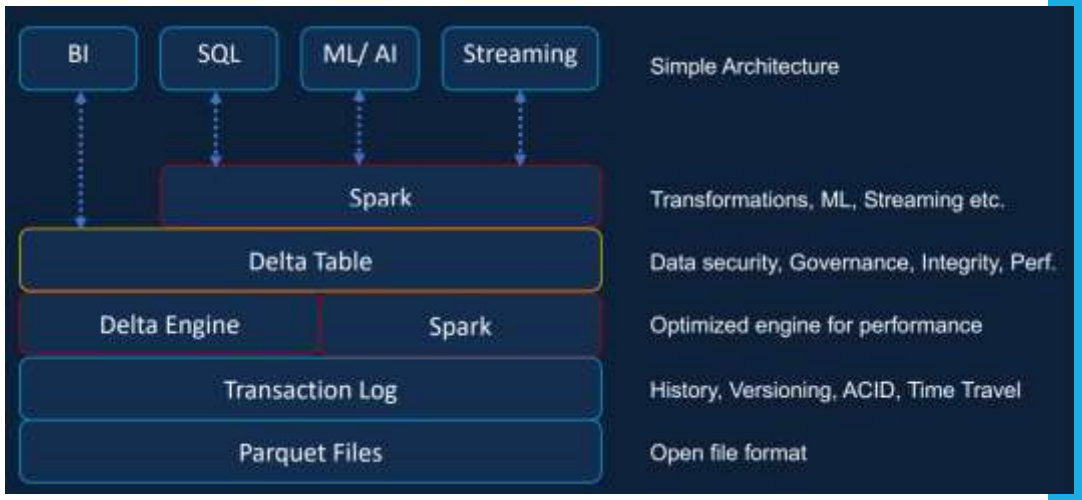
Delta Lake/Lakehouse is an **open-source storage layer** that sits on top of data lake (like **Azure Data Lake, S3,** or **GCS**) and brings **ACID transaction support, schema enforcement,** DML and **data versioning** to **Apache Spark and Databricks**.

Delta file/table Components:

1. **Delta file components:**
    1. **Data Files** – Stored as **Parquet** files in object storage of Cloud.
    2. **Transaction Log (_delta_log)** – **JSON** , deletion vector and checkpoint files that record all table changes.
2. **Delta Table components:**
    1. In addition to the above 2 deltafile components **if we add metadata layer** (**Unity Catalog**)

    ie. If some system/db is going to maintain the table name, column name, datatype, governamce, security, quality and other features of the data.

Feature	Description
ACID Transactions	Guarantees Atomicity, Consistency, Isolation, and Durability, even with concurrent writes.
Schema Enforcement & Evolution	Ensures data adheres to the expected schema, while allowing safe schema updates.
Time Travel	Lets query older versions of data (for debugging, audit, rollback).
Merge, Upserts & Deletes	Supports MERGE, UPDATE, and DELETE directly on large datasets.
Data Versioning	Every change creates a new version tracked in the _delta_log.
Optimized Performance	Uses file compaction, data skipping, and caching for faster reads/writes.



Databricks – Optimization on Delta Lake

Concept	Purpose / Function	When to Use	Example Syntax
OPTIMIZE	Rewrites small files into **larger contiguous** files. *Small files of large numbers to large files of small numbers.* ***Compaction traditionally***	✓ Improve query performance ✓ Efficient metadata management	**OPTIMIZE delta_table;** 10 rs coins of 10 numbers → 1 hundred rupees note
Z-ORDER	**Co-locates** related data physically (by **ordering**) for faster filtering including indexing. Eg. Bringing all remotes in my desk together physically, if any one wanted remote, they will just search in the remote (chunk) using index **(R%)** internally maintained.	✓ After table creation/loading data, run this frequently ✓ Filter performance is improved by reducing the search cycle.	**OPTIMIZE delta_table ZORDER BY (customer_id);** 10 rs, 50 rs, 20 rs, 100 rs →10 rs, 20 rs, 50 rs, 100 rs
Partition By	Horizontal **division/segregation** of data in a form of **folders**	✓ For organizing data efficiently ✓ For improving query performance while filtering using low cardinal column.	**Df.write.partitionBy("cols").saveAsTable("")** 10 rs, 20 rs, 50 rs, 100 rs in one partition of my wallet 10 rs, 20 rs, 50 rs, 100 rs in seperate partitions of my wallet
CLUSTER BY	**Liquid Clustering** lets you declare clustering columns once using CLUSTER BY, and Databricks ***automatically and incrementally reorganizes data files over time***, eliminating manual optimize, partitioning and ZORDER. Simply, ***Cluster by is the combination of all 3***	✓ liquid clustering is a relatively new, revolutionary feature in Delta Lake 3.0 (introduced around late 2023) **designed to replace traditional optimize, partitioning and Z-Ordering** for data layout optimization. ✓ It enables ***automatic, flexible, and incremental data clustering, allowing users to recluster data without rewriting the entire table.***	All the above 3 are achieved in a single feature incrementally… CREATE TABLE t **CLUSTER BY (customer_id)** cols datatype/select * from tablename;
VACUUM	Really - Deletes old/unneeded files to free storage after some period of time, default 168 hours (1 week)	Periodic cleanup of unwanted data for best practice to follow, for space & cost saving.	**delta_table.vacuum(retentionHours=168)** Example: We clean our old mails in the mailbox
CTAS	Create table from a query results (Create table as Select using any query logic) – use only for some transformed **data** copy from one table to another. Operate in the background (resource intensive): data from source disk -> memory -> writtened to the target table disk	When you are merely copying a table from another table (not a best practice, not for saving space & not for cost saving) Drawback1: Entire data movement from table1 to 2 will happen (costly operation). Occupies more space Drawback2: Only data is copied and not the entire metadata of the source table is copied. For eg. If the source table has clustering, the target table will not have cluster.	CREATE TABLE new_table AS SELECT * FROM old_table; -- not good CREATE TABLE new_table AS **SELECT col1,col2 FROM old_table where col=condition; -- good (copy only data)**
Deep CLONE	Full copy of table (data + **metadata**) - Operate in the background (not resource intensive): data from source disk -> writtened to the target table disk	**Deep Clone –** Help us copy the entire table data + metadata also (drawback2 is eliminated), Drawback1 still exists. Conclusion: Use deep clone if you are going to merely copy a table from another table including metadata.	CREATE TABLE clone_table CLONE original_table; (copy data and metadata)
SHALLOW CLONE	Copies only metadata; points to same underlying files	Shallow Clone – Help us not literally copy rather refers the same underlying files by both the tables with the logs maintained individually. Lightweight duplication (best practice to follow, for space & for cost saving)	CREATE TABLE shallow_clone_table **SHALLOW CLONE** original_table

Databricks – Partitioning in Spark

In Databricks, **data partitioning** means organizing large datasets into **directory structures** based on column values so that Spark can **read only what it needs** instead of scanning everything.

- A data partition is a subdirectory of your dataset created based on the distinct values of a column.
- Spark uses these folders to skip irrelevant data when you query with filter (known as partition pruning).

Key Benefits of Partitioning

- **Faster queries** – Spark reads only relevant partitions instead of scanning the entire dataset.
- **Reduced I/O** – Minimizes data read from storage, saving time and resources.
- **Faster incremental loads** – Enables quick updates or inserts for specific partitions.
- **Better parallelism** – Allows Spark to process partitions independently for faster execution.
- **Optimized filtering** – Queries with partition column filters become highly efficient.

/Volumes/inceptez_catalog/outputdb/customerdata/

profession=Pilot/

profession=Teacher/

profession=Lawyer/

profession=Firefighter/

…

Best Practices in Partitioning

- Use low-cardinality columns – Partition on columns with relatively few unique values (e.g., year, region).
- Keep partition size balanced – Aim for 100 MB to 1 GB per partition file for optimal performance.
- Limit the number of partitions – Too many partitions can increase metadata overhead and slow queries.
- Use dynamic partition overwrite – Overwrite only affected partitions during incremental loads.
- Optimize and vacuum regularly – Compact small files and clean up old data in Delta tables.
- Partition based on query patterns – Choose columns that are frequently used in filters.
- Avoid over-partitioning small datasets – Small datasets do not benefit from heavy partitioning.

Databricks – ORC vs Parquet vs Delta

Feature	ORC	Parquet	Delta
Type	Columnar file format	Columnar file format	Transactional layer on Parquet
Ecosystem	Hive, Presto	Spark, Databricks	Spark, Databricks
Read Performance	Very fast (Hive)	Fast	Fast + optimized
Write Performance	Slower	Faster	Slightly slower (logs)
File Size	Larger	Smaller	Slightly larger
Schema Evolution	Limited	Good	Excellent
Predicate Pushdown	Yes	Yes	Yes
ACID Transactions	No	No	Yes
Time Travel	No	No	Yes
Merge/Upsert	No	No	Yes



Databricks Components
Lakeflow

1. LF Ingestion pipeline
2. LF Declarative Pipeline (DLT) - Transformation & Analytics
3. LF Jobs - Orchestration & Scheduling

Databricks Lakeflow Components Flow

Databricks

Lakeflow

Ingestion

ETL
(Declarative/DLT)

Jobs

Cloud storage
Dir listing/file notify

Auto-loader
Filebased

Native Connector
Dozen of app sources

Partner Connector
Huge set of 3rd party sources

Custom Connectors
API/JDBC connectors

API Layer
Custom Python API

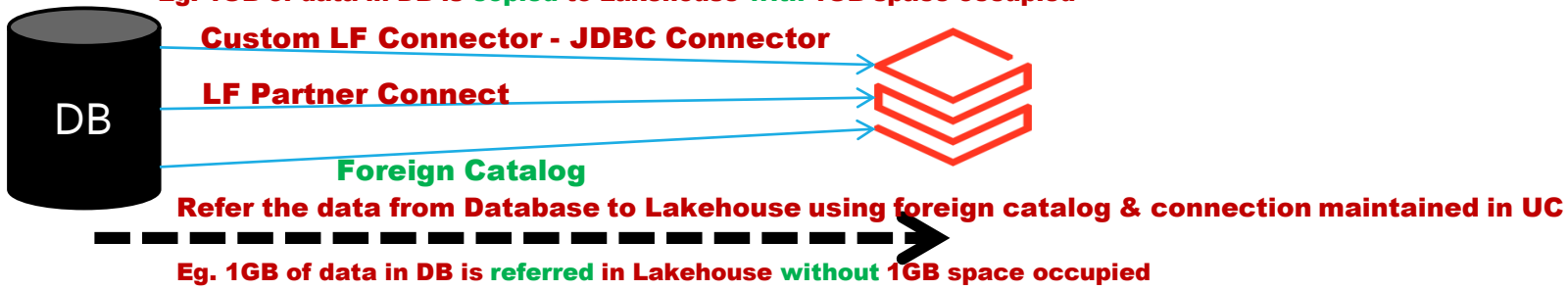
Declarative Pipeline

Expectations & Lineage & Logs

CDC

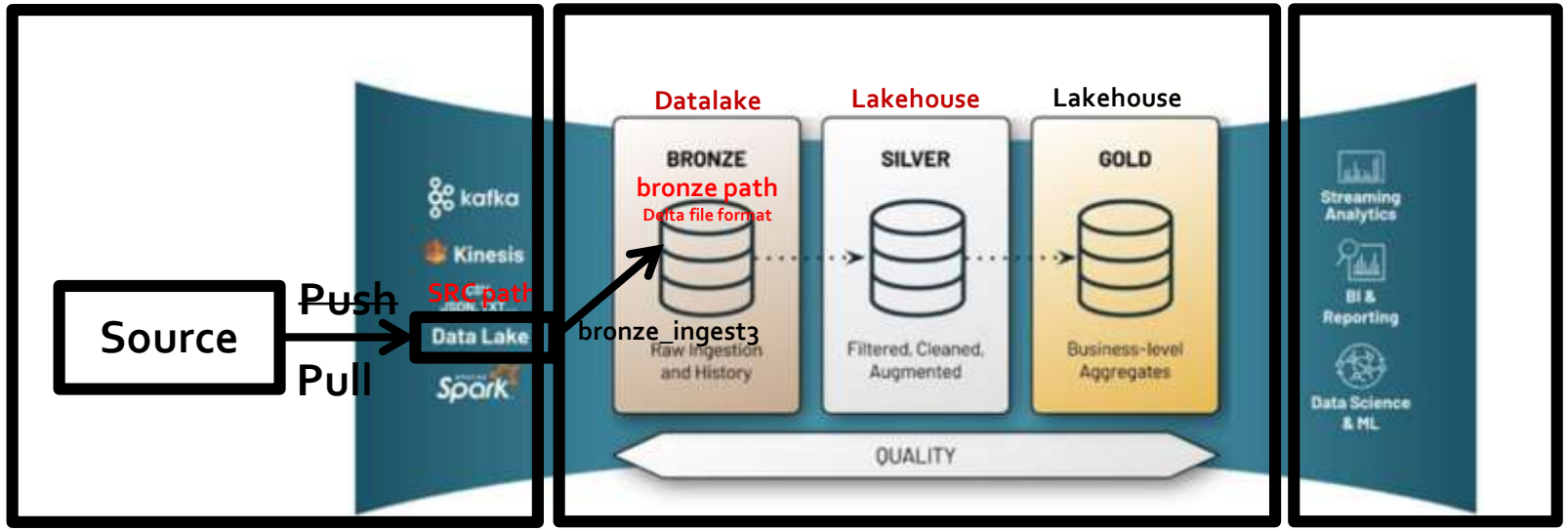
Medallion

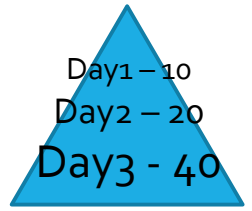
Unity Catalog
Foreign Catalog
Database Connection



Futuristic Databricks (Agnostic) Lakeflow Components

Medallion Architecture





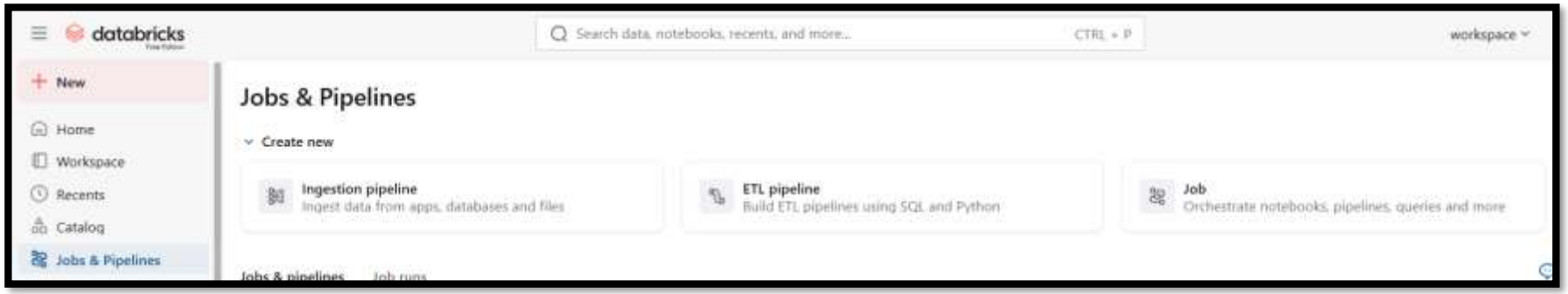
1. Lakeflow Ingestion pipeline **Lakeflow Declarative Pipeline**
DLT Pipeline

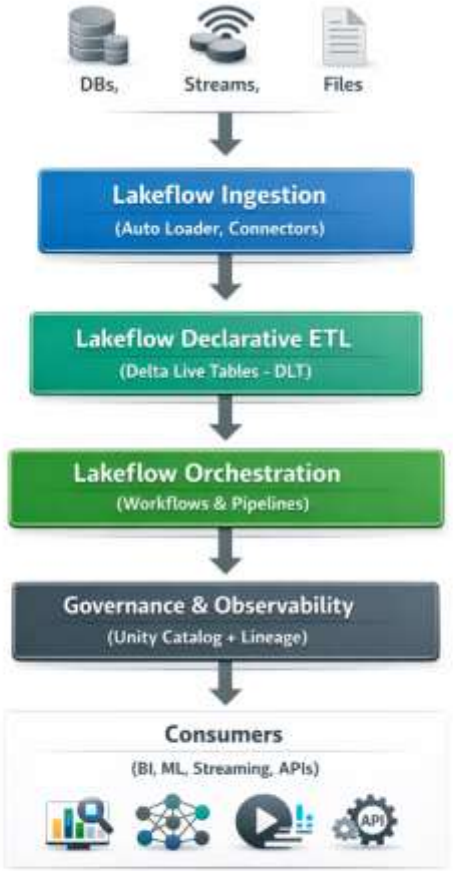
Lakeflow Jobs – Orchestration & Scheduling component of Databricks

Databricks Lakeflow Components

Lakeflow is Databricks' **unified data engineering platform** that brings together:

- ✓ **Lakeflow Ingestion:** Scalable intake from DBs, streams, SAAS,API, CRM and files using efficient ***Auto Loader*** patterns.
- ✓ **Declarative ETL (DLT):** Simply define data transformations with Delta Live Tables; the platform handles the complexity.
- ✓ **Orchestration:** Integrated workflow management to schedule and monitor complex pipelines.
- ✓ **Governance & Observability:** Unity Catalog provides a single layer of security, lineage, and discovery.
- ✓ **Consumers:** Seamless serving to BI, ML models, and real-time streaming applications.







1. Databricks Lakeflow Ingestion

1. Databricks Lakeflow Ingestion

Lakeflow Ingestion is the **data intake layer** of Databricks Lakeflow that enables **reliable, scalable, and governed ingestion** of data from multiple sources into the Datalake/Lakehouse.

1. Lakeflow Ingestion supports:

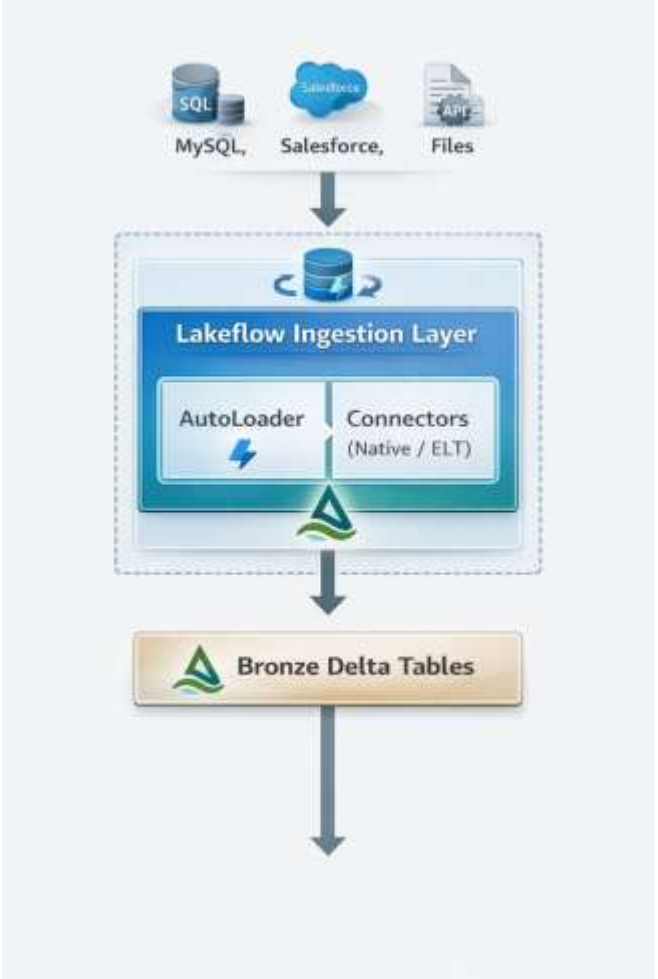
- Batch ingestion(free)
- Continuous Streaming ingestion (cloud paid)
- Incremental & **CDC ingestion** (free/cloud)
- External sources – Eg. SaaS, API and file-based sources(free)

Primary goal: land raw data into Bronze Delta files/tables.

2. Lakeflow Ingestion Mechanism

a. **Auto Loader** (**Free**/Cloud)
b. **Native Lakeflow Connectors (Free)**
c. Partner-based ELT (Fivetran)/Azure **Data factory** (Cloud)
d. **Custom ingestion** (**APIs** / JDBC) (**Free**/Cloud) – Python library/Connection+Foreign Catalog

Lakeflow Ingestion Architecture



1. Databricks Lakeflow Ingestion (File based Auto loader)

A. Auto Loader (File-Based Ingestion)

Auto Loader is Databricks' **cloud-native file ingestion engine** for ingesting **new files incrementally** from cloud **object** storage into Databricks Bronze Layer.

Notebook: lakeflow_ingestion_cloudfile_autoloader_1

Supported Sources

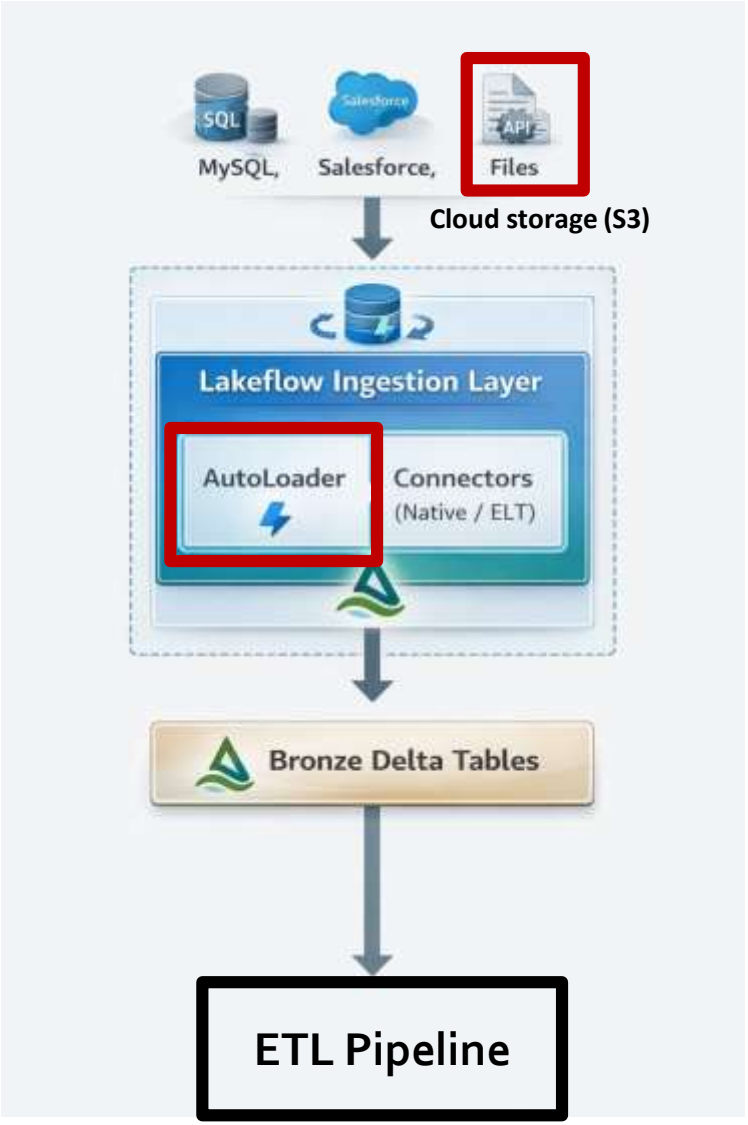
- AWS S3
- Azure ADLS Gen2
- Google Cloud Storage (GCS)

Modes:

- **Directory listing** (every iteration of LF Ingestion job does the cloud directory listing and any newly added files will be autoloaded into bronze layer incrementally)-works in free edition
- File notification (every file is created in the cloud storage will trigger the LF Ingestion job and any newly added files will be autoloaded into bronze layer incrementally)- doesn't work in free edition

Eg.

```
df = (spark.readStream
   .format("cloudFiles")
   .option("cloudFiles.format", "json")
   .option("cloudFiles.schemaLocation", "/schemas/orders")
   .option("checkpoint","/////")
   .load("/catalog/schema/"))
```



1. Databricks Lakeflow Ingestion (Custom ingestion + Auto loader)

B. Custom ingestion + Auto Loader (API -> File-Based Ingestion)

Auto Loader is Databricks' **cloud-native file ingestion engine** for ingesting **new files incrementally** from object storage.

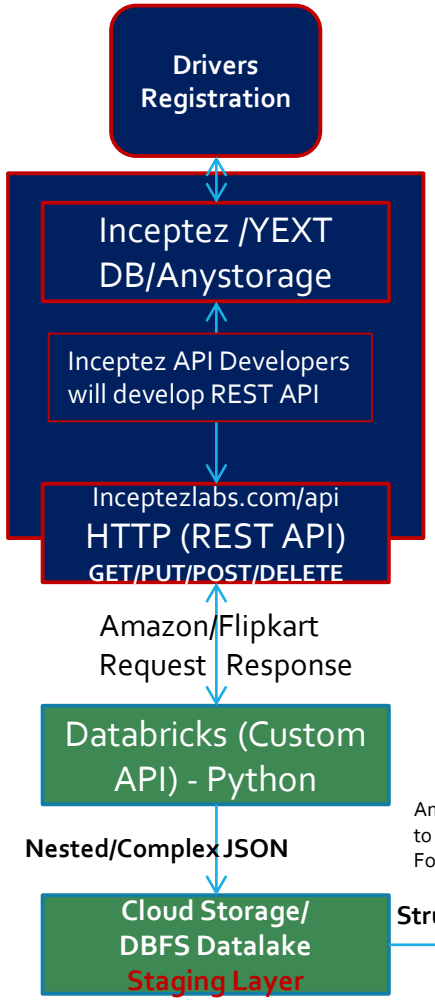
lakeflow_ingestion_api_autoloader_1

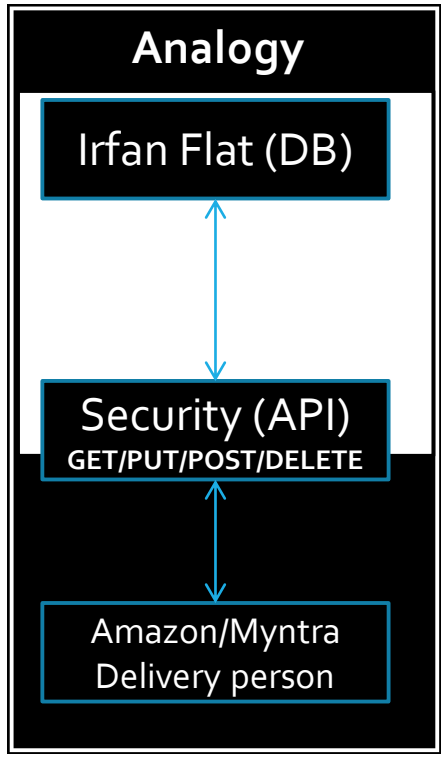
Supported Sources

- AWS S3
- Azure ADLS Gen2
- Google Cloud Storage (GCS)

Modes:

- Directory listing
- File notification







1. Lakeflow Custom Ingestion

Amz/Flipkart **Data Ingestion Developers (ELT)** to develop a "Lakeflow Custom + Autoloader Ingestion Pipeline" For ingesting API data into Bronze layer of our Datalake

Structured

Bronze Layer

2. Lakeflow Autoloader

Silver Layer

Gold Layer

Amz/Flipkart **Databricks Data Engineers (ETL)** to develop a "Lakeflow Declarative/DLT/ETL Pipeline" For transforming data from Bronze to Sil to Gold...

1. Databricks Lakeflow Ingestion (Native Connectors)

C. Native Lakeflow Connectors

Lakeflow connectors are **Databricks-managed ingestion connectors** that directly connect to **operational systems, CRM and SaaS platforms.** They reduce custom ingestion code and are **natively governed (Security/Standards etc.,)**.
Salesforce is a Software As A Service (SAAS) platform to offer SMS (Sales/Marketing/Service) lifecycles.

Salesforce Ingestion using **Lakeflow Native Connector**

1. Generate Free Account & security token from Salesforce
2. Create Salesforce Connection in Databricks (seamless)
3. Configure Salesforce Objects for Ingestion using developer mode and ingest data.
4. Automatically the newly added data is going to be collected

using connection in databricks

5. Follow the detailed instruction given below to achieve the ingestion.



Microsoft Word Document





Databricks Lakeflow Ingestion is completed...

We are achieving Data ingestion without using Lakeflow Ingestion feature, Rather we use Connection + Foreign Catalog of UNITY CATALOG



Databricks Federation
Foreign Catalog & Connection

Connections

A **Connection** defines:

- **How** Databricks **authenticates** to an external system
- **Where** the external system is (host, port, database) if we create DB Connection, If we create SAAS connection we use OAuth methodology.
- **Which credentials** to use (stored securely)
- It **does NOT store data** — it only stores **connection metadata**.

Why Connections Are Needed

- Avoid hard-coding usernames/passwords in notebooks or for avoiding authentication again and again in a short interval(one time authentication and expiration time can be set).
- Enable **centralized governance** via Unity Catalog
- Allow **multiple users and tables** to reuse the same connection
- Support **SQL-based external access**

Database Connections

Used to connect to external RDBMS systems.

Examples:

- MySQL
- PostgreSQL
- SQL Server
- Oracle
- Snowflake

Use cases:

- Query external tables
- Create **Foreign Catalogs (to reference/lookup tables)**
- Read/write data via Databricks SQL

Foreign Catalog

A **Foreign Catalog** is a **Unity Catalog object** that allows **Databricks to reference and query metadata that lives outside Databricks**, in an **external metastore or database**, **without copying the data**.

It acts as a **logical bridge** between Databricks and an **external system's catalog**.

- Databricks does **not own the data, but copy only metadata (hence light weight).**
- Databricks **does not manage the lifecycle** **(of deltalog, deletion vector etc.,)**
- Only **metadata pointers** are registered

Use Foreign Catalog when:

✓ **Ad-hoc analysis**
✓ Alternative for Custom JDBC or 3rd party connectors **(for data copy)**
✓ **Incremental Ingestion** (I just need only data from yesterday)
✓ No need to persist data (If we only want to refer/lookup)
✓ **Real-time lookup** (If any changes made in source DB, will be reflected)
✓ Avoid data duplication (Acts like a shallow copy (without snapshot))

Aspect	Foreign Catalog	External Location
Data location	External database	External cloud storage
Primary Use	Cross-platform querying	Storing large-scale Delta/Parquet files
Storage type	RDBMS / external metastore	ADLS / S_3 / GCS
Access method	Federation / JDBC	File-based

Declarative (What + ~~How~~)
from pyspark import **pipelines**
@Add some decorator provided by DLT

Imperative (What + How)
Pyspark Framework
Python/SQL Transform

I just develop Pyspark (Py/SQL) Ingestion + Transformation Program

Send the final function dataframe to DP Just adding decorator

I will add a Decorator on top of Imperative program, so Databricks can take care of all other activities such as:

1. Data Quality
2. **Data Lineage**
3. **CDC/Incremental/SCD**
4. Governance
5. **Batch/Realtime Streaming**
6. Failure management
7. **Logging** & Scheduling
8. **Performance Optimization**



2. Databricks

Lakeflow ETL (Declarative/DLT) Pipeline



Lakeflow Declarative Pipeline for ETL

What is Databricks? It is an entire Data Platform (Ingestion, Engineering, Orchestration & Scheduling, Analytics, BI, AI, ML)

What is Declarative Pipeline? A Strategy/Concept/Paradigm to define the way of writing standard/modern DE (imperative) pipelines,

where we just declare **WHAT** the data should look like, not **HOW** to perform (Dataquality, Lineage, CDC, SCD, Goverance, Batch/Realtime) as we do in traditional **Imperative** Pipelines manually.

What is Delta Live Tables (DLT) ? A product/tool of Databricks that implements Declarative pipelines

Think of DLT as **"Spark Batch + Structured Streaming + Delta + orchestration + data quality"** bundled and simplified.

Feature	Traditional Spark ETL (Imperative Pipeline)	Delta Live Tables (Declarative Pipeline)
Orchestration	Manual (Airflow, Jobs)	**Automatic** (Dependency Graph)
Data Quality	Custom validation code	**Native** (Expectations)
Infrastructure	Manual Cluster Config	**Fully Managed / Serverless**
Lineage	Manual or 3rd Party	**Built-in Visual Graph**
Error Handling	Custom retry logic/logging mechanisms	**Automatic recovery/tracking** (internally state management) – exactly once semantic (no duplication)
Complexity	High	Low

Core Components of DLT

- **Tables (Live/Streaming, mat view, temp view)**
  - Declarative tables defined in **Python / SQL**
  - DLT manages execution, retries, and dependencies
- **Batch & Streaming Support (Unified)**

  spark.read or spark.readStream
- **Data Quality Expectations**
  - Built-in rules for validation
  - expect, expect_or_drop, expect_or_fail
- **Automatic Dependency Management**
  - No manual job orchestration (DAG is automatic)
- **Incremental Processing**
  - Automatic checkpoints & state handling
  - No offset/watermark management required
- **Managed Orchestration, Scheduling & Monitoring**
  - Built-in UI for lineage, metrics, failures
  - Supports scheduled (Triggered) & continuous runs
- **Delta as the Only Output**
  - Always **writes Delta** tables (stream table/mat views) but **reads any** catalog tables

Before learning about Declarative DLT pipeline
Let's just learn about a python concept called DECORATORS

```
def databricks_decorator_declarative_pipeline(fun):
    def wrapper():
        print("I am now decorating your imperative function to declarative function by adding dataquality, lineage, batch/realtime unification, no cdc/scd...")
        fun()
    return wrapper


@databricks_decorator_declarative_pipeline
def imperative_function():
 print("Vasu - I am running my own spark program without adding dataquality, lineage, batch/realtime unification, no cdc/scd")

imperative_function()
```

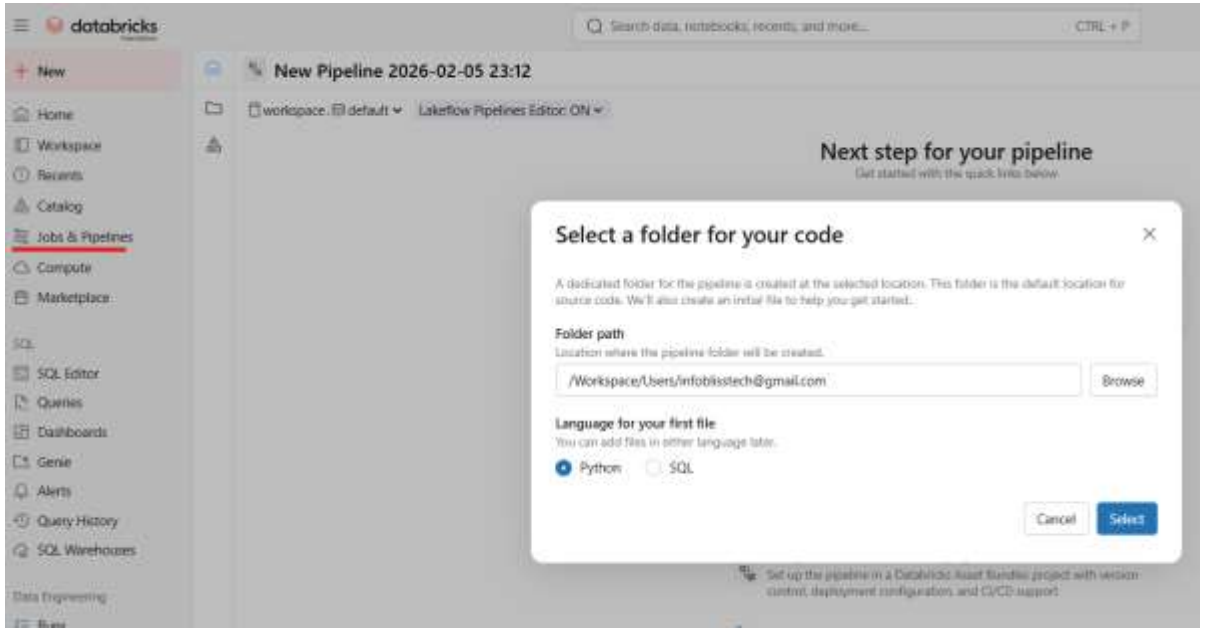
Declarative Pipeline DLT Basics

1. How to write a DLT Pipeline? Goto Workspace -> Create a Folder under git -> Create -> Choose ETL Pipeline -> rename -> Start with an Empty File -> Python/SQL

Python Code: lakeflow1_dp_python

```
from pyspark import pipelines as dp

@dp.table()
def drugstbl_bronze_tbl():
  return spark.readStream.table("lakehousecat.deltadb.drugstbl")
```



Explaination of the Code:

from pyspark import pipelines as dp --------- **3.** Imports the **Lakeflow Pipelines (Declarative Pipelines) Python Decorator function**

@dp.table() --------- 4. The function below defines a **table** managed by a Lakeflow pipeline (with the name we paramterized, if we don't mention the name it will take function name as the name of the table)

def drugstbl_bronze_tbl(): ------- 1. The function name becomes the **materialized view name**

return spark.read.table("lakehousecat.deltadb.drugstbl") ---------- 2. Reads an existing **Delta table/any other tables** from Unity Catalog

SQL Equivalent: lakeflow_dp_sql_2

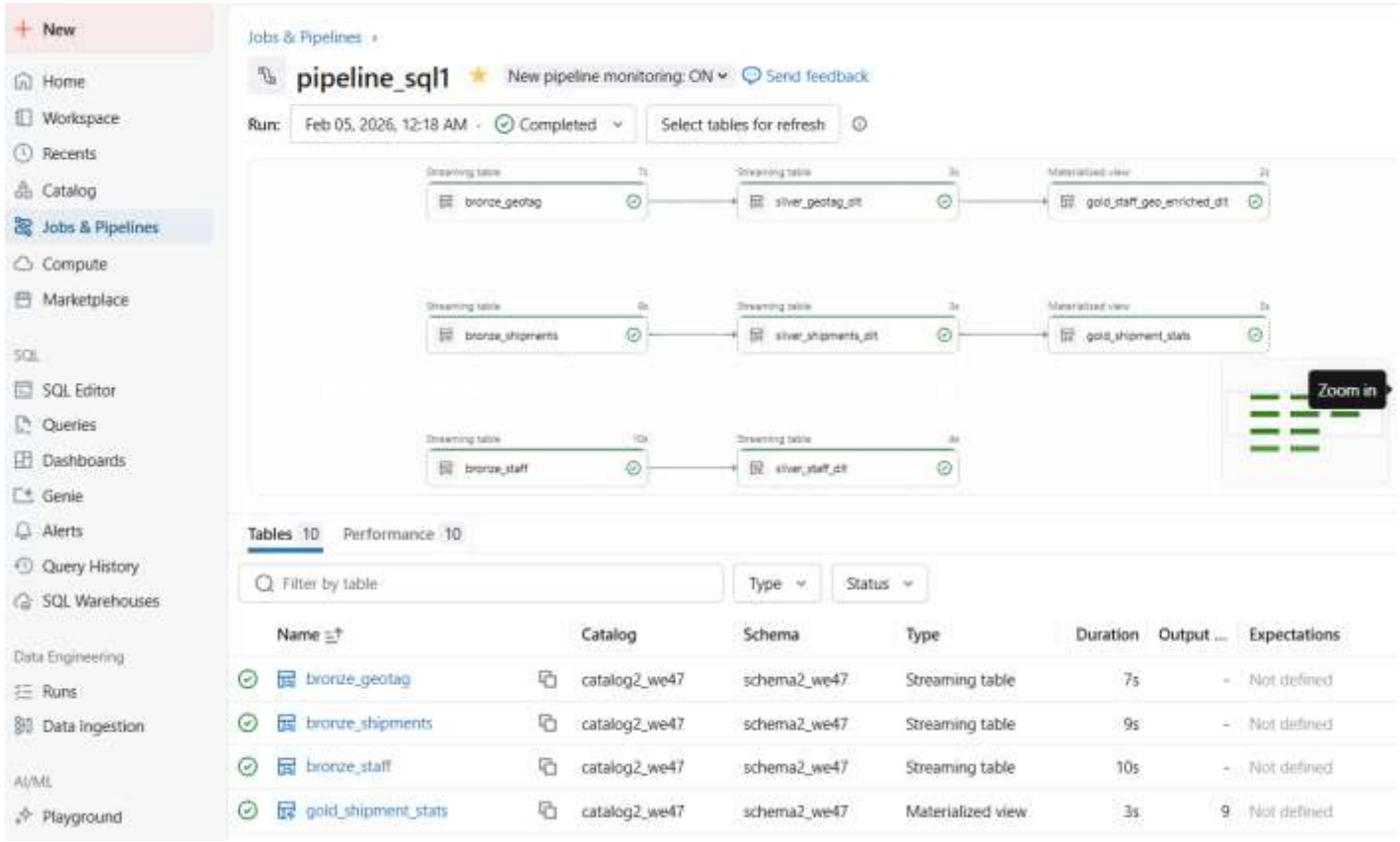
```
CREATE OR REFRESH STREAMING TABLE catalog1_dropme.default.drugstbl_bronze1
AS
SELECT *
FROM STREAM(lakehousecat.deltadb.drugstbl);
```

Declarative Pipeline DLT Basics

2. DP (DLT) DAG & Lineage (How part)

Check the followings:

1. Pipeline Details
2. Run Details
3. Compute Details/Logs
4. Columns
5. Table Metrics
6. Performance Metrics



Declarative Pipeline DLT Basics

3. Streaming vs Batch

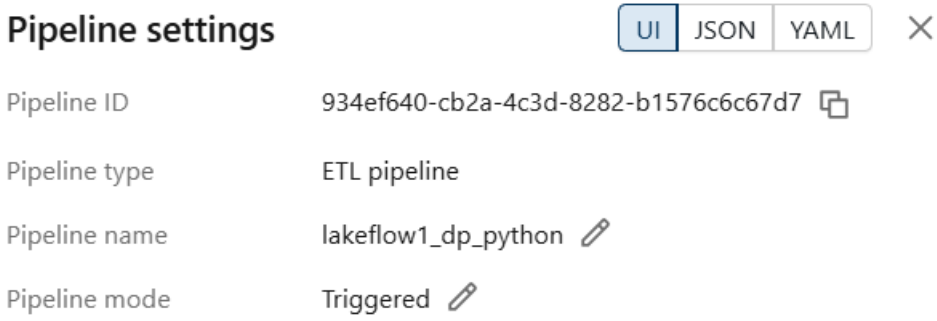
DLT supports both batch and streaming with the SAME code (Change settings -> Pipeline Mode (Triggered/Continuous)

```
#Ins/Upd/Del in the target Delta table using Mat View
from pyspark import pipelines as dp

@dp.table(name="catalog1_dropme.default.drugstbl_batch_bronze1_matview")
def drugstbl_bronze_tbl():
  return spark.read.table("lakehousecat.deltadb.drugstbl")
```

```
#Ins in the target Delta table using Streaming table
from pyspark import pipelines as dp

@dp.table(name="catalog1_dropme.default.drugstbl_stream_bronze1_streamtable")
def drugstbl_bronze_tbl():
  return spark.readStream.table("lakehousecat.deltadb.drugstbl")
```





Only Types of Live Tables – Delta Live Tables (DLT)

Feature	Streaming Table	Materialized View	View
Data Storage	Persisted to Disk	Persisted to Disk	No (Logical Only)
Processing	Incremental (Only new data) – **Bronze layer (SCD2)**	Refresh (Full or Incremental) – **Bronze layer(SCD1)/Gold Layer(Reporting/BI/AI/ML)**	If we need On-the-fly transformation – **Silver Layer**
Visibility	Public in Catalog	Public in Catalog	Internal to Pipeline
Primary Use	High-speed Ingestion (only incremental)	Slightly lesser performance than stream table BI & Reporting	Internal transformation (Performant if used once (since no storage is used)/Not accessible hence we have to create mat view/stream table - if we use this data multiple times)

Implementing Medallion Architecture using DP (DLT)

Which one should use, and when?

- **Streaming Tables**

  Use streaming tables to ingest data into the data lake **quickly, continuously, and cost-effectively**.

- **Materialized Views**

  Use materialized views to **transform, enrich, and aggregate** data into business-ready datasets for analytics and reporting.

- **Views**

  Use views to **organize your logic**, simplify pipelines, and **hide complex intermediate transformations** from end users.

Declarative Pipeline DLT Basics

Feature	Streaming Table	Materialized View	View
Data Storage	Persisted to Disk	Persisted to Disk	No (Logical Only)
Processing	Incremental (Only new data) – **Bronze layer (SCD2)**	Refresh (Full or Incremental) – **Bronze layer(SCD1)/Gold Layer(Reporting/BI/AI/ML)**	If we need On-the-fly transformation – **Silver Layer**
Visibility	Public in Catalog	Public in Catalog	Internal to Pipeline
Primary Use	High-speed Ingestion (only incremental)	Slightly lesser performance than stream table BI & Reporting	Internal transformation (Performant if used once (since no storage is used)/Not accessible hence we have to create mat view/stream table - if we use this data multiple times)

4. Implementing Medallion Architecture using DP (DLT)

Code: lakeflow3_medallion_pipeline

```
from pyspark import pipelines as dp

@dp.table(name="catalog1_dropme.default.drugstbl_medal_bronze")
def drugstbl_bronze_tbl():
  return spark.readStream.table("lakehousecat.deltadb.drugstbl")

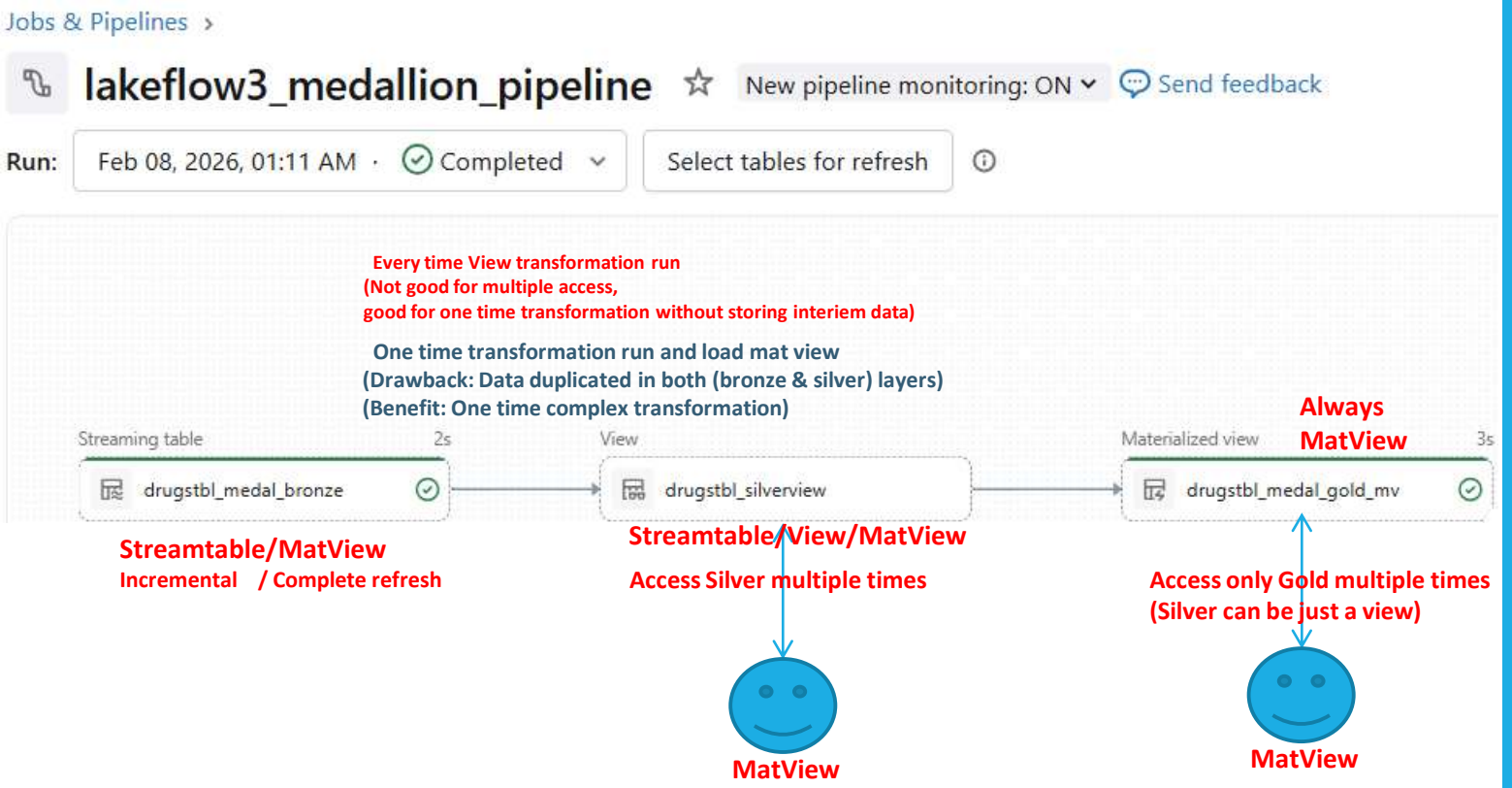
@dp.view(name="drugstbl_silverview")
@dp.expect_all_or_drop({"rating_is_valid": "rating IN (9,10)"})
def drugstbl_silvernewtbl():
  return (
    spark.read
      .table("catalog1_dropme.default.drugstbl_medal_bronze")
      .filter("rating IN (9,10)"))

from pyspark.sql import functions as F
from pyspark import pipelines as dp

from pyspark.sql import functions as F

@dp.materialized_view(
  name="catalog1_dropme.default.drugstbl_medal_gold_mv"
)
def drugstbl_medal_gold():
  df = spark.read.table("drugstbl_silverview")

  return (
    df.groupBy("drugname")
     .agg(
       F.sum("usefulcount").alias("sum_usefulcount"),
       F.avg("rating").alias("avg_rating")
     ))
```



Imperative Pipeline to Declarative Pipeline DLT Project

5. Imperative to Declarative Pipeline (Logistics Usecase Databricks agnostic modernization)

```
from pyspark import pipelines as dp

@dp.table()
def drugstbl_bronze_tbl():
   return spark.readStream.table("lakehousecat.deltadb.drugstbl")

@dp.table()
@dp.expect_all_or_drop({"rating_is_10": "rating = 10"})
def drugstbl_silvernewtbl():
   return spark.readStream.table("drugstbl_bronze_tbl").filter("rating in (9,10)")
```

	Mat View	Stream Table	View (CreateOrReplacetempView)
Bronze	**Y** (Source give every time the Complete (Employee Dim) Data -> Ins/Upd/Del) - SCD1	**Y** (Incremental Data (Dim/Fact) -> Only Inserted) – SCD2/Detail Data	**N** (As we need raw data in its own format for future reprocessing)
Silver	Y (Complete Data -> Ins/Upd/Del)	**Y** (Incremental data transformation from Bronze)	Y (We don't want to preserve, just on the fly simple one time compute, to avoid storing data in both layers)
Gold	**Y** (Whether Bronze & Silver Streams or Batch, prefer Gold with Mat View)	Y (Not Preferred for aggregation (historical), but for joins and other incremental reporting)	**N** (As we need wrangled data in the consumption layer for Reporting, ML & AI solutions)



DLT – Pipeline Modes

In Databricks Delta Live Tables, the **Pipeline Mode** (Continuous vs. Triggered) defines how often data is processed and how compute resources are managed.

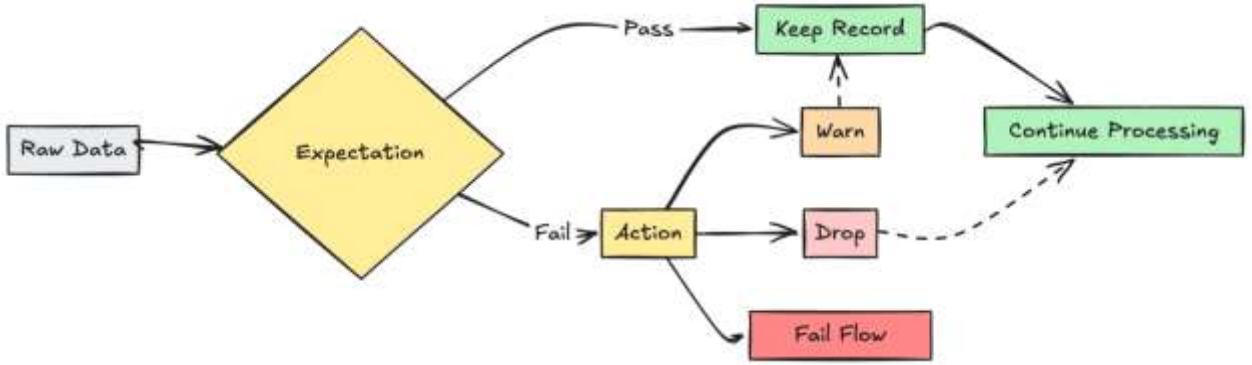
Feature	Triggered Mode	Continuous Mode
Logic	Processes all available data once and stops.	Processes data as it arrives, 24/7.
Data Freshness	Minutes to Days (Batch style).	Seconds to Minutes (Streaming style).
Cluster Behavior	Spins up, runs, and shuts down.	Always running.
Cost	**Lower** (pay only for execution time).	**Higher** (paying for a 24/7 cluster).
Ideal Use Case	Hourly/Daily reports, Batch ETL.	Real-time dashboards, IoT, Fraud detection.

DLT – Expectations (Data Validations)

In **Databricks DLT**, **data validation** is implemented using **Expectations**.

Expectations are optional data quality rules defined in materialized views, streaming tables, or views, using SQL Boolean conditions to validate each record.

Types of Data Validation in DLT



Type	Behavior	Use Case
EXPECT	Logs bad records (permissive)	Monitoring
EXPECT_OR_DROP	Drops bad records (dropmalformed)	Data cleansing
EXPECT_OR_FAIL	Fails pipeline (failfast)	Critical data(strict rule)

Create a **new DLT-managed table** called sales_multi_validation whose data comes from reading the sales table, after applying expectations.

```
@dlt.table(name="sales_multi_validation")
@dlt.expect("valid_amount", "total_amount > 0")
@dlt.expect_or_drop("valid_quantity", "quantity > 0")
@dlt.expect_or_fail("valid_region", "region IS NOT NULL")
@dlt.expect_or_fail("mandatory_fields","sale_id IS NOT NULL AND
customer_id IS NOT NULL")
@dlt.expect_all_or_drop({"valid_quantity":"quantity > 0“,
"valid_amt":“amt > 0“})
def sales_multi_validation():
    return spark.read.table("sales")
```

Logistics/Fleet Management Project

External Systems

Ingestion/Ingress Team

Cloud/Databricks Data Engineers

Data Scientist
AI Engineer
AI Data Engineer

Medallion Architecture (Datalake & Lakehouse)

Lakeflow Declarative (DLT) Pipeline

1. Imperative programming (Pyspark+SQL/Python)
2. Batch/Streaming (MV/View/Streaming table)
3. Lineage, Logging, Metrics, incrementaziation
4. DQ Expectations
5. Applying CDC/Incremental/SCD

Source Systems

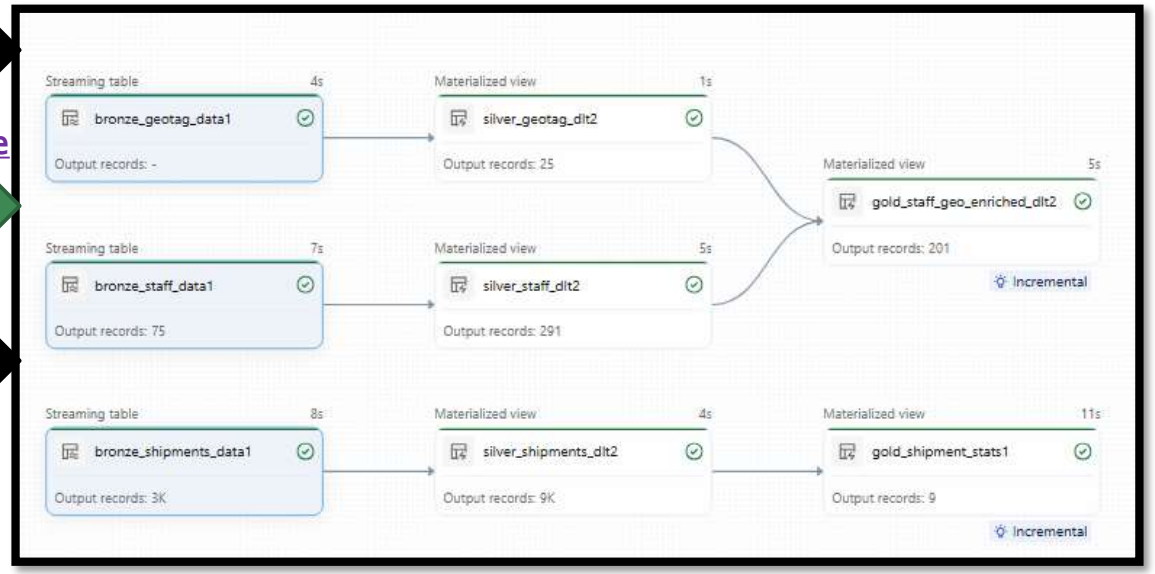
- RDBMS
- Cloud FS
- {API}
- CRM/3rd party

Foreign Catalog

Lakeflow Ingestion Pipeline

AutoLoader

Native Connector
Partner Connect
Custom Ingestion



AI/ML
- Playground
- Experiments
- Features
- Models
- Serving

Data Analysts/
BI Developer
Egress Engineer

SQL
- SQL Editor
- Queries
- Dashboards
- Genie
- Alerts
- Query History
- SQL Warehouses

Lakeflow Jobs (Scheduling & Orchestration)

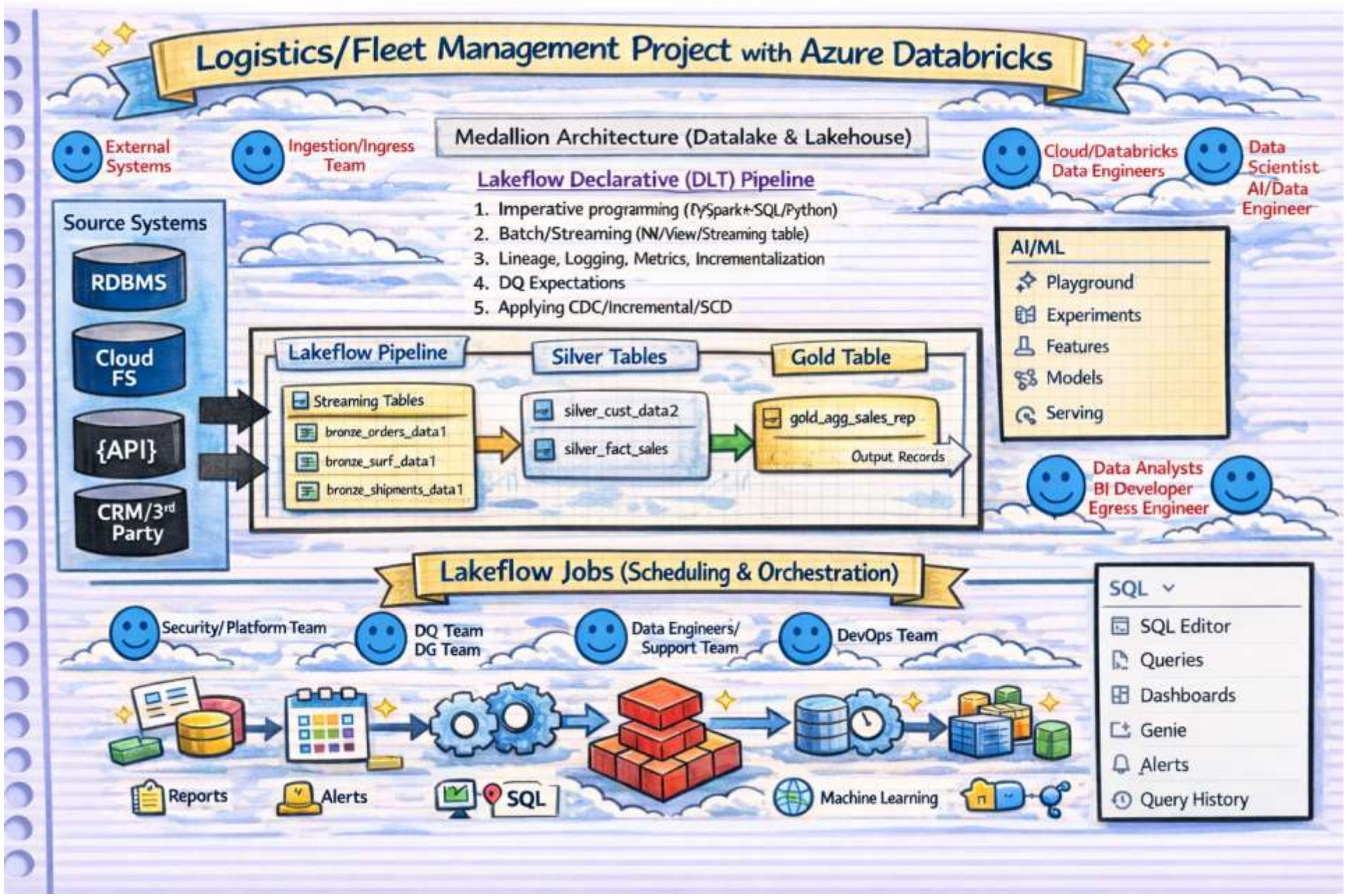
Security/Resource Platform Team

Data Quality, Data Auditing, Observability, Alerting, Data Governance Team

Data Engineers/ Support Team/ Alerting/Audit

Deployment (CICD) Dataengineer/Devops

INCEPTEZ TECHNOLOGIES


Logistics/Fleet Management Project with Azure Databricks
External Systems
Ingestion/Ingress Team
Medallion Architecture (Datalake & Lakehouse)
Cloud/Databricks Data Engineers
Data Scientist AI/Data Engineer
Lakeflow Declarative (DLT) Pipeline
1. Imperative programming (PySpark+SQL/Python)
2. Batch/Streaming (MV/View/Streaming table)
3. Lineage, Logging, Metrics, Incrementalization
4. DQ Expectations
5. Applying CDC/Incremental/SCD
Source Systems
RDBMS
Cloud FS
{API}
CRM/3rd Party
Lakeflow Pipeline
Silver Tables
Gold Table
Streaming Tables
bronze_orders_data1
bronze_surf_data1
bronze_shipments_data1
silver_cust_data2
silver_fact_sales
gold_agg_sales_rep
Output Records
AI/ML
Playground
Experiments
Features
Models
Serving
Data Analysts
BI Developer
Egress Engineer
Lakeflow Jobs (Scheduling & Orchestration)
Security/ Platform Team
DQ Team
DG Team
Data Engineers/ Support Team
DevOps Team
Reports
Alerts
SQL
Machine Learning
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History



3. Databricks Lakeflow Jobs



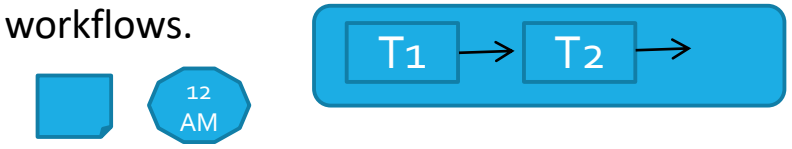
Provide Orchestration, Scheduling on Databricks Components

Airflow (Astronomer/Cloud Composer/MAAS), Oozie, DBT

Shell Scripts/Tivoli/ESPX/Cron/ControlM/Autosys...

2. Databricks – Lakeflow Jobs

LakeFlow Jobs are an enhanced, unified orchestration framework in Databricks for building, scheduling, and managing data pipelines and workflows.



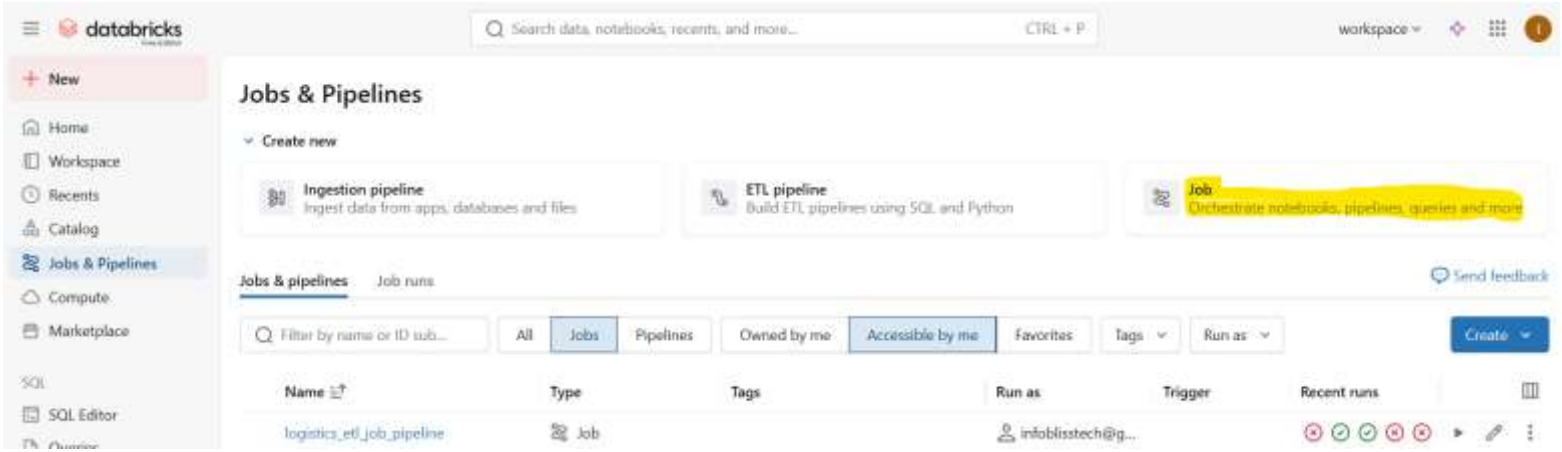
Lakeflow = **Jobs + DLT + Orchestration + Scheduling + Data Quality + Lineage**

LakeFlow Jobs: Core Building Blocks

- **Job -** The overall workflow definition.
- **Task -** A single unit of work

  (e.g., Python script, SQL query, ingestion).
- **DAG (Directed Acyclic Graph)**

  Defines dependencies and execution order.
- **Schedule/Trigger** - Controls when the job runs.
  - 2 Types of (Timer Driven) Scheduling –Time & Continuous
  - 3 Types of (Data Available) Triggers – Manualy, File arrival & Table Update Trigger
- **Monitoring** - Built-in dashboards, lineage, logs, and alerts.



Task Type	Description
Notebook	Run a Databricks notebook (PySpark, Scala, SQL, etc.)
Python Script	Run a .py file directly
Delta Live Table (DLT)	Trigger a Declarative (DLT) pipeline
SQL Query/File/Alert	Execute a SQL statement or script
LakeFlow Connect Task	Ingest data from a source (Postgres, Snowflake, etc.)
Webhook Task	Trigger external APIs
DBT Task	Run a dbt project
Custom Task	Shell or REST command

2. Lakeflow Jobs – Job-Level Options

Option	Description	Example
Name	Name of the workflow	Daily Txn ETL
Description	Detailed description	Ingest, transform, and aggregate daily transaction data
Tags	Key-value metadata for filtering/searching	env=prod, team=data-eng
Owner	Assigned user or service principal	data-team@databricks.com
Permissions	Control who can view/edit/trigger	Viewer, Editor, Owner
Max concurrent runs	Prevent overlapping runs	e.g., 1
Timeout (mins)	Kill job if exceeds limit	e.g., 180
Retry policy	Global retry for job	e.g., max 3 retries, interval: 10m
Notifications	**Email/**Slack/PagerDuty on success/failure	on_failure: Slack channel
Git Integration	Link code to a Git repo/branch	main branch in GitHub

Databricks – Lakeflow Jobs We created

Lakeflow Job1: We Created a **Lakeflow Job** to manually/Filebased **Orchestrated** & **Scheduled** the **Imperative Notebooks (Traditional) with job level parameters and task level (notebook level) parameters used in the config notebooks.**



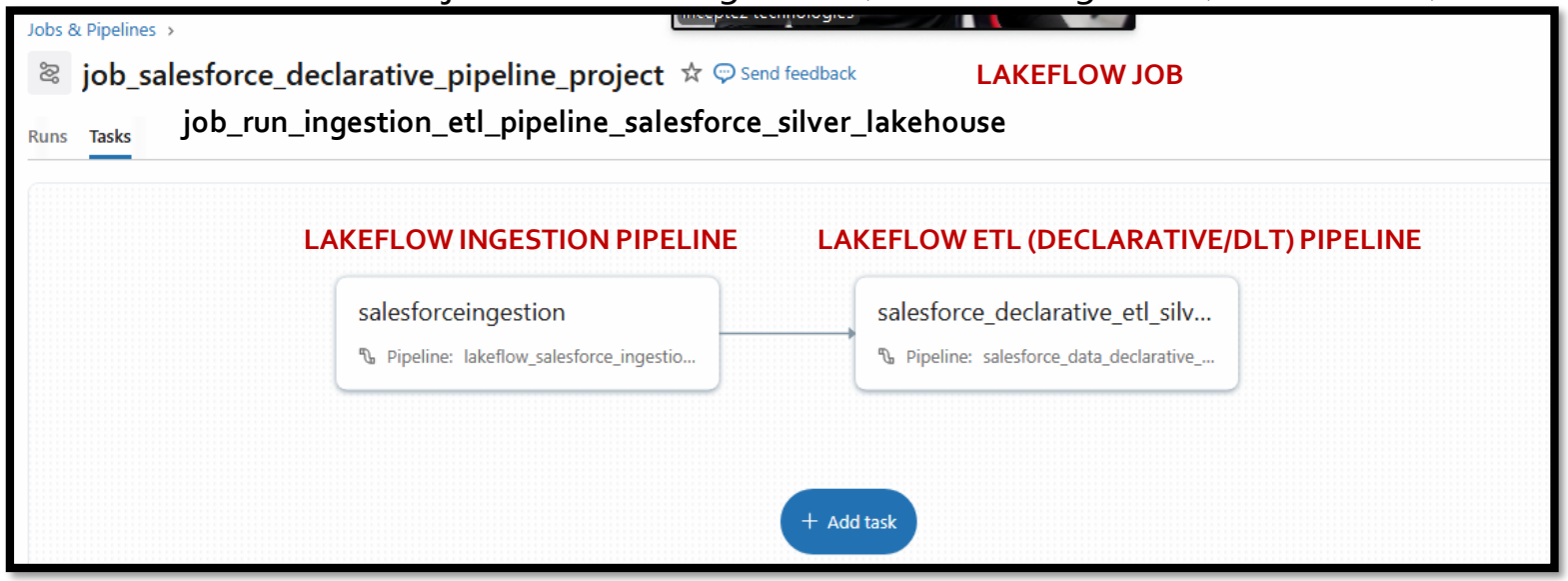
Lakeflow Job2: We Created a Lakeflow Job to Orchestrated & Scheduled the **Declarative ETL Pipeline** (Modernized) – Databricks Agnostic, If I want to pass parameters in declarative pipeline, we have to go to with the spark.conf option *(as we don't have widget to collect using parameters, as we did for imperative)* the declarative pipeline (task) -> go to settings -> configuration -> add key(base_path) & value(some cloud path) pairs-> behind the scene it will run spark.conf.set("base_path", "some cloud path")

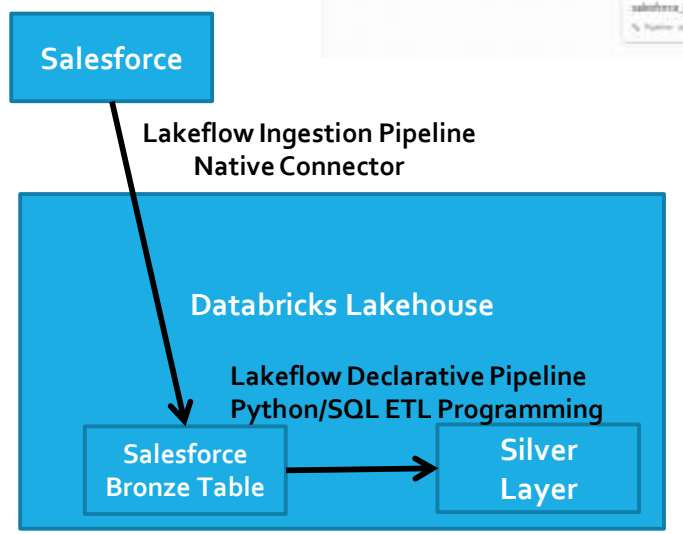


Databricks – Lakeflow Jobs We created

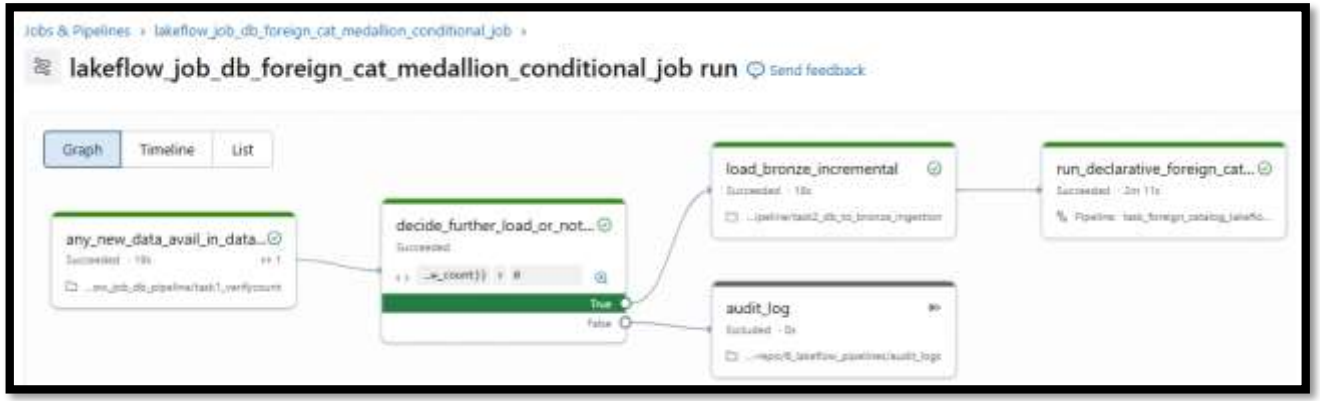


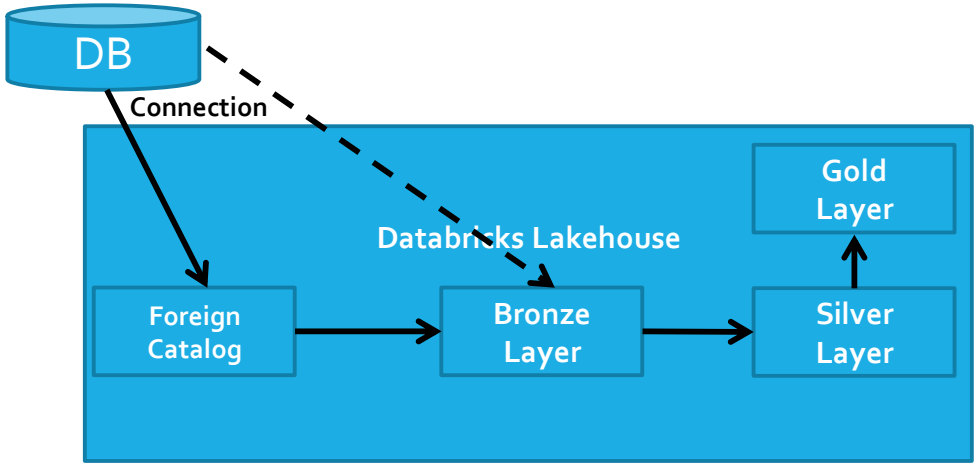
Lakeflow Job3: Create a job with both Ingestion (Lakeflow Ingestion) & Curation (Lakeflow Declarative ETL)





Lakeflow Job4: We Created a Lakeflow Job to Orchestrated & Scheduled the Notebook (Check for data availability) -> triggered load_bronze_incremental notebook to load data (on if data available in the source) then run a Lakeflow Declarative pipeline to load silver and gold medallion layers, otherwise it will run the audit log notebook to put an entry for logging purpose, where Production support team will generate some notifications.







4. Databricks CLI

Databricks CLI

The **Databricks CLI** is a **command-line tool** that lets you **interact with a Databricks workspace programmatically** from your local machine or using Terminal or CI/CD pipeline—without using the UI.

Installation Steps

1. Open PowerShell/Terminal as Administrator
2. Install Databricks CLI

   winget install Databricks.DatabricksCLI *(Windows)*

   brew install databricks (Mac)

3. ***Restart Terminal*** and verify Installation

   databricks -v

4. Start Login Process

   databricks auth login *--host https://dbc-23f124b5-6ef4.cloud.databricks.com/*

5. Complete Browser Authentication
6. Test the Connection

   databricks clusters list

It is commonly used for:

- Workspace automation
- Job deployment
- Cluster management
- ***DBFS operations***
- CI/CD integration (DevOps)

Databricks CLI Commands

Modules	Command	Description
Authentication		
	databricks auth login --host <databricks url>	Authenticate using OAuth (standard for modern accounts).
	databricks auth profiles	List all configured connection profiles.
	databricks auth describe	Check current auth
	databricks version	Check the current version of the CLI.
DBFS Filesystem & Workspace	**Linux/Workspace Commands:** ls / pwd cd Drafts	List files/folders Present working directory Change directory (placeholders -, /, .., .) – back & forth directory toggle, / is a root, .. Is parent, . Is current dir
	databricks fs ls dbfs:/	List files in a root DBFS path.
	databricks fs ls /Volumes/catalog1_we47/schema1_we47/bronze	List files in a specific DBFS path.
	databricks fs mkdir /Volumes/catalog1_we47/schema1_we47/bronze/testdir	Create a new directory in DBFS.
	databricks fs **cp** "E:\1_2025_BATCHES\logistics_project_data\logistics_shipment_detail_3000_1.json" dbfs:/Volumes/catalog1_we47/schema1_we47/bronze/testdir	Copy a file from local to Databricks. ***(Run this command in windows cli)***
	databricks **fs cp** "E:\1_2025_BATCHES\logistics_project_data\logistics_shipment_detail_3000_1.json" **dbfs:/**Volumes/catalog1_we47/schema1_we47/bronze/testdir/logistics_shipment_detail_3000_2.json databricks fs **rm** dbfs:/Volumes/catalog1_we47/schema1_we47/bronze/testdir/logistics_shipment_detail_3000_2.json databricks fs **rm -r** dbfs:/Volumes/catalog1_we47/schema1_we47/bronze/testdir	Delete a file or non empty directory (**recursively**).
	databricks workspace list /Users/infoblisstech@gmail.com	List the files in the workspace
	databricks **workspace export** "/Workspace/Users/infoblisstech@gmail.com/a.ipynb" --file "./Downloads/a.ipynb" databricks **workspace** import "/Workspace/Users/infoblisstech@gmail.com/b.ipynb" --file "./Downloads/a.ipynb" --language PYTHON	Download notebook from the workspace to local & Vice versa.

Databricks CLI Commands (We will see in the cloud env)

Modules	Command	Description
Unity Catalog	Datagovernance Team/DE use this	
	databricks catalogs list	Show all available Unity Catalog catalogs.
	databricks schemas list <catalog-name>	List schemas (databases) within a catalog.
	databricks tables list <catalog-name> <schema-name>	List all tables, including your GCS external tables.
	databricks tables get <catalog-name>.<schema-name>.<table-name>	List all the columns in the table
	databricks storage-credentials list	View all cloud credentials (AWS/GCP/Azure).
	databricks external-locations list	List all GCS/S3/ADLS external paths.
Clusters & Jobs	Platform team/DE use this	
	databricks clusters list	List all clusters and their status (Running/Terminated).
	databricks clusters start --cluster-id <id>	Spin up a specific cluster.
	databricks jobs list	List all scheduled workflows.
	databricks jobs run-now --job-id <id>	Trigger a job immediately.
Secrets Management	Platform or Security/DE use this	
	databricks secrets create-scope <scope-name>	Create a new logical container for secrets.
	databricks secrets put-secret <scope> <key>	Opens an editor to paste your secret value.
	databricks secrets list-scopes	See all available secret scopes.
	databricks secrets list-secrets <scope>	List keys within a specific scope.
Packaging & Deployment	DevOps team/DE team will use this	
-Databricks Asset Bundle (DAB) (2024)	databricks secrets create-scope <scope-name>	Create a new logical container for secrets.

DBSecrets in Databricks

Databricks Secrets provide a ***secure way to store sensitive information*** like API keys, database passwords, and username/password in the connection strings.

Instead of hardcoding these in notebooks, store them in Secret Scopes and retrieve them using the dbutils.secrets utility

Core Concepts

- **Secret Scope:** A logical container (like a folder) used to group related secrets.
- **Secret:** A key-value pair where the value is the sensitive data.
- **Access Control List (ACL):** Defines who can READ, WRITE, or MANAGE secrets within a specific scope.

Managing Secrets using Secret Scope (Vault)

- Secret creation and management (scopes and values) is primarily handled through the Databricks CLI, as the UI intentionally limits direct secret creation for security reasons.
- From UI, allowed to create only **Secret Scope** but not Secret

https://dbc-23f124b5-6ef4.cloud.databricks.com/#wd36secscope/createScope

- dbutils usage is restricted to listing and reading secrets, while creating scopes or storing secret values is deliberately allowed only via the Databricks CLI or UI to ensure security.

DBSecrets in Databricks

Common CLI Commands: SecretScope (Vault/Container/Locker) -> Secret name (Name of the password) -> stored Password (redacted)

- **Create a scope:** databricks secrets create-scope <scope-name>

databricks secrets create-scope izwd36scope2

- **Add a secret:** databricks secrets put-secret <scope-name> <key-name>

databricks secrets put-secret izwd36scope2 secretdbpassword1

- **List scopes:** databricks secrets list-scopes (Containers where passwords are stored under secrets)

databricks secrets list-scopes

Use Secrets in a Notebook

Once a secret is stored, can retrieve it using the dbutils library.

Databricks automatically **redacts** (hides) the output if you try to print it.

```
#List the Secrets
dbutils.secrets.listScopes()
dbutils.secrets.list("scopename")

# Retrieve the secret
db_user = dbutils.secrets.get(scope="scopename", key="scope1")
db_password = dbutils.secrets.get(scope="scopename", key="secret1")

# Use it in a connection string (it will appear as [REDACTED] if printed)
jdbc_url = f"jdbc:mysql://host:3306/db?user={db_user}&password={db_password}"
```



5. Databricks CICD (DAB, Git Action)

1. Package & Deployment Team – Package of Code & Deployment of Code (DAB)
2. CICD/DevOps Team – Git Integration & Git Action / Jenkins / Azure Devops (CICD)

- ✓ DBT (Databricks job/Declarative pipeline) – Data engineers
- ✓ Ansible (databricks.yml) – Devops team
- ✓ Terraform/Shell script (Manual DAB operation) – Devops team
- ✓ Jenkins (video) / Git Actions – Devops team

CI/CD – Continuous Integration & Continuous Delivery/Deployment

CI/CD automates build, test, and deployment, enabling fast, reliable, and frequent software releases.

1. Continuous Integration (CI)
Frequent code merges to main branch
Automatic build & unit testing
Detects issues early

2. Continuous Delivery (CD)
Auto deploy to higher environments
Manual approval for production (HIL)
Always release-ready

3. Continuous Deployment (CD)
Fully automated release to production
No human intervention
Requires strong test coverage

Key Benefits
Faster, safer, and high-quality software delivery with minimal manual effort

Databricks Asset Bundle (DAB)

Databricks Asset Bundles (DAB) is a framework to **package, deploy**, and manage Databricks projects(Assets) *(jobs, notebooks, pipelines, configs)* in a **DevOps-friendly, version-controlled** fashion.
yaml -> Yet ain't markup language (Used to Define configurations Hierarchically)

Before DAB:

- Manual job/package creation
- Environment-specific edits (non standard fashion)
- No proper CI/CD
- Error-prone deployments

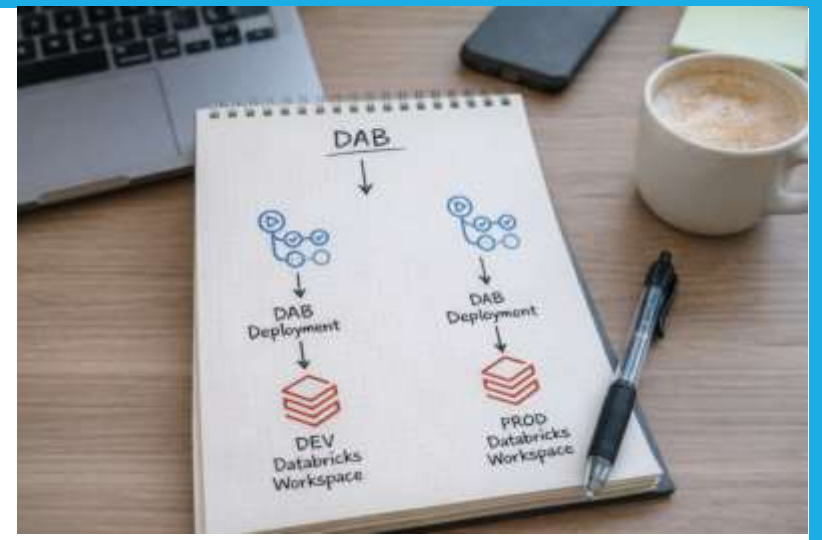
With DAB:

- ✓ One codebase (folder)
- ✓ Multi-environment support (Dev/UAT/Prod)
- ✓ Automated deployment
- ✓ Repeatable & reliable releases (Agile)

Typical Folder Structure: **databricks bundle init**

```
bundle
 ├── databricks.yml        → Bundle brain (orchestrates everything)
 ├── resources/
 |    ├── jobs.yml         → defines jobs
 |    └── pipelines.yml    → defines pipelines
 └── src/
      ├── notebooks        → executable logic
      └── sql scripts/Declarative pipeline transformations folder
```

DAB Commands



databricks bundle init

Purpose:

Creates the standard folder structure and the databricks.yml file.

Dataengineering:

WE47:

Databricks

WD36:

Databricks

databricks bundle validate

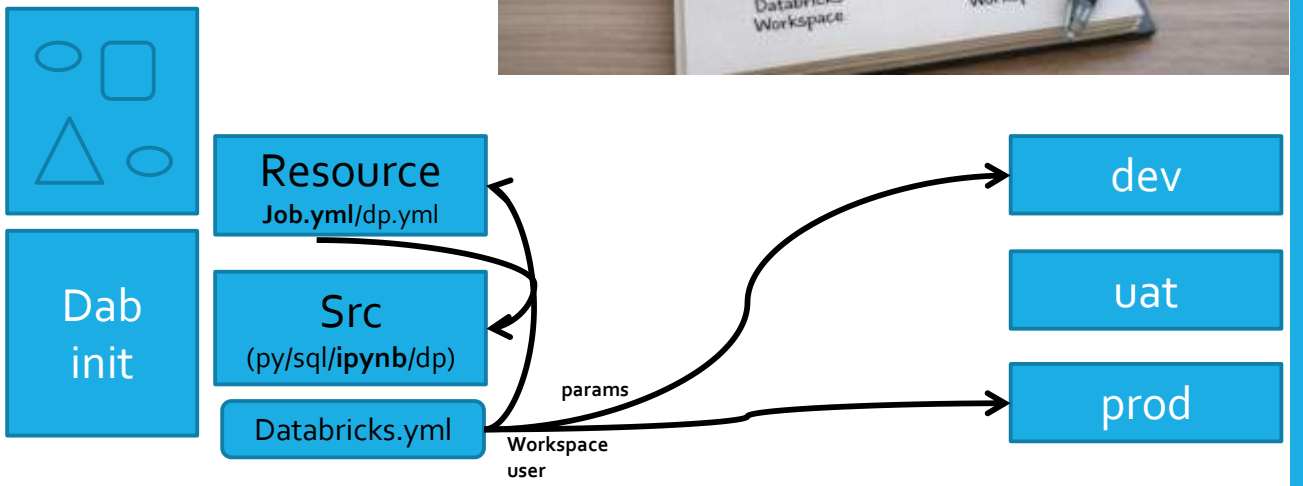
Purpose:

Validates the bundle configuration by checking:

Configuration syntax

Missing references

Invalid target definitions



databricks bundle deploy

Purpose:

*Deploys the bundle to a target environment such as **Dev, UAT, or Prod***

Creates a .bundle folder

Uses target flag -t

databricks bundle deploy -t dev

DAB Implementation

1. Create a folder (wd36dabmain) in the home directory.

2. Enter into DB CLI/UI

3. Goto the sample folder wd36dabmain.
cd wd36dabmain

4. Initialize DAB
databricks bundle init

5. Modify the databricks.YML:
host, catalog and schema

```
workspace:
  host: https://dbc-23f124b5-6ef4.cloud.databricks.com/
variables:
  catalog: workspace
  schema: default
```

6. Create a sample notebook and create a job to orchestrate this notebook, ***Move the notebook code to the src path*** under the dab bundle folder.

7. Goto the Lakeflow job created in the above and click the … button in the top (just before the Run Now Button) -> ***edit as yaml*** -> copy the yaml content

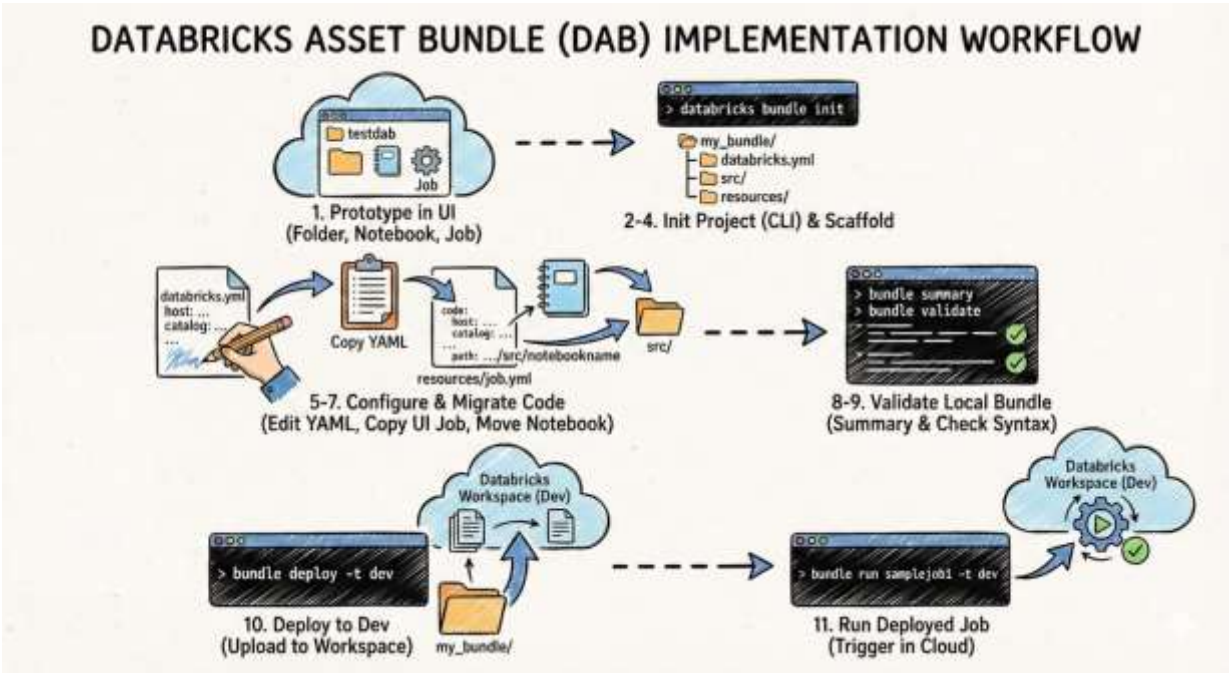
8. Goto the ***Resource folder*** inside the dab folder -> Rename sample_job.job.yml as job.yml (for a better usability) -> paste the copied yaml content inside this ***job.yml*** -> ***change the notebook path to ../src/notebookname.yaml -> Modifying the parameters to ${var.catalog} & ${var.schema}***

9. Goto the project folder and Run the below bundle summary command to verify
databricks bundle summary

10. Do the Databricks bundle validated (ensure to change the notebook extension as notebook.ipynb)
databricks bundle validate

11. Do the deployment of the bundle
databricks bundle deploy -t dev

12. Run the databricks job on triggered fashion (Copy the jobname from the job.yml)
databricks bundle run lakeflow_job_orchestrate_project_nb -t dev



cd databricks-code-repo/wd36dabmain
databricks bundle init

```
Unique name for this project [my_project]: dabprojectwe47
Include a job that runs a notebook: yes
Include an ETL pipeline: no
Include a sample Python package that builds into a wheel file: no
Use serverless compute: yes
Default catalog for any tables created by this project [workspace]:
Use a personal schema for each user working on this project.
(This is recommended. Your personal schema will be 'workspace.infoblisstech'.): yes
```



6. Databricks AI-BI
SQL Serverless WH

Databricks Serverless Warehouse

Databricks Serverless SQL Warehouse is a fully ***managed SQL compute service*** used to run analytics queries on data stored in the Lakehouse without provisioning or managing clusters.

SQL Warehouses use the ***Photon engine*** optimized for SQL analytics, while Spark Serverless runs the **full Apache Spark engine** for data engineering workloads.

Feature	Spark Serverless	Serverless SQL Warehouse
Primary Use	Data engineering, ETL, ML, notebooks	BI, dashboards, SQL analytics
Engine	Apache Spark clusters managed automatically	Databricks **Photon** (C++) SQL engine – MPP (Massively Parallel Processing)
Typical Users	Data engineers, data scientists	Data analysts, BI tools
Interface	Notebooks, jobs, pipelines	SQL Editor, dashboards, BI tools, Genie
Languages	Python, PySpark, Scala, SQL	SQL only
Compute Management	Fully serverless Spark clusters	Fully serverless (***Vectorized SQL execution***)
Latency(Response time)	Seconds–minutes (It involves job submission (warmup))	Milliseconds–seconds (Adhoc request, No much warmup is needed)
Integration	Workflows, Delta Live Tables / Lakeflow, ML	Power BI, Tableau, Looker, dashboards, Genie
Pricing Model	Pay for compute used during Spark workloads	Pay for SQL query execution
Best For	Data pipelines, streaming, transformations, ML	Interactive/adhoc queries and reporting

Databricks Visualization & Dashboard

AI/BI Dashboards:
Fast, low-code interactive **visualizations & Dashboards** natively integrated into the Databricks SQL workspace for standard reporting.

Dynamic Interactivity: Enables deep data exploration by allowing users to click data points to instantly filter related **visualizations** across the entire **dashboard**.

Standardized Logic: Uses Reusable Semantics to define core metrics once, creating a **single source of truth in the Gold by joining/grouping/aggregating (by the DE)** that prevents contradictory data definitions across different teams.

Secure & Automated Delivery: Natively enforces **Unity Catalog data governance** (row/column-level security).

Tailored Branding: Provides extensive customization options for themes, colors, and fonts

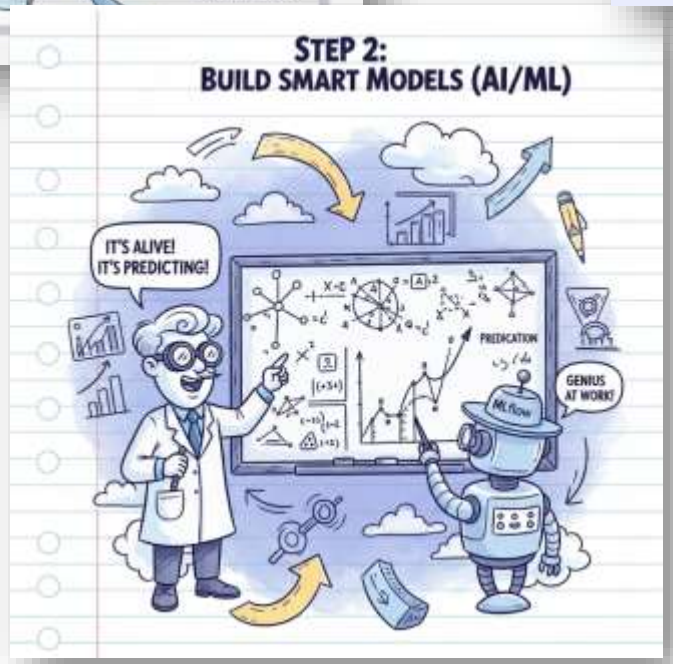
AI-Assisted Creation & Forecasting (beta): Lowers the technical barrier by translating **natural language directly** into SQL and charts, while also offering built-in predictive analytics for forecasting trends without requiring complex machine learning code.

Databricks Dashboard











Databricks Dashboard

Terminologies of Reporting Environments - Visualization & Dashboards:

Dimensions – Different perspective of different measures/metrics. Eg. profession, age, city, state, activity, category, spendby (low cardinality)
Facts – Values of the actual happenings. Eg. online_pos_amount, geo_coordinate (lat/long), age, amount
Measure – *Quantifying* the values of facts. – Measuring the average age, measuring total amount.
Metrics – Standards/*Qualifying* of measuring different Measures across dimensions.
Eg. Total sales happened across the states, Types of transactions happened, what are the recurring activities (*Using the dims+facts+measures we can build metrics*)
KPI – Key Performance Indicators (Vital metrics that helps our business to make some decisions/strategic planning) – promo/profit/demand/supply.
Reporting - Producing the metrics and measures for the given dimension in a value format. Eg. Csv download of the given data or visualization.
Visualization - Producing the metrics and measures for the given dimension in a *single visual representation format*.
Eg. Single Pie chart for showing the occupancy/coverage of sales happened across the city, Single Bar chart for comparing the occupancy/coverage of the product sold across the state.
Dashboard - Producing the metrics and measures for the given dimensions in multiple visual formats as a single dashboard *(collection of different visualizations)*.
Drill Down/Up – Drill up means I want to visualize the data in a higher ***granular*** level (state level) or Drill down means I want to visualize the data in lowerer *granular* level (area level)

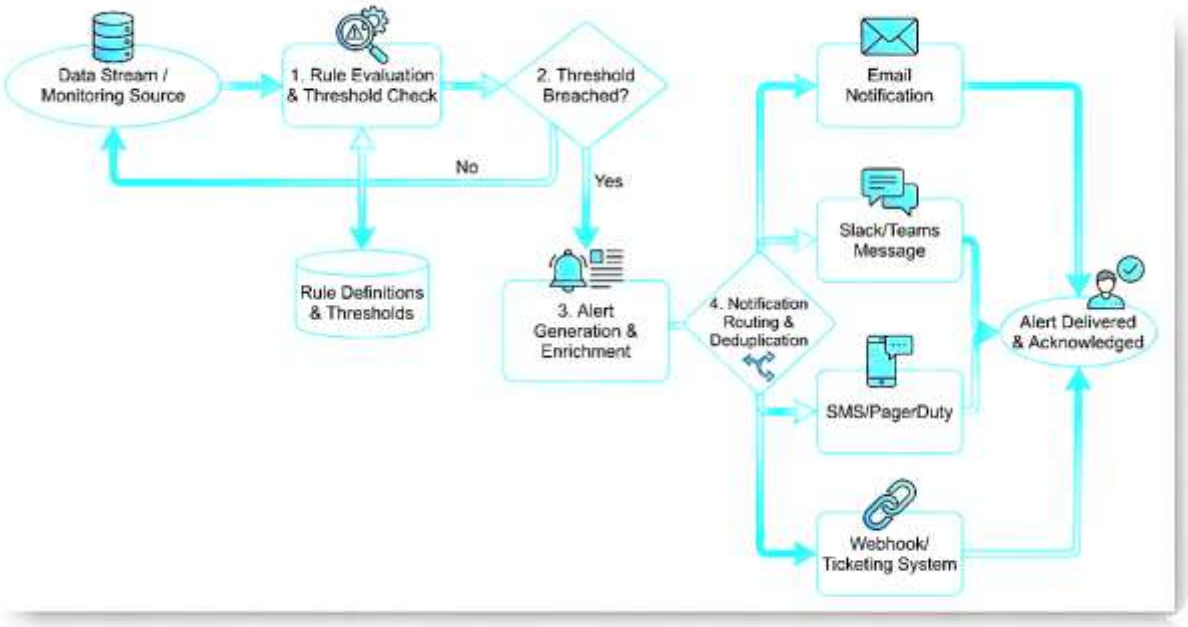
Different Types of Visual Charts (map, geo, grid, trends, metrics, gauges):

Scorecard/Metrics - To display a summary of a single metric/dimension.
Table/Grid - To turn your data into a detailed chart with aggregation/sorting/filtering/visuals(bar/heatmap).
Pie - To show data as "slices of pie," or *proportions of a whole*
Bar - To categorize with subcategories the groups of data used mainly for comparison.
Time Series – To look at how your data changes over time
Combo - To show each data series as a different marker type, either a column or a line.
Geo - To show a map of a country, continent, or region
Map - To explore geographic data
Area - To emphasize graphically the differences of changes in value between categories of data.
Scattered - To look for trends and patterns between two variables (x & Y Axes)
Gauge - To see how well a given metric is performing against a target goal
Sankey - Use a sankey chart to display a flow from one set of values to another
Pivot Table - Use a pivot table to reorganize and summarize your data

Databricks Alerts

Alerts in Databricks SQL is a highly effective way to ***proactively*** monitor your pipelines, compute performance, and data quality without constantly checking dashboards.

- ✓ **Navigate to Alerts:** Click **Alerts** in the left sidebar of your Databricks workspace and click **Create Alert**.
- ✓ **Write the Query:** In the query editor, write the SQL query that returns the specific data or metric you want to monitor, and click **Run**.
- ✓ **Define the Condition:** In the **Condition** field, set the logical threshold that will trigger the notification. You can evaluate the first value of a returned column, or apply an aggregation like SUM or AVERAGE across all rows in a single column.
- ✓ **Test the Logic:** Click **Test condition** to preview the alert and verify if your current dataset would trigger it based on your defined rules.
- ✓ **Configure the Schedule:** Click the calendar icon to set a periodic schedule (e.g., every 5 minutes, hourly) for Databricks to automatically run the query and check the condition.
- ✓ **Set up Notifications:** Choose the users, groups, or external notification destinations (such as Slack) that should receive the alert.
- ✓ Scenario 1: SLA & Compute Performance Tracking
- ✓ Scenario 2: Automated Data Quality Checks
- ✓ Scenario 3: Pipeline & Job Failure Monitoring (Lakeflow Jobs)



We are entering into the (First cut) AI learning with the following considerations:

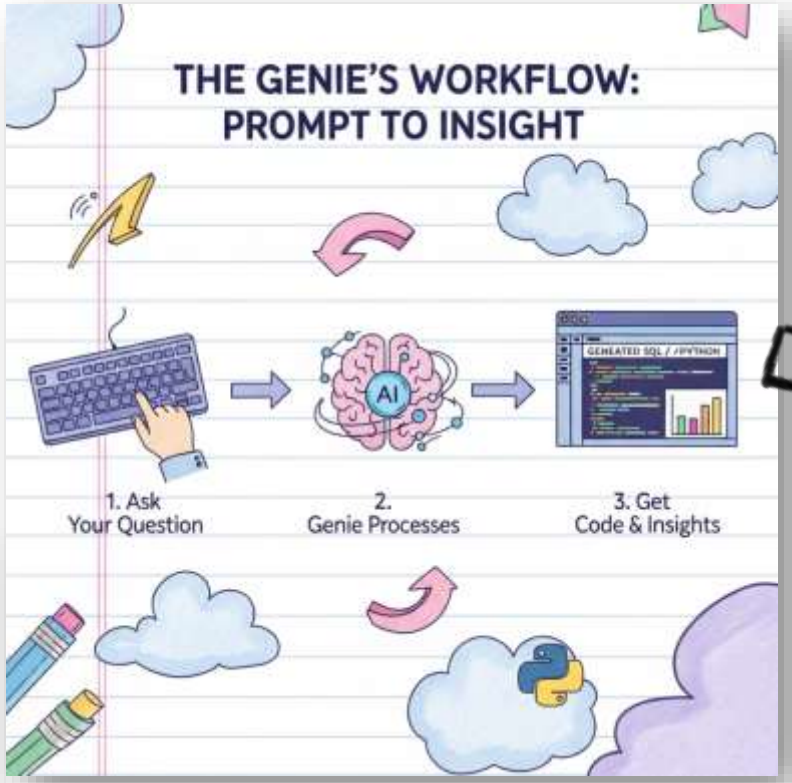
1. Our learning is Grounded & Guardrailed with Databricks (Platform)
2. We used Databricks **Genie** as a Ground for learning the foundation of AI.
    1. **Gen AI – Irfan (Gen HI)**
    2. **Prompt/Context Engineering – Questions Jerlin is asking (Eyes)**
    3. **Agentic AI (Agent/Multi Agents) – Medula Oblangata + Organs (Agents)**
    4. NLP – Natural Language Processing (Agent1) – Cerebrum (Convert english to tamil (tokenization, pos, fuzzy/semantic, vectors & embeddings))
    5. NLU – Natural Language Understanding (Agent2) – Cerebellum (NLU - intent, entity)
    6. LLM (Agent3) - Brain
    7. ~~RAG/~~Vectorization ~~DB –~~ Cerebellum (RAG – grounding in vector)
    8. AI additional/Common terminologies (Grounding, Hallucination, Guardrail, Human in loop, NLG, RAG, Vectors/embedding, Similarity Scoring)

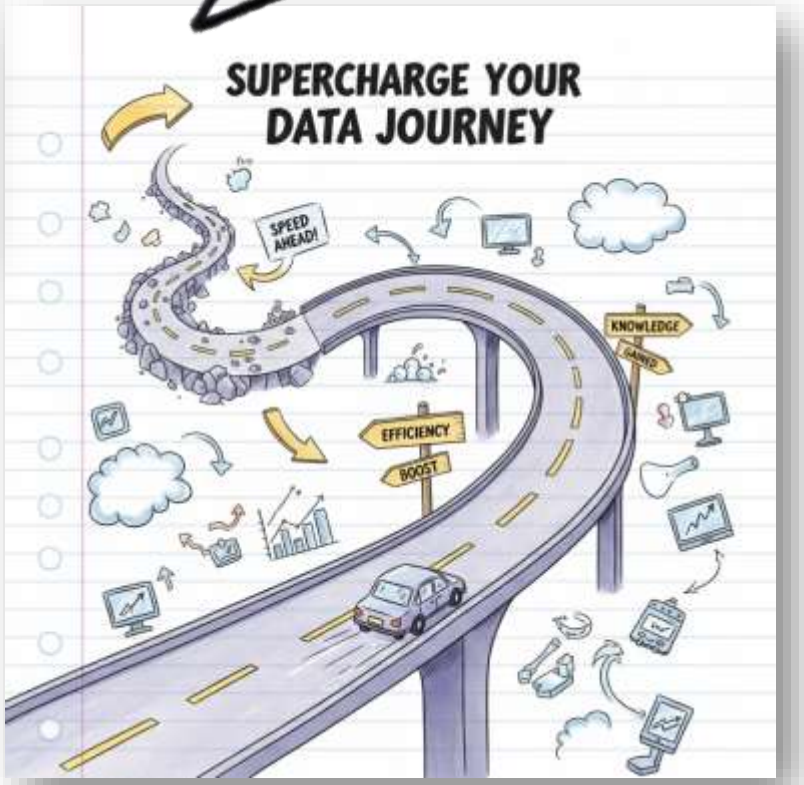
9. Rather depending on Genie (where all these implementations are abstracted), How we can use the GenAI techniques (step by step) to achieve Our own AI solutions.

1_genai_sql_nlp_sentiment_scoring

Databricks Genie (AIBI)







Databricks Genie

AI/BI Genie Spaces: A ***conversational AI*** interface that allows users to ask natural language questions and receive instant, data-backed answers.

The Genie AI Engine (Under the Hood):

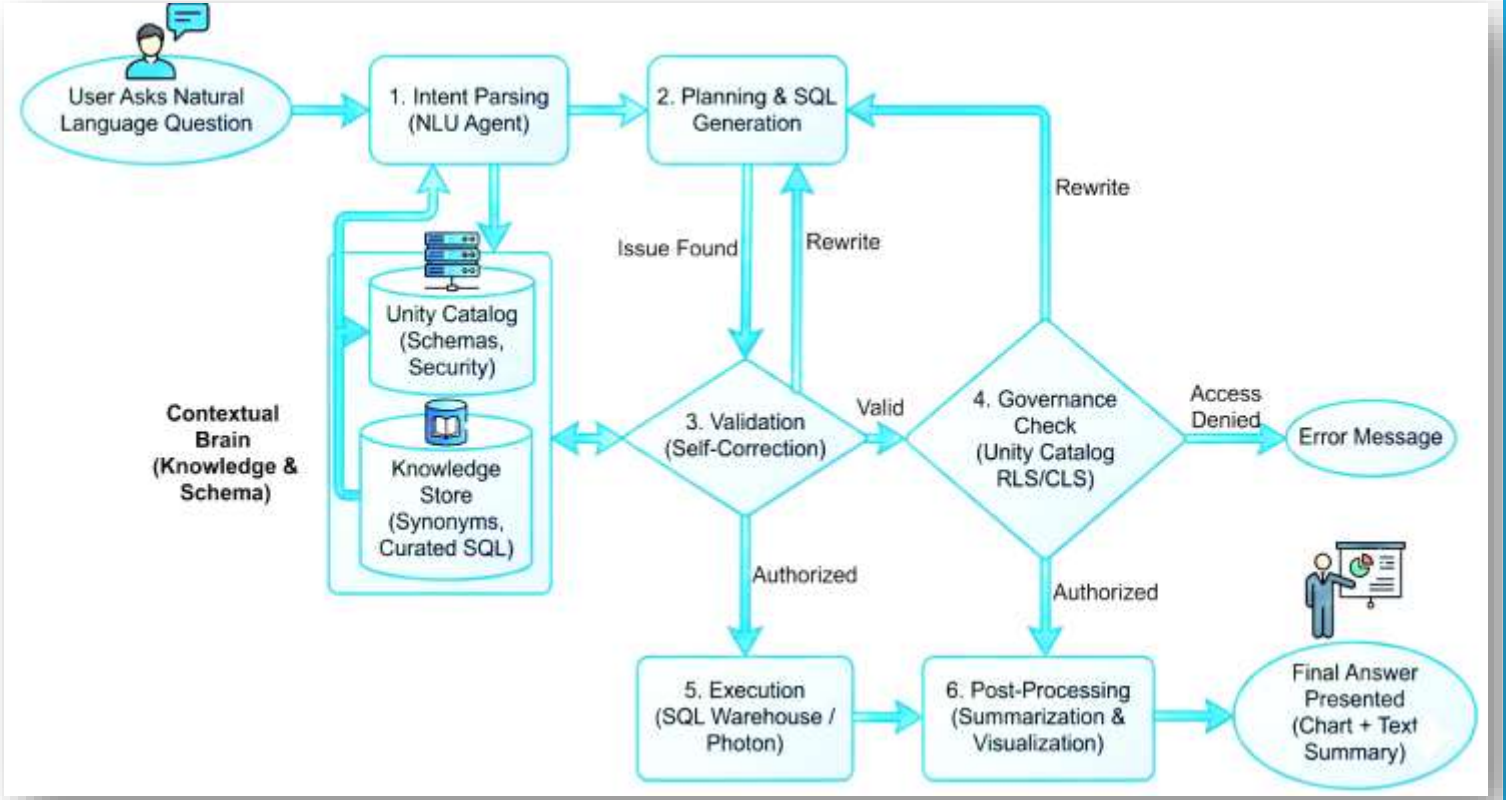
Compound AI System: Uses specialized **agents** to orchestrate **Intent Parsing** (**NLP/NLU**), SQL planning, and automated self-correction before execution.

Contextual Brain: Grounds every user question using your specific data models via Unity Catalog (schemas) and a Knowledge Store (synonyms, curated SQL).

Strict Governance: Automatically enforces Unity Catalog Row-Level and Column-Level Security (RLS/CLS) before authorizing any SQL execution.

Automated Delivery: Processes validated queries through a SQL Warehouse and automatically generates the most appropriate results in a form of tables, chart and text summary for the final answer.

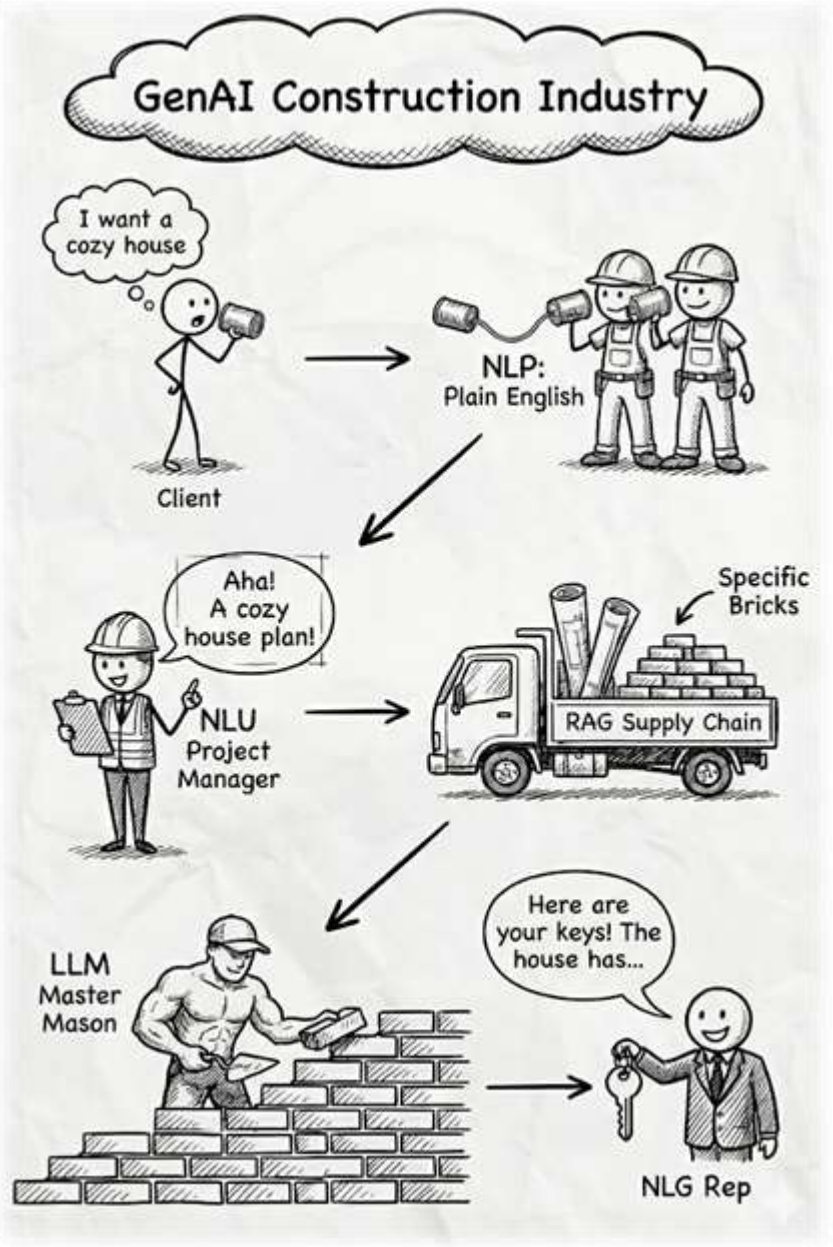
Genie Architectural Flow



Genie GenAI Terminologies

1. **Gen AI**
2. **Prompt/Context Engineering**
3. **Agentic AI (Agent/Multi Agents)**
4. NLP – Natural Language Processing (Agent1)
5. NLU – Natural Language Understanding (Agent2)
6. LLM (Agent3)
7. ~~RAG/~~Vectorization ~~DB~~

Analogy: Think of building a house.
NLP is the shared language that allows you to speak to the builders in plain English instead of architectural codes.
NLU is the project manager who listens to your words and figures out exactly what you want built. (Intent & Entities/Specs)
RAG (Vectors & Embeddings) is the blueprint and the supply chain bringing the exact bricks you need to the site.
The LLM is the master mason who actually takes those specific bricks and builds the wall. (SQL Query Creation & Execution)
NLG is the firm's representative who hands you the keys and explains the features of your newly built house back to you in plain English. (Text/Tables/Visuals)
GenAI is the entire construction industry (Overall Umbrella of all the above)



Databricks Genie Functional Workflow (Question -> NLP ->NLU-> LLM -> GenAI)

Analogy: Think of building a house.
NLP is the shared language that allows you to speak to the builders in plain English instead of architectural codes.
NLU is the project manager who listens to your words and figures out exactly what you want built.
RAG is the blueprint and the supply chain bringing the exact bricks you need to the site.
The LLM is the master mason who actually takes those specific bricks and builds the wall.
NLG is the firm's representative who hands you the keys and explains the features of your newly built house back to you in plainEnglish.
GenAI is the entire construction industry.

What Happens when I ask a question in Genie? "Show me a trend of the top 5 customers by revenue for the last quarter."
Step A: The NLP Layer (The Translator Agent1)
NLP handles the "mechanics." It breaks the sentence into tokens (Show, me, last, month, revenue) and identifies parts of speech (show me trend, top 5 cust, by revenue...) . It's like a linguist looking at the structure of a sentence.
Eg. It identifies that "Show me a trend" implies a **time-series visualization**, "top 5" is a **limit/ranking** clause, and "last quarter" is a **date filter**.

Step B: The NLU Layer (The Analyst Agent2)
NLU looks for **Intent** and **Entity** using the Unity Catalog..
Intent: The user wants to "retrieve data" (Select).
Entity: "Last month" means Feb 2026; "Revenue" means the total_sales column.
Context: It realizes the user is asking about business metrics (data behind the entity/columns), not a dictionary definition of revenue.

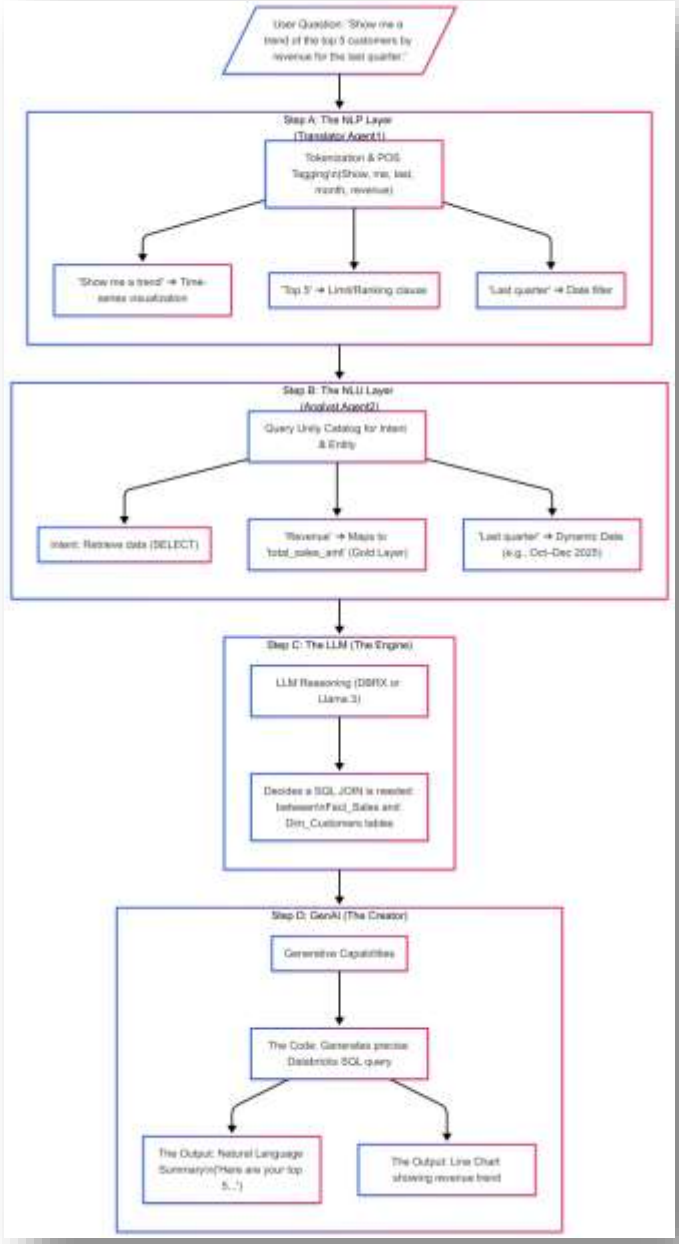
Eg.
- "Revenue" isn't a column name, but it maps to total_sales_amt in your **Gold Layer** table – using these features (vectors)
- "Last quarter" needs to be calculated dynamically based on today's date (e.g., if today is March 2026, it looks at Oct–Dec 2025).

Step C: The RAG + LLM (The Engine)
The system takes your question and searches your database metadata, The LLM is the "brain" and it reads the context, applies its reasoning capabilities about how to connect these pieces and knows that "revenue" and "sales" are synonyms and that "last quarter" is relative to today's date. It uses its internal **billions parameters** to map these concepts together and generates the SQL query.

Eg. The model (like **DBRX** or **Llama 3** acting as the **LLM model behind the scene**) uses its reasoning to combine these parts. It decides that a **SQL JOIN** is needed between your Fact_Sales and Dim_Customers tables to get the customer names and revenue totals.

Step D: GenAI (NLG) (The Creator)
Once the LLM understands the request (NLP/NLU), it uses its **Generative** (AI Brain) capabilities to produce something new/extra/additional as per the need:
It executes the **SQL query** provided by LLM to fetch the data.
It build Charts/text/tables and explain the chart/table it just built in a **Natural Language response**.

Eg.
The Code: A precise Databrics SQL query.
The Output: A natural language summary (e.g., *"Here are your top 5 customers from Q4..."*) and a **line chart** showing the revenue trend.



Databricks Genie Description

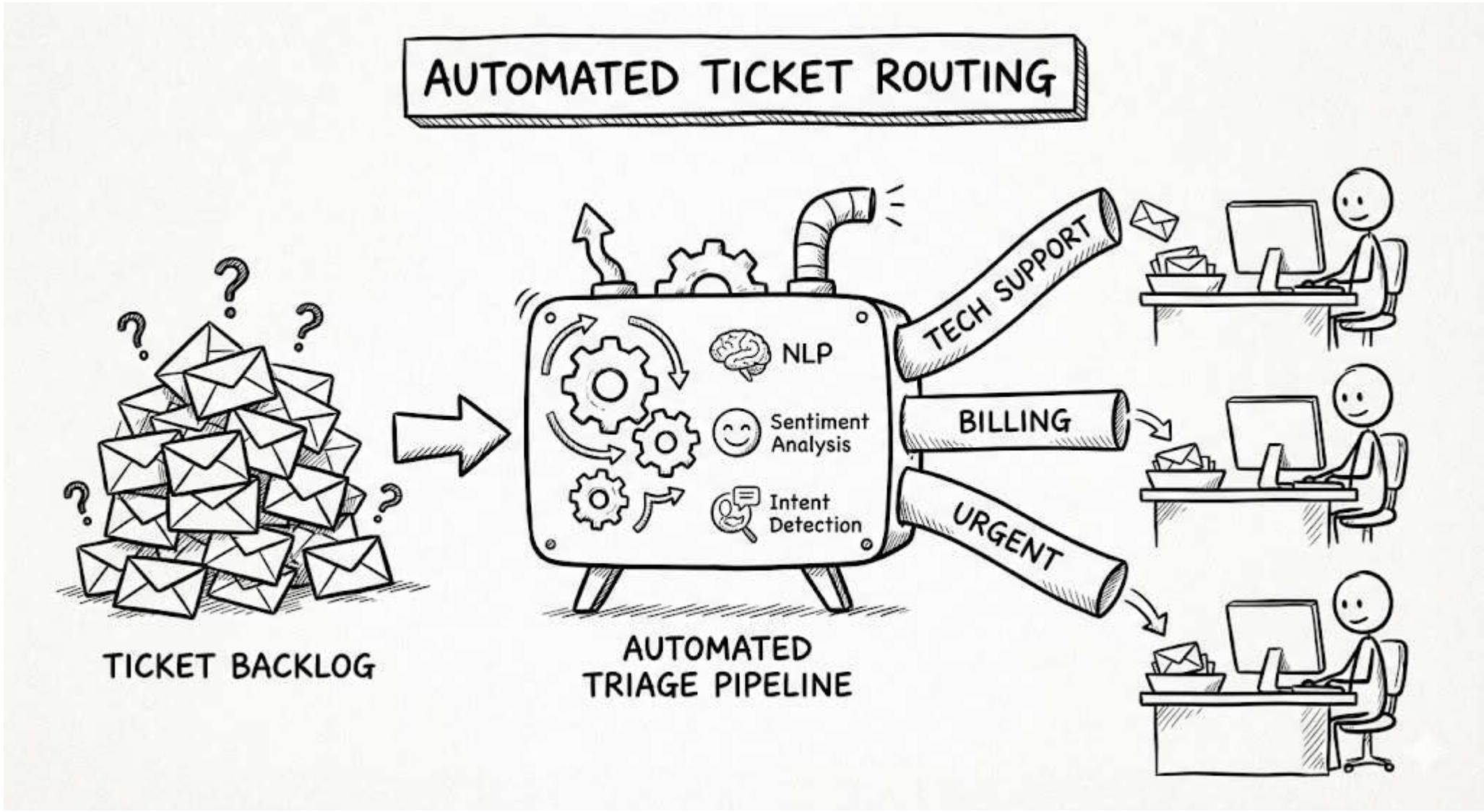
Genie -> Instructions -> SQL Expression
Name: This is the internal identifier for the filter (e.g., filter1).
Code: drug_info.rating > 9 tells Genie that this filter specifically looks for high-rated items.
Synonyms: Enter phrases like: *"top-rated," "highly recommended," "best drugs,"* or *"rating over 9."* When a user types one of these, Genie knows to trigger the SQL in the Code box.
Instructions: Here, you give the AI "guardrails." For example: *"Only apply this filter when the user is specifically looking for high quality; do not use if they ask for 'all drugs'."*

Genie -> Instructions -> SQL Queries
"What is the average rating broken down by drug category?" - SELECT category, AVG(rating) FROM drug_info GROUP BY 1
Usage Guidance. While the "Question" box tells Genie *when* to use the query, the **Usage Guidance** tells Genie *how* to interpret the results or any limitations it should mention to the user.

Benchmarks -> Add Benchmarks
What is the "Ground Truth"?
When you add a benchmark question, you have the option to provide a **SQL Answer**. This SQL statement is the Ground Truth. When you run the benchmark,
Genie will Generate its own SQL based on the natural language question.
Compare the results of its generated SQL against the results of your **Ground Truth SQL**.
Score the response: If the data matches perfectly, it's marked as **Correct**. If not, it's marked as **Needs Review**.

How Auto Visualization Works
Instead of just being a "dumb" chart builder, the feature uses several AI layers to make that line chart happen:
Intent Reasoning: The AI identifies that "over the last 12 months" implies a **time-series** trend.
Metadata Analysis: It looks at the resulting data columns. Because it sees a date/time column and a numeric value (Revenue), it recognizes these as the X and Y axes.
Heuristic Selection: The system follows a "Best Fit" logic. Since time-series data is best represented by continuity, it automatically chooses a **Line Chart** over a bar chart or pie chart.
Semantic Formatting: It uses the metadata from the **Unity Catalog** to label the axes correctly (e.g., formatting "rev_amt" as "Revenue" and adding currency symbols if defined).

Databricks LLM Intent Workflow

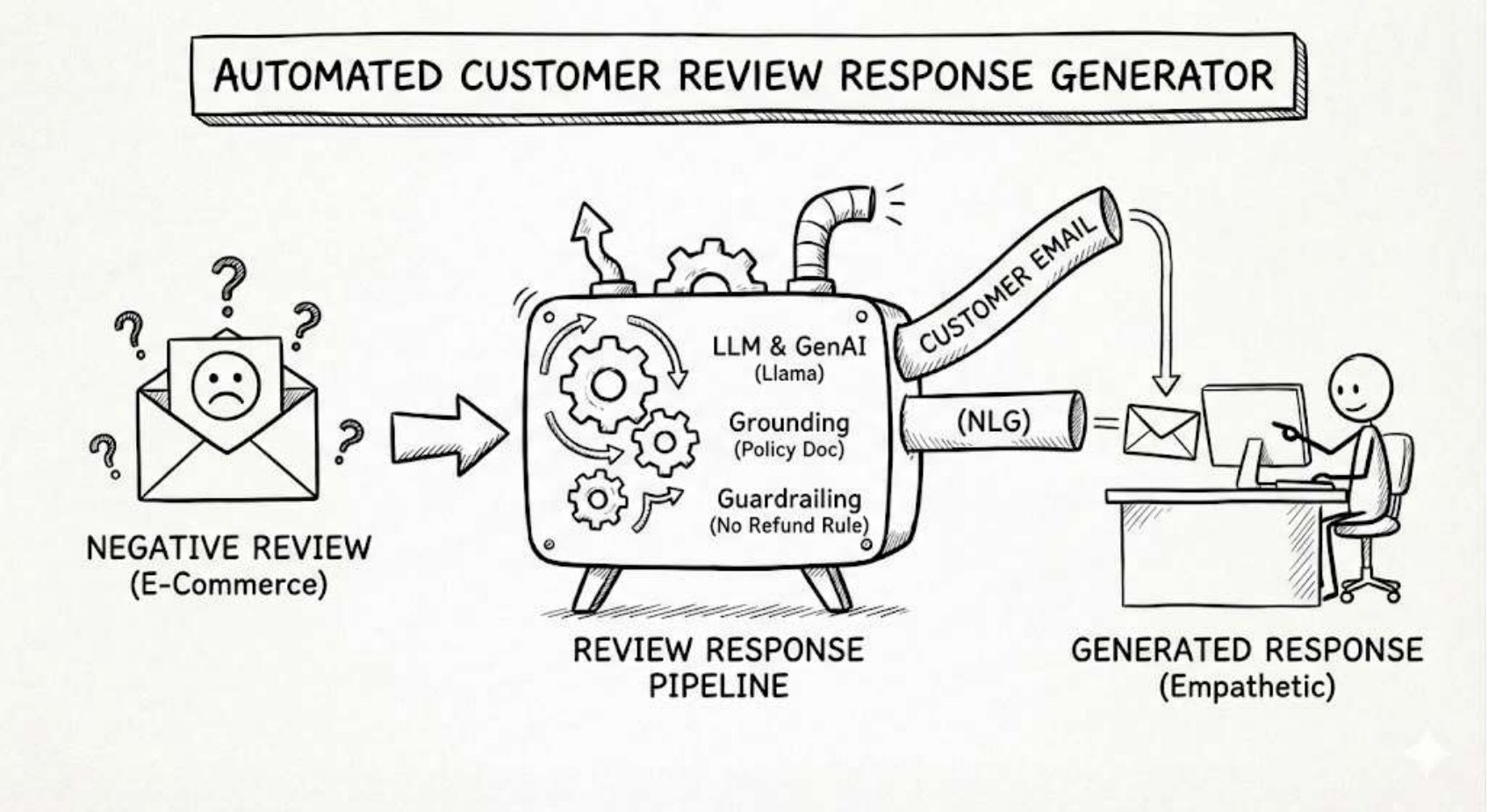


Databricks LLM Workflow

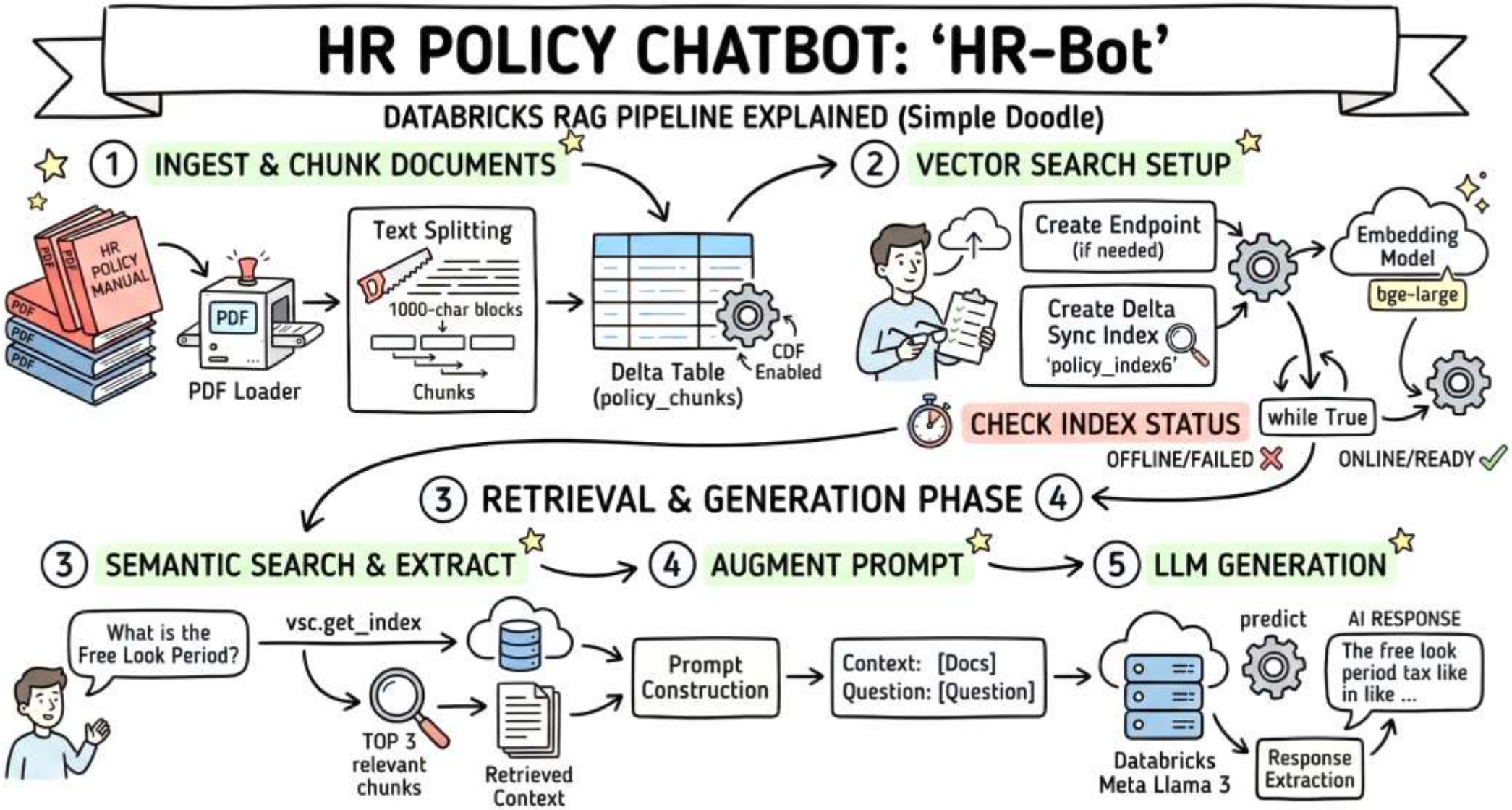
Databricks - Meta LLM Model

Python Program
User/Program
Databricks Platform
1. Token(Password), DB Secret
2. Client Creation (DBToken, API Endpoint)

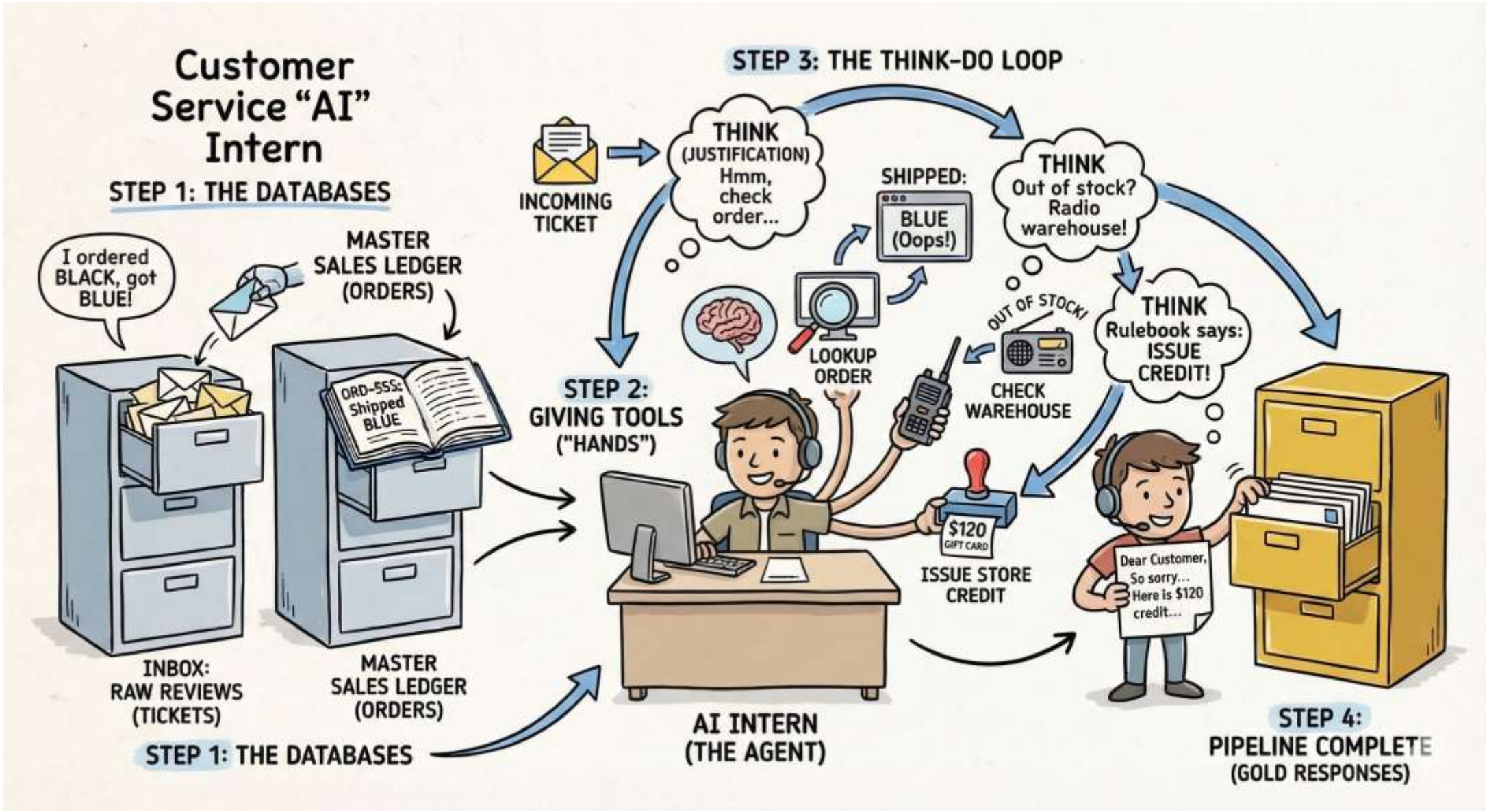
SQL Program (ai_query) –
Databricks manages all these implicitly



Databricks RAG + LLM Workflow



Databricks Agentic Workflow (ReAct Framework)



Databricks Agentic Workflow (Reason & Act Framework – Think -> Decide -> Act -> Observe -> Finish)

Analogy: Autonomous Customer Service Agents

Step 1: Setting up the Office (The Filing Cabinets)
The Inbox: Where new letters from complaining customers arrive (e.g., "You sent the wrong color!").
The Master Ledger: The company's official record book containing the truth about every order (what was bought, what shipped, and the price).

Step 2: Equipping the Intern (Giving them "Hands")
Hand 1 (The Ledger Look-up): The ability to look up a specific order ID in the Master Ledger to see what really happened.
Hand 2 (The Warehouse Radio): The ability to check if a specific item is currently currently sitting on a shelf in the warehouse.
Hand 3 (The Credit Printer): The authority to generate an official store credit code for a specific amount.

Step 3: The "Think-Do" Loop (The Intern at Work)
Start: Intern reads ticket: "I got blue, ordered black!" (Order #555).
Loop 1 (THINK): "I need to investigate. I don't know if they are telling the truth. I must check the Master Ledger for Order #555 to see what we shipped."
Loop 1 (DO): Uses the **Ledger Look-up**. *(Result: The ledger confirms the company mistakenly shipped blue.)*
Loop 2 (THINK): "Okay, it was our fault. I should send them the correct black shoes. But wait, do we even have them right now? I need to check inventory."
Loop 2 (DO): Uses the **Warehouse Radio**. *(Result: Warehouse reports black shoes are out of stock.)*
Loop 3 (THINK): "We are out of stock so I can't send a replacement. The rules state I must issue a refund. The order was $120, so I need to issue store credit for that amount."
Loop 3 (DO): Uses the **Credit Printer**. *(Result: A new credit code is successfully generated.)*
Loop 4 (THINK): "I have investigated fully and taken the necessary actions. I have nothing left to 'do'. Now I will write the final response to the customer explaining the mix-up and providing their credit code."
Loop 4 (DO): Generates the final email text.

Step 4: Finishing the Daily Tasks
Finally, the code takes the whole stack of today's complaints, runs the intern through that "Think-Do" loop for every single ticket autonomously, and files all the finished responses neatly into a "Completed Work" cabinet.

About GenAI

inceptez

📌 What is GenAI?

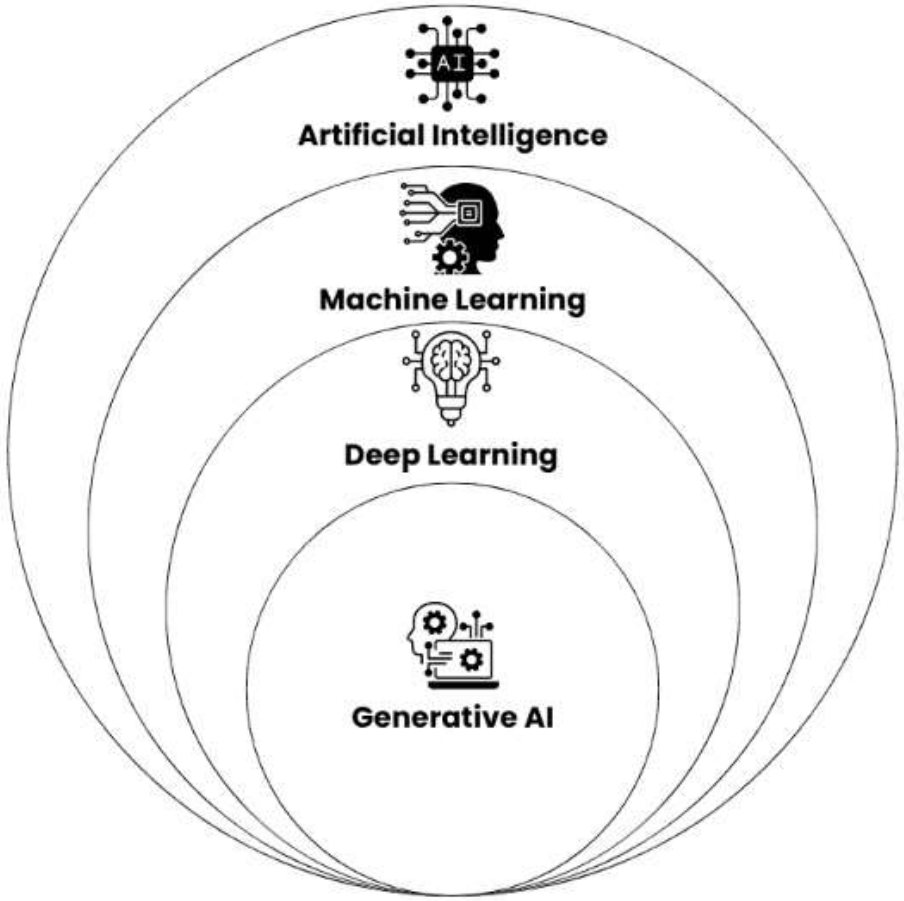
> GenAI creates new content (text, images, code, etc.) by learning from existing data.

🔧 How Does It Work?

- Learns patterns from **huge datasets**
- Uses **models like Transformers**
- Generates content based on **prompts or input**

💡 Examples of GenAI

- **Text:** ChatGPT, Gemini
- **Images:** DALL·E, Midjourney
- **Code:** GitHub Copilot
- **Audio/Video:** Sora, MusicGen



Must Know Gen AI Terms

LLM

Large Language Model; processes and generates ***text*** in a larger context with complex model output

Prompt

Input text instruction to guide the AI

Multi Model AI

AI that process and generates Audio, Video, Image, text/code

GenAI

We discussed already in detail

RAG

Retrieval-Augmented Generation; adds external knowledge

Hallucination

AI generates plausible but false information

Parameters

Used to tune the AI with right input value

SLM

Small Language Model (smart light weight assistant who's great at few things)

Vector

AI generates plausible but false information

Agents

AI system performing tasks autonomously

Agentic AI

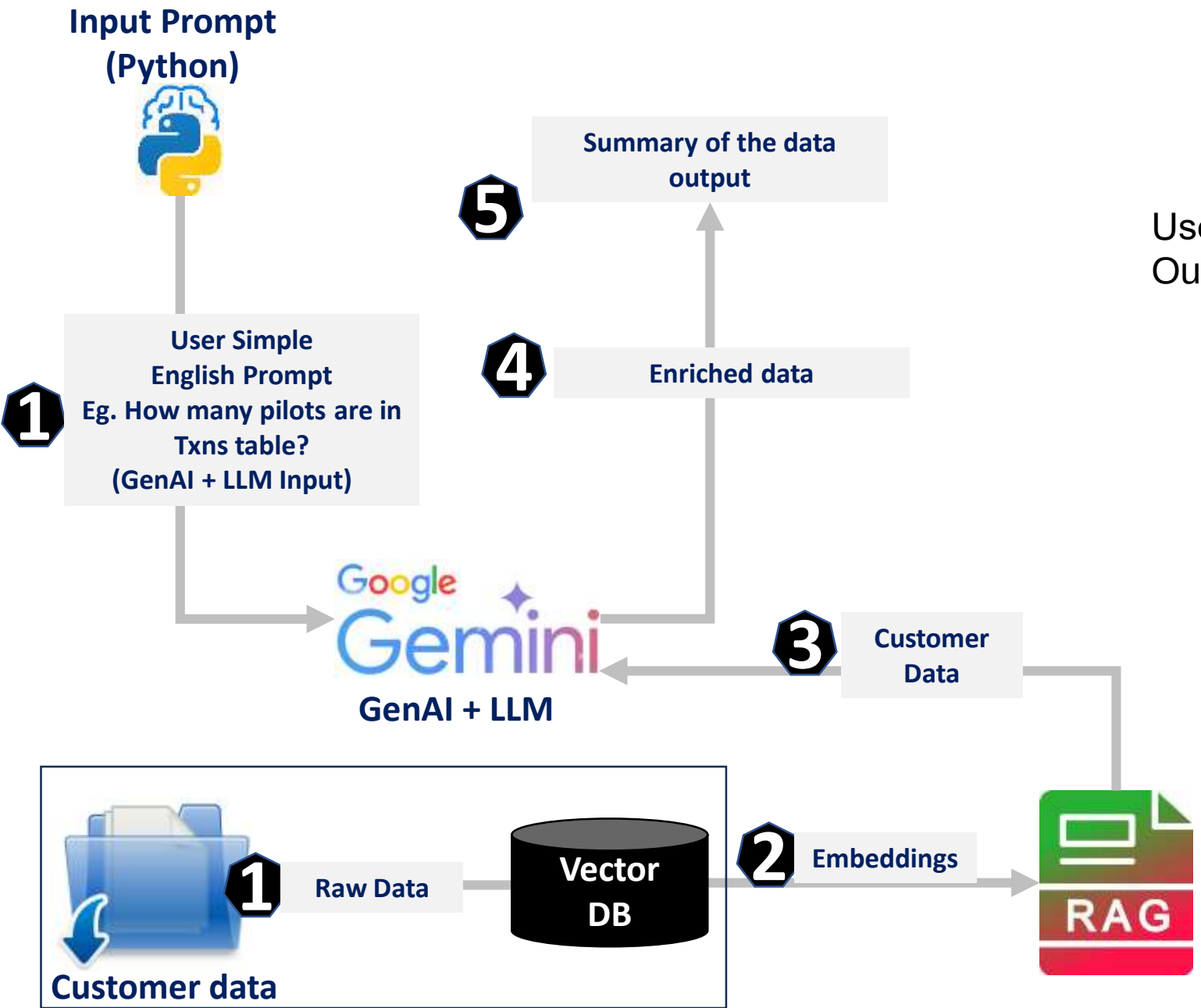
AI system performing tasks autonomously

Eg. thinks, plans, changes strategy

Embeddings

Is the way for AI to turn words and numbers so it can easily understand and work with LLM
Eg. How computer understands 0s and 1s, LLM understands only numbers (embeddings)

GenAI + LLM based Metric Bot – Data Summarizer



User Input → vector + query → LLM →
Output = Gen AI application

Databricks – TempViews

- A Temp View is a virtual table created from a DataFrame.
- It does not store data physically but allows you to query the DataFrame using Spark SQL.
- Two types:
  - Local Temp View – **createOrReplaceTempView** – session-scoped.
  - Global Temp View – **createOrReplaceGlobalTempView** – cluster-scoped (global_temp database prefix).

Feature	Local Temp View	Global Temp View
Scope	Current Spark session only	All Spark sessions in cluster
Lifetime	Until session ends	Until Spark application ends
SQL Reference	view_name	global_temp.view_name
Shareable Across Notebooks	No	Yes

Databricks – Managed vs External Table

- In Databricks Spark SQL, there are two main types of tables
  - **Managed Table**
    - Spark manages both the metadata and the data.
    - Data is stored inside the warehouse directory .
    - Dropping the table that deletes both metadata and data.
  - **External Table**
    - Spark manages only the metadata, not the data.
    - Data is stored in a user-defined location (e.g., DBFS, S_3, ADLS).
    - Dropping the table that deletes only the metadata, keeps the data.

Use Managed Tables for:

- Staging and internal pipeline tables
- Tables where Spark manages lifecycle

Use external tables when:

- Data is shared across systems.
- Data is already present and managed elsewhere.
- Need to preserve data even if the table is dropped.

Feature	Managed	External
Data storage	Inside warehouse directory	User-specified path
Data deletion on DROP	Yes	No
Requires LOCATION	No	Yes
Best for	Internal pipeline data	Shared or existing data
Metadata in metastore	Yes	Yes

Databricks – Regular Views Vs Mat Views

A **View** (also called a *Permanent/Persistent View*) is a **saved SQL query** stored in the **Databricks metastore or Unity Catalog**.

Unlike temporary views, it **persists across sessions, clusters, and users** until you explicitly drop it.

It acts like a **virtual table** — it doesn't store physical data, only the SQL logic used to fetch the data.

Regular (Non-Materialized) View

- Stores only the query definition.
- Data is fetched live from base tables when queried.
- The standard view type in Databricks.
- Always reflects current data from the base tables.
- Best for dynamic data and ad hoc analytics.

Materialized View

- A precomputed and cached view.
- Used for large datasets or frequently accessed aggregations.
- Data is physically stored and automatically refreshed on schedule.
- Improves performance and reduces compute cost.

Scenario	Recommended View Type
Frequently changing base data	Regular View
Static or slowly changing data	Materialized View
Dashboard or reporting performance optimization	Materialized View
Exploratory / ad-hoc analysis	Regular View